

Parameterladdning av inbyggda system

-Programmering av ST7 flashminne med ett Visual
C++ program.

Parameter load system

-Programming of ST7 flash memory with a Visual
C++ program.

Mikael Brolin
Martin Åkesson

Organisation/ Organization VÄXJÖ UNIVERSITET Matematiska och systemtekniska institutionen Växjö University School of Mathematics and Systems Engineering		Författare/Author(s) Mikael Brolin Martin Åkesson
Dokumenttyp/Type of document Examensarbete/ Diplomawork		
Titel och undertitel/Title and subtitle Parameterladdning av inbyggda system. Programmering av ST7 flashminne med ett Visual C++ program. Parameter load system Programming of ST7 flash memory with a Visual C++ program.		
Sammanfattning (på svenska) Vid Micropower pågår ett arbete med utveckling av en ny generation microdatorstyrda batteriladdare. Laddarens styrning består av en microdator utan operatörsinterface. All data ska laddas ner direkt i till microdatorns flashminne via ett parameterladdningssystem. Microdatorn utgörs av en ST7a med tillhörande "flashminne". Denna rapport tar upp kommunikationen mellan PC och ST7 och det gränssnitt som används där emellan. Den behandlar även programmeringen av ST7s flashminne. Det program som används i PCn är ett Windowsprogram, skapat med Microsoft Visual C++.		
Nyckelord ST7 flashminne, Visual C++		
Abstract (in English) Micropower develops a new generation of microcomputer controlled batteri chargers. The chargers consist of a microcomputer without man-machine interface. All data are loaded directly to the microcomputers flashmemory via a parameter loading system. The microcomputer is a ST7 with flashmemory. This rapport discusses communication between PC and ST7 and the interface that is used between them. It also discusses the programming of ST7 flashmemory. The program necessary for this task is developed in Microsoft Visual C++.		
Key Words ST7 flash memory, Visual C++		
Utgivningsår/Year of issue	Språk/Language	Antal sidor/Number of pages
2002	Svenska/Swedish	135
Internet/WWW http://www.ips.vxu.se		

Förord

I högskoleingenjörsutbildningen i Datateknik 120 poäng ingår som ett avslutande moment att göra ett examensarbete på 10 poäng. Det ska helst ske på ett företag, där man får en uppgift att lösa. Syftet med detta är tillämpa teori från föreläsningarna för att få en inblick i hur ett företag löser ett projekt i verkligheten och att lära sig att ta hänsyn till ekonomiska krav och prestandakrav på en produkt.

Tack till:

Björn Sigvardsson

Tomas Sandstedt

Karl Sannesson

Alla tre är medarbetare på Micropower i Växjö där vi har fått möjligheten att utföra vårt examensarbete under våren 2002.

Gösta Bryllert våran handledare på universitetet.

Växjö den 4 juni

Mikael Brolin

Martin Åkesson

Inledning

Bakgrund

Vid Micropower pågår arbete med utveckling av en ny generation mikrodatorstyrda batteriladdare. Laddarens styrningen består av en mikrodator utan operatörsinterface. All data skall laddas ner direkt till mikrodatorns programminne via ett parameterladdningssystem. Mikrodatorn utgörs av en ST7:a med tillhörande "flashminne".

Vid tillverkning av mikrodatorstyrda batteriladdare sker tillägg och förändringar i mikrodatorprogrammets parameterlistor vid ett antal tillfällen. Under kretskortsprovning sker laddning av testprogramparametrar och tex. kalibreringar av mätängångar. Vid leveransanpassningen (innan laddaren levereras till kund) väljs bland annat laddningskurvor och batteristorlek. Vid leveransanpassningen knyts också batteriladdarens märkdriftdata till mikroprogrammets internt normerade beräkningsenheter. Parametrar relaterade till dessa förändringar skall laddas ner till mikrodatorn via ett PC-program i Windowsmiljö.

Uppgift

Förutom att sköta nerladdningen av mikrodatorparametrar skall PC-programmet generera en spår- och sökbar leveransdokumentation. Med spårbar menas att t.ex. program- och hårdvaruversion lätt skall bli tillgängligt med hjälp av t.ex. serienummer.

Det är av största vikt att systemet blir användarvänligt med ett väl genomtänkt operatörsgränssnitt och att självkontrollerande funktioner byggs in. Det innebär bland annat att rimlighetskontroller skall göras på inmatade parametrar.

Innehållsförteckning

1. Systembeskrivning för parameterskrivning/läsning till flash på ST7.....	sida 7
1.1 Skrivning och läsning av följande parametrar.....	sida 7
1.2 Katalog.....	sida 8
1.3 Skrivning och läsning ska kunna göras på två olika sätt.....	sida 8
1.4 Spårbarhet.....	sida 8
1.5 Menyegenskaper.....	sida 8
1.6 Parameterkontroll	sida 8
1.7 Användarfall.....	sida 8
2. Programmering av ST7.....	sida 9
2.1 ISP hårdvaru-beskrivning	sida 9
2.2 Minneskarta.....	sida 9
2.3 Val av ISP mode.....	sida 10
3. Kommunikation.....	sida 12
4. Parallellporten.....	sida 15
4.1 Portadresser.....	sida 15
4.2 Hårdvaruegenskaper hos parallellporten.....	sida 16
4.3 Parallellportsregister.....	sida 17
4.4 Kopplingsprotokoll.....	sida 18
5. Handskakningsprotokoll	sida 19
6. Programmering av Windows-programmet.....	sida 21
6.1 Att skapa programfönster med Visual C++.....	sida 22
6.1.1 Lägg till kontroller.....	sida 23
6.1.2 Koppla kod till dialogrutan.....	sida 23
6.1.3 Skapa en meny.....	sida 23
6.2 Programmets olika delar.....	sida 23
6.2.1 Menyn.....	sida 23
6.2.2 Anpassning.....	sida 23
6.2.3 Produktion.....	sida 24
6.2.4 Tekniker.....	sida 24
6.3 Kommandon till ST7 interface.....	sida 24
6.4 Programkod.....	sida 24
6.4.1 Funktionen OnInitDialog().....	sida 24
6.4.2 Skicka byte.....	sida 26
6.4.3 Ta emot byte.....	sida 27
6.4.4 Knappen ”Skriv 16 Byte”, OnSkriv16byteKnapp().....	sida 28
6.4.5 Knappen ”Läs 16 Byte”, OnLas16byteKnapp().....	sida 29
6.4.6 Knappen ”Skriv”, OnSkrivKnapp().....	sida 29

6.4.7 Knappen "Skriv och Spara loggfil", OnSparaloggfilKnapp().....	sida 30
6.4.8 Knappen "Läs", OnLasKnapp().....	sida 30
6.4.9 Checkboxar i Visa-menyn.....	sida 31
6.4.10 När är en editbox ändrad?.....	sida 31
7. Diskussion.....	sida 32
7.1 Problem.....	sida 32
7.2 Resultat.....	sida 32
Referenslista.....	sida 33
Bilagor.....	sida 34

1. Systembeskrivning för parameterskrivning/läsning till flashminne på ST7

Skapa ett Windowsprogram med Microsoft Visual C++ som med ISP (In System Programmable) kan skriva och läsa i ST7:s flash.

1.1 Skrivning och läsning av följande parametrar

- huvudprogram
- laddningskurva
- serienummer
- hårdvaruversion
- amperetimmar
- antal celler
- grundladdartyp volt
- grundladdartyp ampere
- laddartyp volt
- laddartyp ampere
- kalibreringsdata

Placering av parametrar i flashminne (bild 1).

Adresser till följande parametrar		
Parameter	Start	Slut
Programnamn	E000	E008
Programrevision	E009	E00F
Optionsbyte0	E010	E010
Optionsbyte1	E011	E011
Programstorlek	EBFF	EBFF
Laddningskurvanamn	FC00	FC08
Laddningskurvarevision	FC09	FC0F
Laddningskurvastorlek	FC7F	FC7F
Serienummer	FCC0	FCC0
Hårdvaruversion	FCB0	FCB1
Amperetimmar	FCA2	FCA3
Antal celler	FCA0	FCA1
Laddartyp Volt	FC90	FC91
Laddartyp Ampere	FC92	FC93
Grundladdartyp Volt	FC80	FC81
Grundladdartyp Ampere	FC82	FC83
Kalibreringsdata		

Bild 1. Parameteradresser

1.2 Katalogfil

- Laddningskurvafiler
- Programfiler
- Loggfiler
- Adressfil

Skapa en textfil där dessa filers kataloger är angivna.
Sökning i denna fil vid start av programmet.

1.3 Skrivning och läsning ska kunna göras på två olika sätt

- skriv/läs hela flashminnet
- skriv/läs delar av flashminnet, 16 byte i taget.

1.4 Spårbarhet

- skapa en loggfil
- Använda textprogram för läsning av sparad fil

1.5 Menyegenskaper

- Ruta för meddelande
- Parametervärde som ändras ska markeras
- Startvärde för valt fönsterutseende sparas

1.6 Parameterkontroll

- Rimligt värde
- Antalet tecken
- Vilka tecken som får användas

1.7 Användarfall

- Anpassning
Val av följande parametrar ska kunna skrivas och läsas.
 - Laddningskurva
 - Antal celler
 - Amperetimmar
 - LaddartypLoggfil sparas automatiskt
- Produktion
Alla parametrar ska kunna skrivas och läsas.
Val om loggfil ska sparas eller inte
- Tekniker
Alla parametrar ska kunna skrivas och läsas.
Läsa eller skriva 16 byte från en viss adress
Val om loggfil ska sparas eller inte

2. Programmering av ST7

Programmering av ST7 <B5> kan ske på flera sätt, men det sätt vi ska använda oss av är ISP mode. Det innebär att programmeringen av flashminnet ska ske när kretsen sitter på plats på ett kretskort, i vårt fall i en batteriladdare. ISP seriekommunikation baseras på master/slave arkitektur där slave är den ST7 som ska programmeras.

2.1 ISP hårdvaru-beskrivning

ST7 behöver matningsspänning (V_{DD}) (bild 2) och jord (V_{SS}) och en intern eller extern klocksignal. ISP mode behöver 5 stycken signaltrådar. Dessa är:

- RESET (aktivt låg)
- V_{SS} Jord ST7
- ISPCLK : ISP serieklocka utgång
- ISPDATA : ISP serie data ingång
- ISPSEL : Val av ISP mode

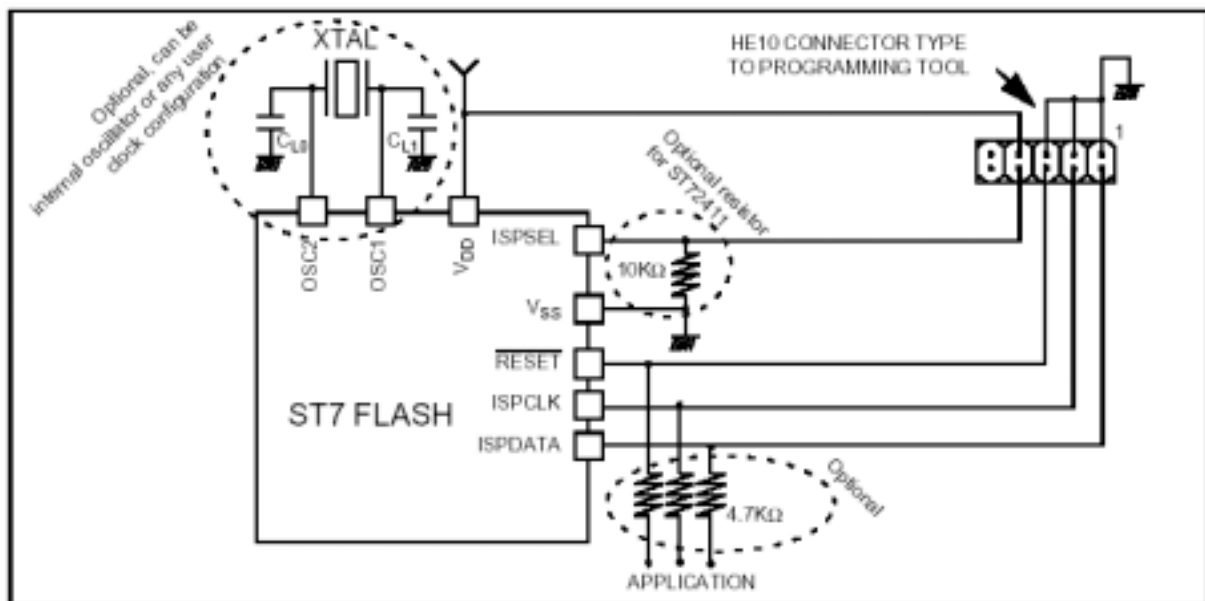


Bild 2. Kopplingsschema för ISP mode. <R1, sida 2>

2.2 Minneskarta

Vi ska programmera flashminnet på ST7, vilket är lika med minnesarea (Program Memory och Interrupt & Reset Vectors) E000h till FFDFh + FFE0 till FFFFh (h = hexadecimalt) (bild 3). Det är totalt 8 Kbyte som kan utnyttjas för programmering. ST7 kräver även ett 'bootstrap'program för att kunna programmera flashminnet. Detta program används för att ta hand om inläsningen av vårt huvudprogram.

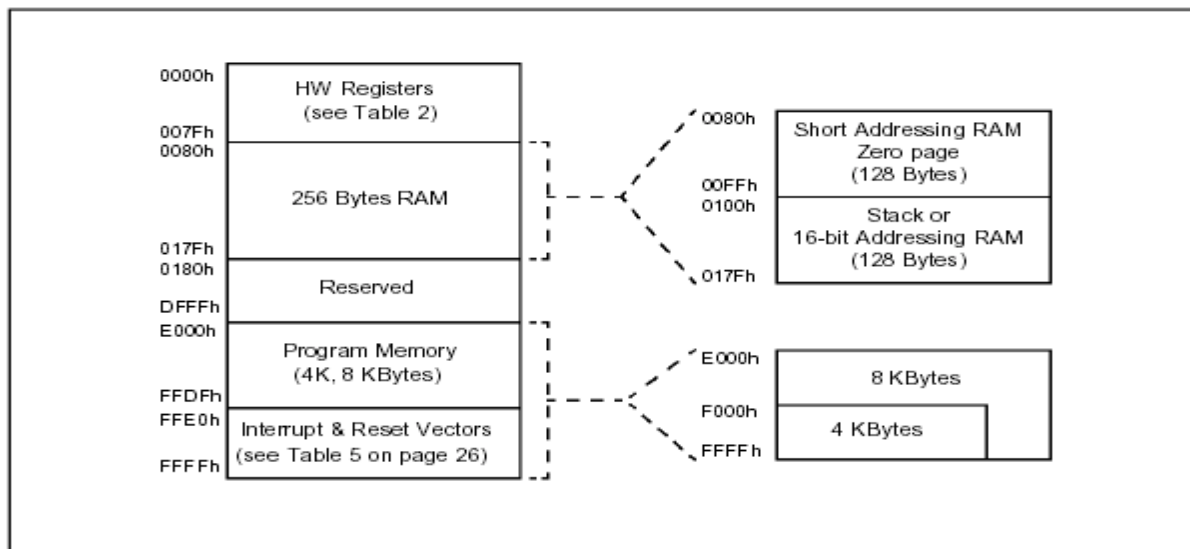


Bild 3. Minneskarta över ST7ans minnesarea. <R2,sida 10>

2.3 Val av ISP mode

ISP utförs i tre steg.

- Val av ISP mode.
- Ladda ner kod i RAM.
- Exekvering av den nerladdade koden för att programmera användarprogrammet i flashen.

För programmering av ST7 med hjälp av 'bootstrap' program används ett flödesschema enligt bild 4. Beroende på hur 'Bootstrap' programmet som ligger i RAM-minnet ser ut kan flashminnesprogrammeringen skräddarsys (antal byte, adress på programmet, programmering av optionbyte).

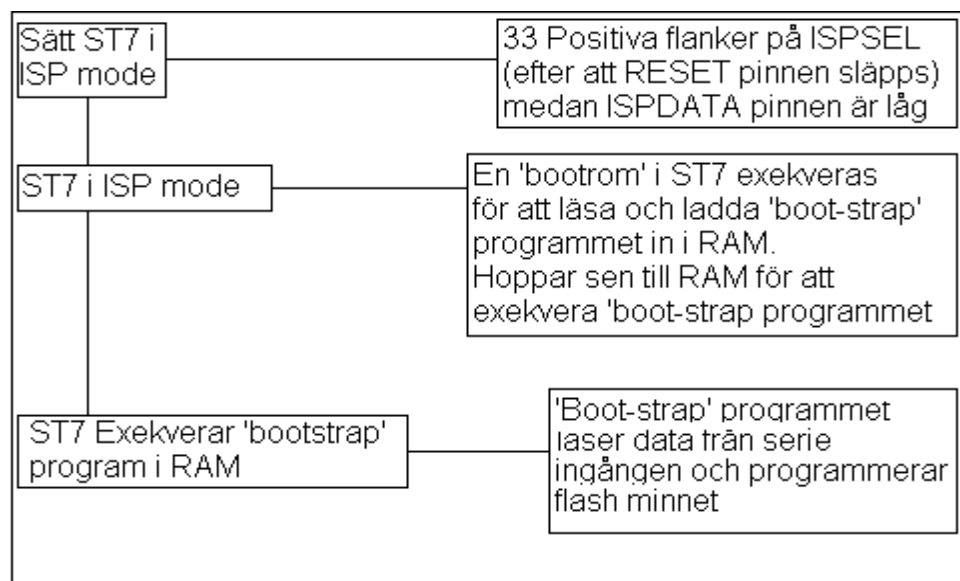


Bild 4. ISP protokoll-flödesschema. <R1, sida 3>¹

¹ Egen översättning av texten i bilden.

För att sätta ST7:an i ISP mode behöver man lägga ut följande sekvenser på de olika signalpinnarna (bild 5).

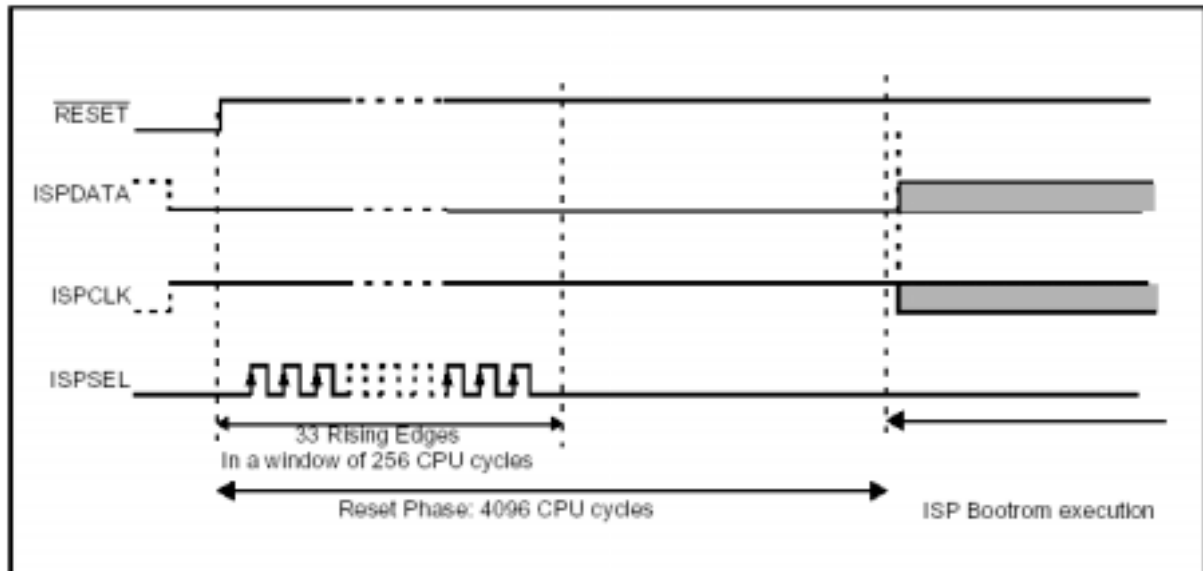


Bild 5. Val av ISP mode. <R1, sida 4>

3. Kommunikation

Det fanns inga krav på vilken port av serie eller parallellporten som vi skulle använda oss av. Därför valde vi att börja undersöka serieporten. Denna har vi använt oss av vid ett flertal andra tillämpningar, så därför kändes det naturligt att undersöka om det skulle vara möjligt att även vid detta tillfälle utnyttja serieporten.

Började med att programmera enkla testprogram för serieporten. I Microsoft Visual C++ finns det färdiga funktioner som man kan anropa för att öppna en serieport. Man anger vilken port som ska användas (com1 eller com2), vilken hastighet man vill sända med (Baudrate = bit/s), om paritetsbitar ska användas eller ej. Man ska även ange antalet stoppbitar. Det är dessa variabler man minst bör ange för att kunna kommunicera. För att testa vad vi skickar ut på serieporten använder vi oss av hyperterminalen. I hyperterminalen anger man samma värden på uppkopplingen till serieporten som man har angett i testprogrammet (förutom att om man valt com1 i testprogrammet blir det com2 i hyperterminalen). En RS232 kabel kopplas mellan com1 och com2. Inga problem att skicka data mellan PC och hyperterminal. Skickar enstaka bytes eller hela filer och får upp detta i hyperterminalen.

På Micropower finns det en utrustning med tillhörande program som för närvarande används vid programmering av microprocessorn ST7. Men det är denna del som vi ska ersätta med vårt Visual C++ program. Programmet heter ST7 ISP – v.205 och det ingår även en hårdvara (allmänt kallad dong på Micropower) som tar hand om interfacet mellan PC och ST7. Inköpt på <http://www.kanda.com>.

Vi började med att undersöka hur ”dongen” programmerar ST7, genom att mäta upp vilka signaler som skickas mellan ”dongen” och ST7. Det visade sig i stort sett omöjligt med hjälp av ett vanligt oscilloskop. Mailade till <http://www.kanda.com> för att försöka få denna information, men de ville ej ge oss denna information.

Vi hittade beskrivning på ett interface mellan PC och ST7an <R3, sida 26 - 27>, men den gällde parallellporten, så vi bestämde oss för att gå vidare på denna väg istället. När man programmerar en ST7 kan man inte koppla parallellporten på PC:n direkt till pinnarna på ST7, pga att det behövs ett handskakningsprotokoll för datakommunikationen och för att möta vissa krav på hastigheten.

Vi gjorde om ritningen vi hittat <R3, sida 27>. På den togs klockan (kristallen) bort för en intern klocka på ST7:an används istället. Tog även bort ST662A med tillhörande passiva komponenter pga att uppgiften för denna komponent är att höja matningsspänningen från 5V till 12V, vilket vi inte hade något behov av efter att ha mätt upp nivåerna på signalerna som ”dongen” skickar.

Kretsschemat och layouten <B1> skapades i Edwin2000, ett CAD-program som finns på universitetet, där vi även tillverkade vårt mänsterkort.

Väl färdiga med kortet började vi att mäta upp parallellporten för att se att den klarade av att skicka data i den takt vi behövde. Det visade sig att den var för långsam för att klara kraven på resetsekvensen. Vi borde mätt upp detta innan vi tillverkade kretskortet, men antog att PC:n skulle klara av detta. Krav enligt tabell och även de verkliga tiderna för vår parallellport framgår av bild 6.

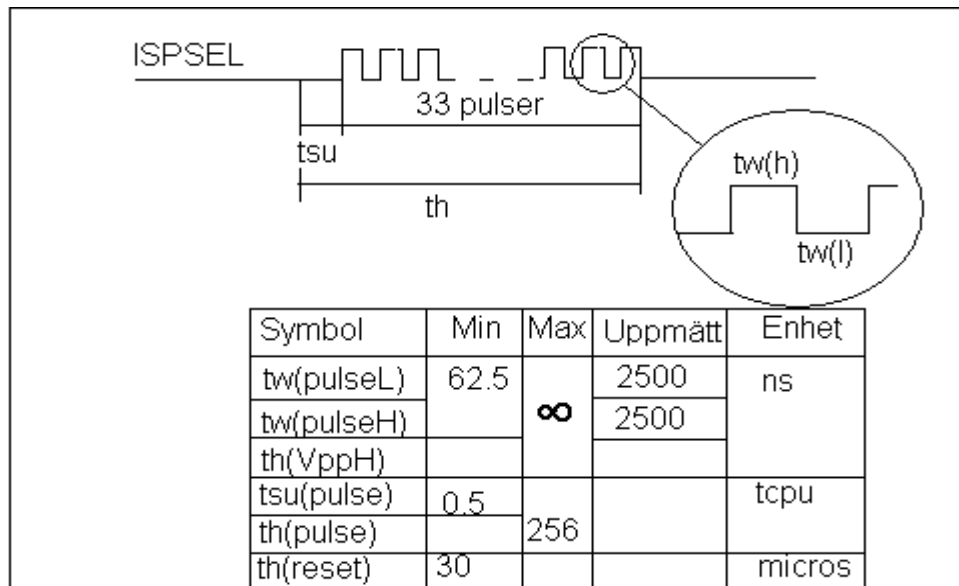


Bild 6. Reset-pulser och tidskrav för signalöverföring <R3, sida 10>²

Frekvens_{cpu} = 2 MHz (processorhastigheten på batteriladdarens ST7a).

$t_{cpu} = 0.5 \mu s$

$t_{su} = 0.5 * 0.5 \mu s = 0.25 \mu s$.

$tw_{max} = (0.5 * 256) / ((33 * 2) + 1 (t_{su})) = 0.84 \mu s$.

$t_h = 0.5 \mu s * 256 = 0.128 ms$ (t_h = maxtiden för resetpulserna är $256 t_{cpu}$)

t_h (uppmätt) = $2500 ns * ((33*2) + 1) = 0.170 ms$

De två sista beräkningarna visar att parallellporten är för långsam. Maxtiden var 0.128 ms medan vår parallellport krävde 0.170 ms för att kunna skicka ut denna resetsekvens, vilket är ca. 30 % för mycket.

Det är även osäkert att programmera i windows eftersom det kan finnas flera processer igång samtidigt som ska dela på processorn. Detta innebär att det är svårt att bestämma exakta tider för våra pulser.

Vi valde då ännu en ny väg att gå, där ytterligare en ST7:a används för att programmera ST7an på batteriladdaren (bild 7).

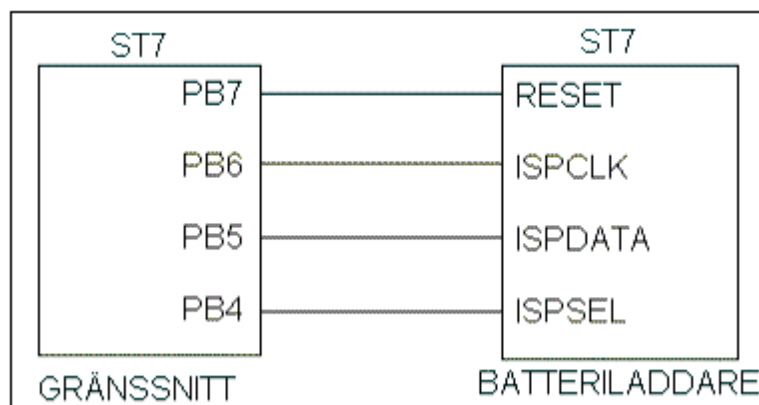


Bild 7. Gränssnittshantering <R2, sida 13>³.

² Del av bild.

I den ena ST7an ska själva gränssnittet programmeras. Här är det lätt att få exakta tider på våra pulser, eftersom ST7an har en egen klocka.
Schemaritning och layout gjorde vi även denna gång på universitetet <B2>, där även tillverkningen av det nya kortet gjordes (bild 8).

Programmering av gränssnittet som ska ligga i ST7:an mellan PC och batteriladdarens ST7a ska Micropower göra utifrån ett definierat handskakningsprotokoll (se avsnitt om Handskakningsprotokoll).

Detta innebär för vår del att vi får skapa ytterligare ett windowsprogram (bild 9) som simulerar denna del, så att vi kan testa vårt huvudprogram innan Micropower hunnit göra detta åt oss.

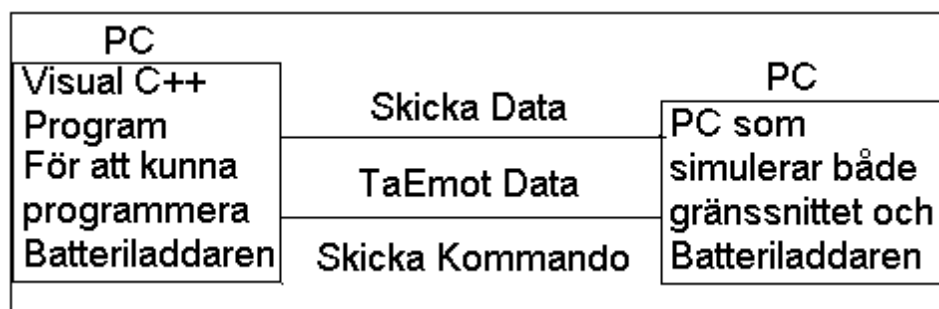


Bild 9. Det simulerade systemet.

Gränssnittet ska alltså programmeras i en ST7 (bild 10).

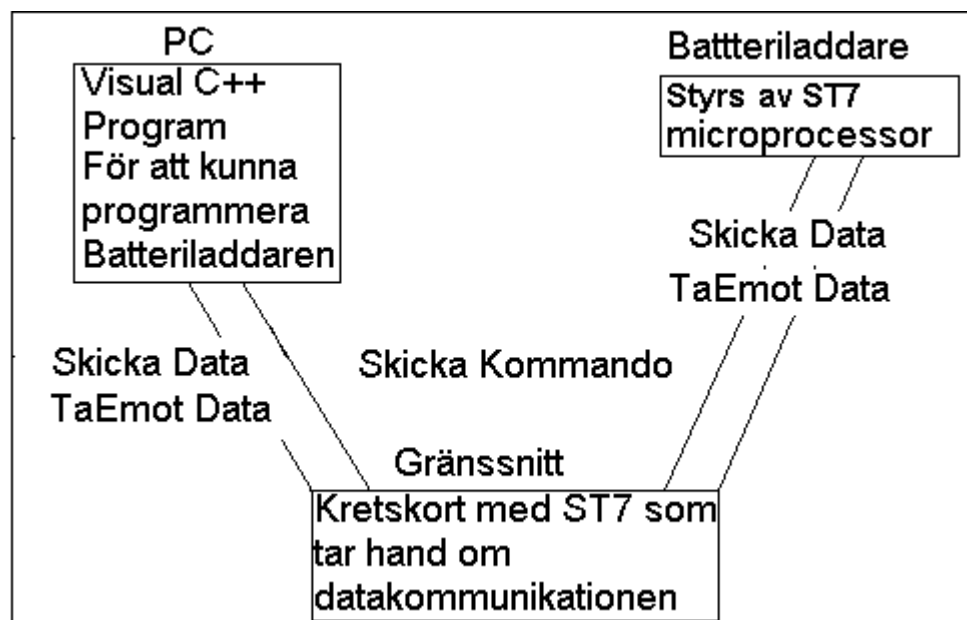


Bild 10. Systembeskrivning

³ Egen översättning av text.

4. Parallellporten

Parallellporten är den mest använda porten som gränssnitt mellan PC och mindre projekt. Denna port tillåter indata på upp till 9 pinnar eller utdata på upp till 12 pinnar vid vilken tidpunkt som helst. Det krävs minimalt med externa kretsar för att klara av många enkla uppgifter. Porten består av 4 kontrollinjer, 5 status linjer och 8 datalinjer.

Utgången på PC är en 25 pinnars honkontakt.

Parallellporten är standardiserad under IEEE 1284. Denna standard definierar 5 operationsmoder <R4>, vilka är följande.

1. SPP mode.
2. Nibble mode.
3. Byte mode.
4. EPP mode.
5. ECP mode.

I mode nr 1 så kan man endast sända data, medan man i mode 2 eller 3 även kan ta emot data. Mode 4 och 5 använder ytterligare hårdvara för att klara av handskakningen, vilket leder till att färre instruktioner används för handskakningen. Hastigheten ökas på detta sätt. Vi har valt att använda ECP mode pga av att vi vill skicka data i båda riktningarna. Vi har även valt att använda samma beteckningar på porten som vid SPP mode, istället för de som förekommer vid ECP mode.

4.1 Portadresser

Parallellporten har tre normalt använda adresser (bild 11). Basadress 3BCh användes ursprungligen för parallellporten på tidiga videokort. Denna adress har sedan försvunnit, när parallellporten togs bort från videokorten. Men har nu börjat användas som tillval för parallellportar integrerade på moderkort.

LPT1 använder normalt basadressen 378h, medan LPT2 använder 278h. Men detta är inte alltid fallet även om det är det normala (bokstaven 'h' indikerar att det är en hexadecimal adress).

Adress	Förklaringar
3BCh - 3BFh	Basadress 3BCh användes ursprungligen för parallellporten på tidiga videokort.
378h - 37Fh	Normala Adressen för LPT 1
278h - 27Fh	Normala Adressen för LPT 2

Bild 11. Tabell för basadresser <R4>

När en dator startas, kommer BIOS (Basic Input/Output system) att bestämma hur många portar det finns och namnge dessa till LPT1, LPT2 och LPT3. BIOS tittar först på adressen 3BCh. Om en parallellport finns där får den namnet LPT1, sen söker den på 378h. Om den hittar en parallellport så får den nästa lediga basadress, vilket blir LPT1 om det inte fanns någon port på 3BCh eller LPT2 om det fanns en port på 3BCh. Till sist kontrolleras 278h på samma sätt. Därför kan LPT2 hamna på 378h och inte 278h som man förväntar sig. LPT står för Line PrinTer Devices.

4.2 Hårdvaru-egenskaper hos parallellporten

Nedan följer en tabell över namn och egenskaper för parallellportens pinnar.
Bild 12 använder "n" för att markera att det är en aktivt låg signal.

Pin Nr (D-Type 25)	Pin Nr	SPP Signal (ECP Signal)	Riktning In/ut	Register	Hårdvaru Inverterad
1	1	nStrobe	In/Ut	Control	Ja
2	2	Data 0	Ut	Data	
3	3	Data 1	Ut	Data	
4	4	Data 2	Ut	Data	
5	5	Data 3	Ut	Data	
6	6	Data 4	Ut	Data	
7	7	Data 5	Ut	Data	
8	8	Data 6	Ut	Data	
9	9	Data 7	Ut	Data	
10	10	nAck	In	Status	
11	11	Busy	In	Status	Ja
12	12	Paper-Out / Paper-End	In	Status	
13	13	Select	In	Status	
14	14	nAuto-Linefeed	In/ Ut	Control	Ja
15	32	nError / nFault	In	Status	
16	31	nInitialize	In/ Ut	Control	
17	36	nSelect-Printer / nSelect-In	In/ Ut	Control	Ja
18 - 25	19-30	Ground	Gnd		

Bild 12. Benämning av pinnarna på en D-Type 25-pinnars parallellports anslutning.

4.3 Parallellportsregister

Offset	Namn	Läs/Skriv	Bit Nr.	Egenskaper
BasAdress + 0	Data Register	Läs/Skriv	Bit 7	Data 7
			Bit 6	Data 6
			Bit 5	Data 5
			Bit 4	Data 4
			Bit 3	Data 3
			Bit 2	Data 2
			Bit 1	Data 1
			Bit 0	Data 0

Bild 13.Tabell Dataregister <R4>.

Om parallellporten är Bi-Directional så kan man både läsa och skriva i dataregistret (bild 13), vilket är en möjlighet om man sätter parallellporten i ECP mode. Basadressen, vanligtvis kallad Dataporten eller Dataregistret används för att lägga ut data på parallellportens pinnar (pin 2- 9). Normalt sett används detta register endast för att skriva till.

Offset	Namn	Läs/Skriv	Bit Nr.	Egenskaper
BasAdress + 1	Status Register	Läs Enbart	Bit 7	Busy
			Bit 6	Ack
			Bit 5	Paper Out
			Bit 4	Select In
			Bit 3	Error
			Bit 2	IRQ (Not)
			Bit 1	Reserved
			Bit 0	Reserved

Bild 14.Tabell Statusregister <R4>.

Statusregistret (basAdress + 1) används enbart för att läsa (bild 14). All data som skrivs till detta register kommer att ignoreras. Statusregistret består av 5 in-bitar, en IRQ statusbit och två reserverade bitar.

Notera att bit 7 (Busy) är aktivt låg. Med detta menas att om man läser av +5V, så innebär detta en logisk '0'. Detsamma gäller för bit 2 (nIRQ). Om denna bit visar en '1', så har inget avbrott skett.

Offset	Namn	Läs/Skriv	Bit Nr.	Egenskaper
BasAdress + 2	Kontroll Register	Läs/Skriv	Bit 7	Inte använd
			Bit 6	Inte använd
			Bit 5	Enable Bi-Directional Port
			Bit 4	Enable IRQ Via Ack Line
			Bit 3	Select Printer
			Bit 2	Initialize Printer (Reset)
			Bit 1	Auto Linefeed
			Bit 0	Strobe

Bild 15. Tabell Kontrollregister <R4>.

Kontrollregistret (BasAdress + 2) var avsett för att enbart kunna skrivas till (bild 15). Men det finns en möjlighet att läsa av bit 0-3. Men den möjligheten använder vi inte.

Man sätter då kontrollregistret till xxxx0100 för att kunna läsa data. Då ligger det normalt +5V på pinnarna och man sänker dessa till 0V för logisk '0'. Det kan även krävas att man lägger en extern resistor på utgångarna som kan vara av typen open collector.

Bit 4 och 5 är interna kontroller. Bit 4 är för att göra IRQ enable och bit 5 (bi-directional) är för att kunna använda dataregistret till att både läsa från och skriva till. När bit 5 sätts till '1', sätts pinnarna 2- 9 i ett högimpedanstillstånd. När man är i detta läge kan man skicka in data på dessa pinnar och läsa av det i Dataregistret. Om man själv skriver data till datapinnarna samtidigt som man har satt dataregistret till bi-directional sparas data men blir inte tillgängligt på datapinnarna. För att slå av läsmöjligheten sätter man bit 5 till '0' igen.

4.4 Kopplingschema

För att kunna kommunicera mellan PC och ST7 så använder vi PC:ns parallellport där vi kopplar den till parallellporten på interfacet enligt följande figurer (bild 16). Beteckningar (bild 15).

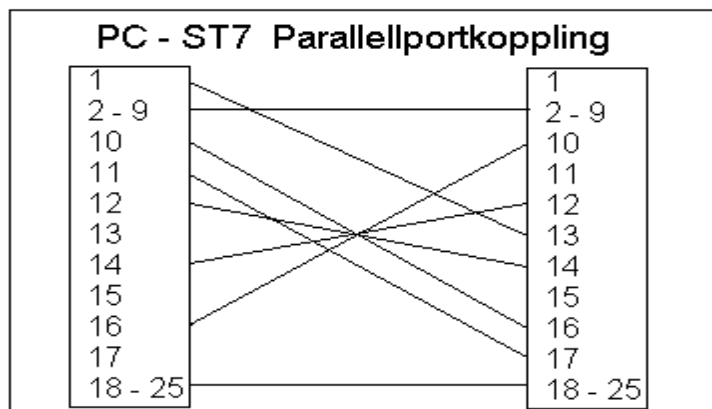


Bild 16. Kopplingsschema

5. Handskakningsprotokoll

Vi har valt att använda oss av följande handskakningsprotokoll för att överföra data, kommando eller adresser på ett säkert sätt <R5>.

När vi skickar data (bild 17) från PC till ST7 så är det följande pinnar som sätts resp. läses av. Detta protokoll används även vid skickande av adresserna.

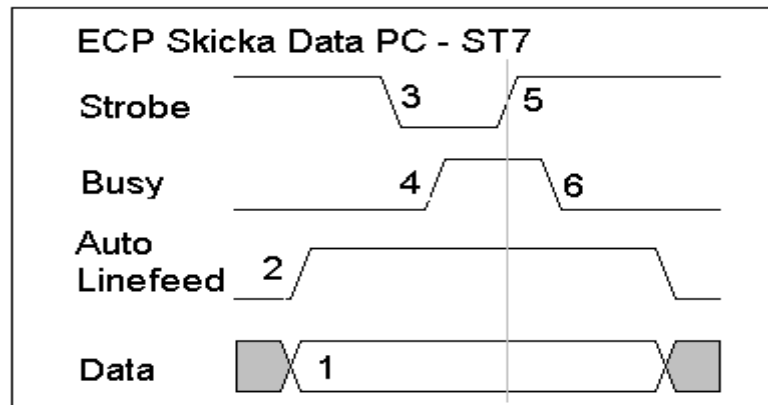


Bild 17. Prototokoll över skicka data <R5>.⁴

ECP Skicka Data PC – ST7.

1. Data placeras på datakanalerna (D7 – D0).
2. PC indikerar att det är data som skickas genom att sätta AutoLinefeed hög.
3. PC indikerar att det är giltig data som ligger på D7 – D0 genom att sätta Strobe låg.
4. ST7 bekräftar giltig data på D7 – D0 genom att sätta Busy hög.
5. PC sätter Strobe hög för att ST7 ska skifta in data.
6. ST7 har läst in data och bekräftar detta genom att sätta Busy låg.

När vi skickar ett kommando (bild 18) är det följande protokoll som används. Det skiljer sig enbart på AutoLinefeed-signalen jämfört med att skicka data.

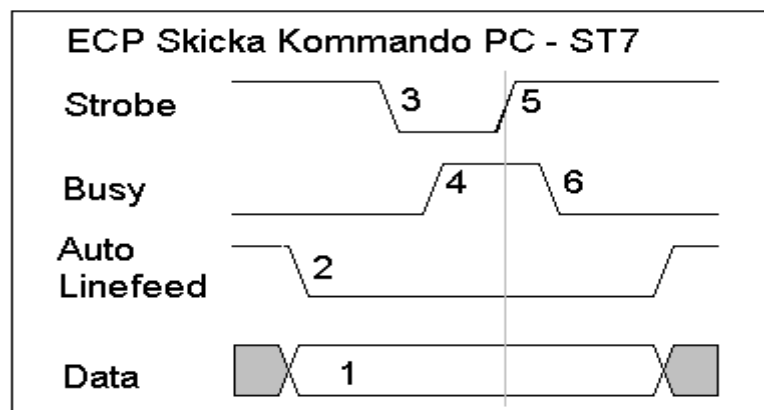


Bild 18. Prototokoll över skicka kommando <R5>.⁴

⁴ Signalnamn ändrade från ECP till SPP

ECP Skicka Kommando PC – ST7.

1. Data placeras på datakanalerna (D7 – D0).
2. PC indikerar att det är ett kommando som skickas genom att sätta AutoLinefeed låg.
3. PC indikerar att det är giltig kommando som ligger på D7 – D0 genom att sätta Strobe låg.
4. ST7 bekräftar giltig data på D7 – D0 genom att sätta Busy hög.
5. PC sätter Strobe hög för att ST7 ska skifta in kommandot.
6. ST7 har läst in ett kommando och bekräftar detta genom att sätta Busy låg.

Till sist har vi protokollet för att ta emot data (bild 19) från ST7 till PC.

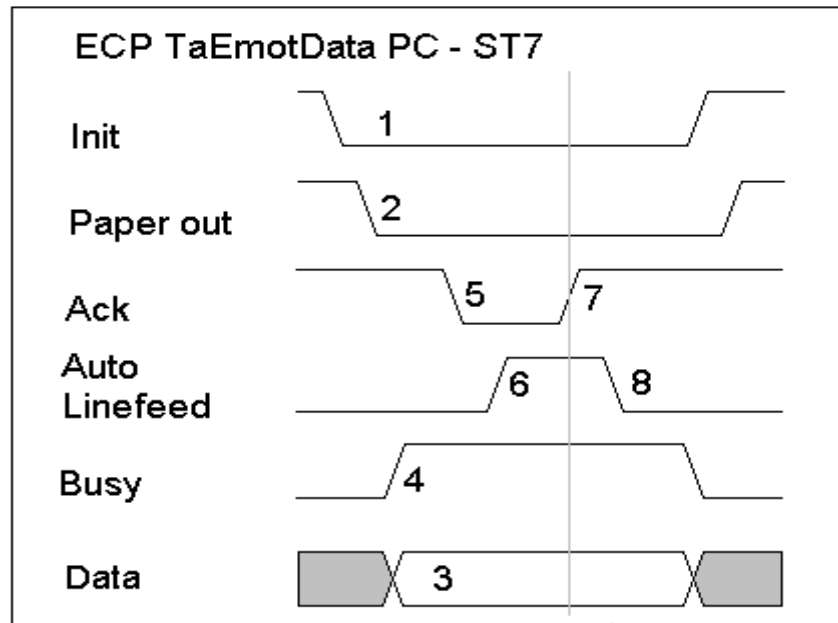


Bild 19. Prototokoll över skicka kommando <R5>.⁵

ECP TaEmotData PC – ST7.

1. PC indikerar att den vill ta emot data genom att sätta Init låg.
2. ST7 bekräftar att den ska skicka data till ST7 genom att sätta Paper out.
3. ST7 placerar data på D7 – D0.
4. ST7 indikerar att det är data genom att sätta AutoLinefeed hög.
5. ST7 indikerar att det är rätt data som ligger på D7 – D0 genom att sätta Ack låg.
6. PC bekräftar rätt data genom att sätta AutoLinefeed hög.
7. ST7 sätter Ack hög för att PC ska skifta in data.
8. PC har tagit emot data och bekräftar detta genom att sätta AutoLinefeed låg.

⁵ Signalnamn ändrade från ECP till SPP

6. Programmering av Windows-programmet.

6.1 Att skapa programfönstret med Visual C++

I Visual C++ finns det mängder med verktyg för att konstruera program. När ett nytt projekt ska skapas används guiden Application Wizard för att bygga ett programskal. Det startas genom att under file-menyn välja new och där skapa ett projekt av typen MFC AppWizard. Man ger det ett lämpligt namn, i vårt fall ST7Laddning. Då skapas en projekt-katalog och sen startar guiden AppWizard. Denna guide innehåller ett antal frågor man ska besvara. De är till för att rätt typ av program ska skapas och att det innehåller de funktioner som behövs.

Programmet som skapats är en grundkonstruktion som är färdigt att kompilera och köra direkt. Det program vi har gjort är baserat på dialogrutor, därför ska det alternativet markeras i "Step 1". I "Step 2" ska det anges vilka funktioner som ska användas i programmet. Här kan man till exempel ange om man vill använda ActiveX-kontroller. I det tredje steget, "Step 3" ska inget ändras. I "Step 4", det sista steget i guiden visas de C++ klasser som kommer att skapas. Här går det att ändra namn på dem, sen avslutas guiden när man trycker på finish. När programskalet har skapats visas de vanliga programfönstren i Microsoft Visual C++.

Rutan till vänster är projektfönstret "Workspace". Det innehåller tre flikar:

Class View, som visar källkoden baserat på C++ klasser,

Resource View, där man kan ändra i programmets olika dialogrutor, ikoner och menyer samt

File View, som visar de filer som programmet är uppbyggt av.

Det program som har skapats ligger till höger, i redigeringsfönstret "Editor" och innehåller nu en dialogruta med en text och två knappar, OK och Cancel, som om man klickar på dem båda avslutar programmet när det körs. När programmet är klart ska det se ut som bild 20.

LaddningST7

Arkiv Visa Hjälp

Anpassning

Laddningskurva: Laddningskurva.s19 Namn: LK-3 Revision: 05.28

Antal celler: 6

Amperetimmer: 12 Laddartyp: 12 Volt: 6 Ampere

Produktion

Program: Program.s19 Namn: Styrprg. Revision: 12.34

Hårdvaruversion: 2 Serienummer: 02051234

Kalibreringsdata: 3 Grundladdartyp: 15 Volt: 10 Ampere

Tekniker

Adress: E000

Data 0-7: 00 02 80 00 00 64 00 20

Data 8-15: 05 04 00 20 00 44 FF 01

Läs 16 Byte Skriv 16 Byte

Meddelande

Skrivning lyckades!

Läs Skriv Skriv och Spara loggfil

Bild 20. Programfönster.

6.1.1 Lägg till kontroller

För att programmet ska se ut som vi vill ha det tar vi bort texten och de knappar som finns från början. Det görs enkelt genom att markera dem och trycka på delete. För att lägga till egna kontroller använder man de inbyggda standardkontrollerna som finns på toolbar controls (bild 21). De kontroller vi använder oss av är: Statisk text, redigeringsrutor (editbox), kommandoknappar, listrutor (combobox) och grupp-rutor.

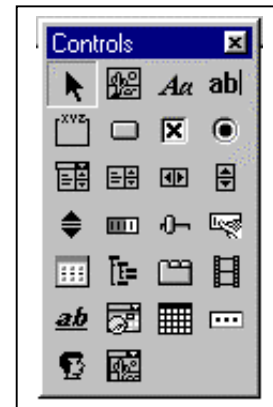


Bild 21. Toolbar controls.

Vi använder tre grupp-rutor. Genom att högerklicka på dem och välja properties kan man ange den text som ska stå på dem. Vi ger dem namnen Anpassning, Produktion och Tekniker. Förutom namn anger man också ett ID till varje grupp-ruta. MFC-programmens kod innehåller ett antal makron för de funktioner som är kopplade till de olika programkontrollernas ID och händelser. I grupp-rutorna placeras sen editboxar och comboboxar. Knapparna placeras sen ut. Till alla kontroller ges ett ID med hjälp av properties.

6.1.2 Koppla kod till dialogrutan

Med hjälp av guiden Class Wizard kopplas sen kod till dialogrutan. Till editboxarna kopplas variabler av typen CString och till comboboxarna kopplas en variabel av typen CComboBox. För att det ska hända något när man trycker på de olika knapparna när programmet körs måste man koppla funktioner till knapparna. Det görs också med Class Wizard. Till de olika kontrollerna finns olika så kallade "messages" beroende på vilken typ av kontroll det är. Till knappen "Läs" gör vi exempelvis en funktion som kallas OnLasKnapp(). Till denna funktion har vi valt att använda ett "message" som kallas BN_CLICKED, som innebär att funktionen anropas när knappen klickas. Vi gör sen samma sak med de övriga knapparna. Men för att det ska hända något när man trycker på knapparna måste man lägga till kod för de funktioner man nu har skapat.

Till editboxar finns ett "message" som heter ON_CHANGE. Skapar en funktion med detta message till alla editboxar för att kunna kontrollera när något ändras i en ruta. Till comboboxarna finns inte detta message, utan där används istället ett message till funktion som märker när en combobox stängs.

6.1.3 Skapa en meny

Menyer definieras som resurser och utformas därför via fliken Resource View i projektfönstret. För att skapa en meny måste man lägga till en mapp för menyer i resursträdet, vilket man gör genom att högerklicka på trädet och välja insert menu. Där kan man skapa meny-rubriker, som inte behöver ha något ID, utan bara en text som syns för användaren, eftersom de är pop-up menyer och inte startar någon funktion. De meny-rubriker vi skapat är arkiv, visa och hjälp. Under dessa läggs sen menyalternativ som man ger ID. Dessa ID kopplas sen till funktioner precis som knapparna kopplades till funktioner. När menyn är gjord ska den kopplas till programfönstret. Det gör man genom att välja properties för själva programfönstret och där välja menyn i listrutan Menu.

6.2 Programmetts olika delar

6.2.1 Menyn

Arkiv-menyn

I Arkiv-menyn (bild 22) kan man läsa och skriva till flashen. Där går också att avsluta programmet.

Visa-menyn

Visa-menyn (bild 23) används till att välja vad i programmet man vill ska visas. Där finns default-inställningar för anpassning, produktion och tekniker.

Hjälp-menyn

Om är den enda under-menyn i hjälp-menyn (bild 24). Den ger information om bland annat när programmet skapades.

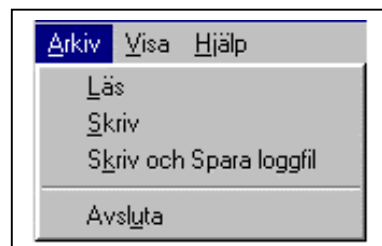


Bild 22. Arkiv-menyn



Bild 23. Visa-menyn



Bild 24. Hjälp-menyn

6.2.2 Anpassning

Innan en laddare skickas till kund måste den anpassas för den typ av batterier som den ska ladda. Därför kan man vid anpassning välja en laddningskurva ur en combobox. Laddningskurvan är en fil som beskriver hur laddaren bäst laddar en viss sorts batteri (bild 25).

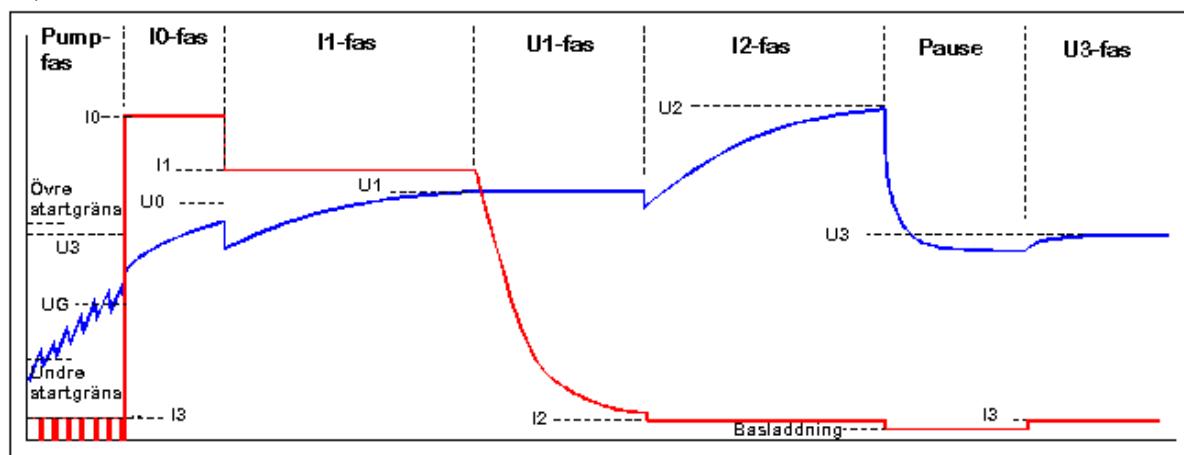


Bild 25. Laddningskurva

I varje fil finns ett namn och en revision som automatiskt skrivs i "Namn" och "Revision" när en laddningskurva har valts.

För att laddningen ska bli ännu mer optimal för den typ av batteri som ska laddas ska också "Antal celler" och "Amperetimmar" anges. I "Laddartyp" skrivs antal volt och ampere. Dessa värden får inte vara högre än de värden som har ställts in i "Grundladdartyp" vid produktion.

6.2.3 Produktion

Vid produktion finns också en combobox, där väljs ett program ur listan. Detta program ska sen användas de andra inmatade parametrarna för att ladda batteriet på korrekt sätt. Den hårdvaruversion som laddaren har ska anges, också ett serienummer, som består av åtta siffror. De första fyra tecknen är år och månad, och sen fyra siffror som är ett löpande nummer. Här ska också kalibreringsdata och grundladdartyp ställas in.

6.2.4 Tekniker

Väljs default-tekniker i visa-menyn kommer man åt gruppboxen för tekniker. Där kan man välja att läsa eller skriva till en viss adress i minnet.

6.3 Kommandon till ST7-interface

Windows-programmet "Laddning ST7" ska kommunicera med interfacet, och använder sig av några kommandon för att tala om för ST7:an på interfacet vad den ska göra.

Kommando 1:

En etta från programmet betyder att den ska göra reset på ST7:an på batteriladdaren.

Kommando 2:

När data ska skrivas till flashminnet på ST7:ans laddare skickas en tvåa till interfacet för att tala om att den ska vara beredd på att ta emot 16 bytes data. Före datan kommer adressen, som är på 2 bytes, som talar om var i flashen som datan ska läggas.

Kommando 3:

När flashen ska läsas av skickas först en trea till interfacet och sen den adress som ska läsas av. Varje gång läses 16 bytes, som sen skickas tillbaka till Windows-programmet.

6.4 Programkod

6.4.1 Funktionen OnInitDialog()

Denna funktion körs innan själva programmet startar. Den ska starta de funktioner och initiera de värden som ska finnas när programmet startar.

Funktionen börjar med att initiera parallellporten. För att göra det används två funktioner:

```
IO1.Open(1);  
IO1.SetECPMode2(2);  
<B5, OnInitDialog()>.
```

De öppnar parallellport LPT1 och sätter den till mode2, ECP mode, detta för att den både ska kunna läsas och skrivas till.

För att programmet ska starta med de senaste visa-inställningarna som användes sparas en fil, startfil.txt varje gång en ändring i visa-menyn görs. I OnInitDialog läses den sen av och sätter

de variabler som styr vilka rutor och knappar som ska synas till de värden de tidigare hade. Filen består bara av ettor och nollor, som talar om ifall exempelvis rutan för serienummer var synlig, TRUE eller dold, FALSE.

För läsning och skrivning i C++ används så kallade strömmar. En ström är en följd av bytes. I olika bibliotek finns operationer för att skriva eller läsa till strömmar. En ström kan kopplas exempelvis till ett tangentbord eller en skärm, men i det här fallet har strömmen kopplats till en fil. För att kunna läsa eller skriva till en fil måste någon av klasserna ifstream eller ofstream användas. När man ska läsa från en fil deklarerar man ett objekt av klassen ifstream. Vid skrivning används ofstream. Här deklarerar man variabel startfil som är en ström av klassen ifstream (se kod nedan). Som parameter anger man filnamnet på den fil som ska öppnas. Används fler parametrar är de för de egenskaper som filen ska ha. Den andra parametern i detta fall talar om att det ska gå läsa från filen. Det är egentligen onödigt här eftersom strömmar av klassen ifstream får in som defaultvärde. Några andra egenskaper som går att ange förutom in är :

Out, som används för att skriva till en fil. Finns inte filen skapas en ny. Finns den redan skrivs den gamla över.

App, Skapar en ny fil om det inte finns någon. Finns det en fil läggs utskriften till sist i filen.

```
ifstream startfil("startfil.txt",ios::in );           //öppna fil
for (antal=0;antal< 11;antal++){                     //sätt de tio variablerna till true/false
    startfil.seekg(antal*sizeof(data));
    startfil.read((char *) &data , sizeof(data));
    switch(antal){
        case 0:      m_LaddningskurvaCheck = data; break;
        case 1:      m_AntalcellerCheck = data; break;
        case 2:      m_AmperetimmarCheck = data; break;
        case 3:      m_ProgramCheck = data; break;
        case 4:      m_HardvaruversionCheck = data; break;
        case 5:      m_KalibreringsdataCheck = data; break;
        case 6:      m_SerienummerCheck = data; break;
        case 7:      m_LaddartypCheck = data; break;
        case 8:      m_GrundladdartypCheck = data; break;
        case 9:      m_DataCheck = data; break;
        case 10:     m_Sparautanloggfildcheck = data; break;
    }
    UpdateData(FALSE);
}
GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
<B5, OnInitDialog()>.
```

Kodraden ovan sätter rutan för serienummer till TRUE eller FALSE beroende på vad som lästes in från filen. GetDlgItem tilldelas ID för den kontroll som ska ändras.

Medlemsfunktionen till det objektet som funktionen returnerar anropas med TRUE eller FALSE.

Filer med laddningskurvor och program finns sparade under varsin katalog, var denna katalog finns någonstans står i filen Sokvag.txt. Denna fil läses in, en rad i taget, och kollar efter om raden börjar på L och A. Gör den det vet den att sökvagen till laddningskurvor står på nästa rad. Samma sak görs med Program. I filen Sokvag.txt står också var någonstans loggfiler ska sparas och var katalogen med alla adresser finns. En loggfil är en fil som sparas varje gång

man har skrivit till en laddare med knappen "Skriv och Spara loggfil". I adresser finns en fil med alla adresser, som definierar var i minnet på ST7:an som olika delar ska sparas. Man läser dessa adresser och sparar de till olika variabler. Till exempel tilldelas variabeln m_Programnamnstart E000, som är adressen till var programmet ska börja sparas på <B4>.

Hittas katalogen med var laddningskurvor ligger, ska filerna i denna katalog skrivas in i Comboboxen för laddningskurvor. Detta görs genom att söka igenom katalogen och se vilka filer den innehåller och lägga till dem i Comboboxen med funktionen addstring.

```
fil = m_LaddningskurvaSokvag + ".*.*";
hFile = ::FindFirstFile(fil, lpFileData);
::FindNextFile(hFile, lpFileData);
while(::FindNextFile(hFile, lpFileData))
    m_Laddningskurva.AddString(lpFileData->cFileName);
::FindClose(hFile);
<B5, OnInitDialog()>.
```

m_Laddningskurva är en variabel av typen CComboBox, som är knuten till comboboxen för laddningskurvor.

Samma sak görs sen för comboboxen för program.

6.4.2 Skicka byte

```
Bool SkickaByte(byte data){

Int raknare = 0;
IO1.WriteData(data);          //Lägger ut byten data på parallellportens dataregister.
IO1.WriteControl(0+0+0+0+0+0+2+1); //Sätter bit 1 hög. Data (om kommando, bit 1 låg)
                                   //ska skickas.
IO1.WriteControl(0+0+0+0+0+0+2+0); //Sätter bit 0 låg. Talar om att ny data finns att hämta.
m_Status = IO1.ReadStatus();      //Läser av parallellportens statusregistret.
while(((m_Status & 0x80) >> 7) == 0 && raknare <= 10000){ //Väntar tills bit 7 är hög
    m_Status = IO1.ReadStatus(); //eller raknare > 10000.
    raknare++;
}
if(raknare < 10000){
    raknare = 0;
    IO1.WriteControl(0+0+0+0+0+0+2+1); //Sätter bit 0 hög igen.
    m_Status = IO1.ReadStatus();
    while(((m_Status & 0x80) >> 7) == 1 && raknare <= 10000){ //Väntar tills
        m_Status = IO1.ReadStatus(); //bit 7 är låg eller
        raknare++; // raknare > 10000.
    }
    if(raknare < 10000){
        IO1.WriteControl(0+0+0+0+0+0+2+1);
    }
    else
        return 0; //Skrivningen av byten misslyckades pga timeout.
}
else
```

```

        return 0;    // Skrivningen av byten misslyckades
                    // pga timeout (raknare > 10000).
return 1;           //Skrivningen av byten lyckades.
}
<B5, SkickaByte()>.

```

När funktionen SkickaByte() anropas ska den ha den byte som ska skickas som parameter. För att kunna använda parallellporten användes en klass som heter CParallelPort <R6>. Ett objekt av denna klass är skapad, den kallas för IO1. I klassen finns en funktion för att lägga ut en byte på parallellportens dataregister, som heter WriteData(). När den har lagts ut sätter man bit 1 i kontrollregistret till hög, för att tala om att det är data som ska skickas. (När ett kommando ska skickas sätts istället bit 1 till låg. Då används en funktion som kallas SkickaKommando()). För att mottagaren ska veta att det finns ny data på porten sätts strobe, bit 0 låg. När mottagaren har upptäckt att det finns ny data på porten sätter den busy, bit 7 hög. Detta väntar sändaren på i en while-sats. Där kollas bit 7 i statusregistret, och så länge som den är låg fortsätter man att kolla av den. Skulle något fel uppstå så att mottagaren inte sätter bit 7 hög finns en timeout i form av en räknare som räknas upp varje gång man läser av statusregistret utan att bit 7 är hög. Skulle timeout uppstå skickas en nolla som returvärde till funktionen som anropade SkickaByte(). Om räknaren är mindre än 10000 (ingen timeout) fortsätter handskakningen med att sätta strobe, bit 0 hög, för att tala om att man tagit emot det ack som mottagaren skickade när den förstod att det fanns ny data att hämta på parallellporten. Nu läser mottagaren av dataregistret och tar emot den byte som skickades. När den har gjort det sätter den busy, bit 1 låg igen. Sändaren väntar på denna busy-signal i en while-sats. Även här finns en räknare för att kunna göra timeout om den inte skulle få något svar. Har funktionen gått så här långt utan en timeout returnerar den en 1 för att tala om att byten har blivit sänd.

6.4 3 Ta emot byte

Programmet tar emot byte i två funktioner, Las16Byte() och LasVardefranAdress().

```

m_Status = IO1.ReadStatus();
while(((m_Status & 0x40) >> 6) == 1 && raknare <= 10000){    //Väntar tills bit 6 är låg
    m_Status = IO1.ReadStatus();                             //eller raknare > 10000.
    raknare++;
}
if(raknare < 10000){
    raknare = 0;
    IO1.WriteControl(0+0+0+0+0+4+0+0);                       //Sätter bit 2 hög.
    m_Status = IO1.ReadStatus();
    while(((m_Status & 0x40) >> 6) == 0 && raknare <= 10000){    //Väntar tills
        m_Status = IO1.ReadStatus();                           //bit 6 är hög eller
        raknare++;                                              // raknare > 10000.
    }
    if(raknare < 10000){
        IO1.WriteControl(0+0+32+0+0+4+0+0); //Bi-directional port.
        m_TaEmot = IO1.ReadData();
        m_Status = IO1.ReadStatus();
    }
    else

```

```

        return 0;    //Läsning av byten misslyckades pga timeout.
    }
    else
        return 0;    // Läsningen av byten misslyckades
                    // pga timeout (räknare > 10000).
    IO1.WriteControl(0+0+0+0+0+0+0+0);    //Klar med läsning på porten.
    return 1;    //Läsning av byten lyckades.
}
<B5, Las16Byte() och LasVardefranAdress(>.

```

När en byte ska tas emot har alltid en adress skickats innan så att sändaren vet på vilken adress som datan som ska skickas tillbaka finns på. Sen väntar programmet på att strobe, bit 6 i kontrollregistret ska gå låg för att tala om att det finns data i dataregistret. Precis som när data skickades finns här en räknare som ger timeout ifall det skulle ta för lång tid innan någon data kommer. Blir strobe låg sätts bit 3 hög för att tala om att man har uppfattat att det finns data på parallellporten. Sen väntar man igen på att strobe ska gå hög. Gör den det kan man läsa av porten. Men för att avläsningen ska bli rätt måste bit 5 sättas hög. Det är en intern bit för att tala om att dubbelriktad port används.

6.4.4 Knappen ”Skriv 16 Byte”, OnSkriv16byteKnapp().

Denna knapp används för att skriva 16 byte till flashminnet på ST7:an. Vilka dessa 16 byte ska vara skrivs in i Data 0-7 och Data 8-15. I Adress skrivs in var i flashminnet de ska sparas. Först kontrolleras det att adressen är fyra tecken lång och att sista tecknet i adressen är en nolla. Det sista tecknet ska vara en nolla eftersom man hela tiden ska skriva 16 byte. Sen kontrolleras datan med hjälp av funktionen Kontroll0till9ellerAtillF(), så att den bara innehåller siffror, eller bokstäver mellan A till F eftersom det är hexadecimalt. Längden kollas också, så att det alltid är två tecken i varje ruta.

```

if (skickadata.GetAt(0) > 64)
    high = skickadata.GetAt(0) - 55;
else
    high = skickadata.GetAt(0) - 48;
if (skickadata.GetAt(1) > 64)
    low = skickadata.GetAt(1) - 55;
else
    low = skickadata.GetAt(1) - 48;
high = high << 4;    //Skiftar high 4 bitar åt vänster.
temp = high + low;    //Lägger de två tecknen i byten temp.

SkickaByte(temp);
<B5, OnSkriv16byteKnapp(>.

```

Innan datan skickas görs den om från ascii-tecken till byte. Man läser in ett tecken i taget med funktionen GetAt() (kod ovan) och tecknets position i strängen som parameter. För att byten ska få rätt värde minskas värdet med 48 eller 55 beroende på om det är en siffra eller ett tecken. Byten får värden mellan 0 till 15 (0 till F). Detta går att skriva med bara 4 bitar, därför läggs två tecken in i varje byte innan den skickas med SkickaByte().

6.4.5 Knappen "Läs 16 Byte", OnLas16byte().

Knappen "Läs 16 byte" läser av 16 byte från flashminnet på ST7ans minne från den adress som har angetts i rutan för adress. Precis som vid skrivning av 16 byte kallas adressen av så att den är giltig. Innan adressen skickas, skickas kommando 3 för att tala om att man vill läsa från flashminnet. När ett kommando skickas sätts bit 1 i kontrollregistret till en nolla. För att ta emot en byte använder man koden i "Ta emot Byte" Den mottagna byten tas om hand och görs om till ascii-tecken och sparas i strängen som hör till data 0. Detta upprepas sen tills alla de 16 byten har lästs in.

6.4.6 Knappen "Skriv", OnSkrivKnapp()

Först kontrolleras det så att läsningen av filer i OnInitDialog() gick bra, så att de adresser som behövs när data ska skrivas i flashminnet är rätt. Gick det inte bra kommer bara ett felmeddelande upp i rutan för meddelande.

Det är bara de variabler eller filer som har blivit ändrade som ska skrivas till flashen. När en variabel ändras gråmarkeras texten som hör till den rutan. Ändringar märks av funktionerna OnChange som hör till varje ruta, se "När är en textbox ändrad?". De rutor som har ändrat kontrolleras så att de innehåller rätt tecken och inte har för höga värden med funktionerna Kontroll0till9() respektive KontrollMaxvarde(). Sen kontrolleras värdena i rutorna för laddartyp, de får inte vara högre än de i grundladdartyp. Om alla de här kontrollerna går igenom utan fel börjar skrivningen till flashminnet med att skicka kommando 1, som gör reset. Sen kallas alla olika rutor igenom och ser vilka som har ändrats. Om en ruta är ändrad så sparas dess värde i en buffert. Denna buffert görs om från integer till hexadecimalt med funktionen FranInttillHex(). Eftersom det alltid ska vara 16 byte som skrivs till flashminnet i taget så fylls bufferten på tills den blir 16 byte stor. Utfyllnadstecknen är alltid 'F'. Ofta är det två variabler som ska skrivas inom samma 16 byte, därför sätts två buffertar ihop innan utfyllnaden med 'F' börjar. Innan bufferten skickas måste kommando 2 skickas för att tala om att det kommer 16 byte data. De adresser som lästes in i OnInitDialog används nu för att tala om var datan ska placeras i minnet.

Om laddningskurva eller program skulle vara ändrad ska en hel fil skickas. Då läser man av i comboboxen vilken fil det är som ska skickas.

Den fil som ska skickas är uppbyggd enligt Motorolas hex format <R7>, där varje rad ser ut så här: Stnnaaaaddddddddddddddd...cc

S – Början på ny rad med data

t - Typ av data, 0=header, 1=data och 9=end of file

n – Antalet byte i raden,

a – Adress

d – data

c – Checksumma av antal, adress och data.

Denna fil öppnas och man börjar läsa av en rad i taget ur filen. Eftersom ST7an bara behöver adressen och den data som ska skrivas in tas de fyra första och de två sista tecknen i varje rad bort. Så länge som det går att läsa in en ny rad skickas först kommando 2. Sen skickas den adress och den data som filraden innehåller. Skulle raden innehålla mindre än 16 byte data skickas 'F' som utfyllnadstecken.

6.4.7 Knappen "Skriv och Spara loggfil", OnSparaloggfilKnapp()

Denna knapp skriver inte bara till flashen på ST7:an, utan den sparar också en loggfil. Loggfilen sparas om skrivningen med funktionen OnSkrivKnapp() till flashminnet lyckas. En loggfil ska sparas till varje laddare som programmeras. Detta för att det ska vara lätt att med hjälp av serienumret se vilka inställningar som har gjorts på just den batteriladdaren. Tillsammans med alla inställningar ska också datum och tid för när batteriladdaren programmerades anges. Därför har en variabel av tiden CTime deklarerats. Med hjälp av denna kan man med funktionen GetYear få fram vilket år det är, med GetMonth få fram vilken månad det är, osv. Detta sparas sen i loggfilen tillsammans med övriga inställningarna.

```
m_tid= m_tid.GetCurrentTime();
tid = m_tid.GetYear();           //Vilket år?
int tusental = 0, int hundratal = 0, int tiotal=0, int ental=0;
char tio,en,tusen,hundra;
while (tid >999){                 //Räknar ut tusental
    tid=tid-1000;
    tusental++;
}
while (tid >99){                  //Räknar ut hundratal
    tid=tid-100;
    hundratal++;
}
while (tid >9){                  //Räknar ut tiotal
    tid=tid-10;
    tiotal++;
}
while (tid >0){                  //Räknar ut ental
    tid=tid-1;
    ental++;
}
tio = tiotal + 48;
en= ental + 48;
tusen = tusental + 48;
hundra = hundratal + 48;
loggfil.write((char *) &Tiden, sizeof(Tiden));    //Skriver "Datum:"
loggfil.write((char *) &tusen, sizeof(char));       //Skriver år till fil
loggfil.write((char *) &hundra, sizeof(char));
loggfil.write((char *) &tio, sizeof(char));
loggfil.write((char *) &en, sizeof(char));
<B5, OnSparaloggfilKnapp(>.
```

6.4.8 Knappen "Läs", OnLasKnapp()

Läs-knappen läser av flashminnet i ST7:an och skriver in de avlästa värdena i respektive ruta. I funktionen OnInitDialog() lästes adresserna, till var datan är sparad in. Därför vet programmet vilka adresser den ska läsa av för att hitta den data som ska skrivas in i rutorna. För att tala om att avläsning ska ske skickas kommando 3. Sen används funktionen LasvardefranAdress() som har den adress som ska avläsas och ett val som paramter. Denna funktion lägger ut adressen på parallellporten och väntar sen på att ta emot de 16 byte som

ligger på denna adress. Den mottagna byten görs om, beroende på vilket värde val har, till rätt typ innan den sparar den i rätt ruta.

Detta upprepas i OnLasKnapp() tills alla adresser har blivit anropade i funktionen LasvardefranAdress().

6.4.9 Checkboxar i Visa-menyn

I visa-menyn finns det checkboxar som används för att välja vad som ska visas i programmet. För att uppdateringen av dessa ska gå rätt till används en funktion som kallas OnKickIdle() som kollar igenom hela menyn och gör uppdateringar. De delar av programmet som inte ska visas och inte går att ändra på är gråmarkerade.

När man klickar på en knapp i visa-menyn körs de här två funktionerna.

```
void CLaddningST7Dlg::OnMenyVisaAntalceller()
{
    m_AntalcellerCheck = !m_AntalcellerCheck;          /Växlar mellan true/false
    GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
    SkrivStartfil();
}
void CLaddningST7Dlg::OnUpdateMenyVisaAntalceller(CCmdUI* pCmdUI)
{
    pCmdUI->SetCheck(m_AntalcellerCheck);
}
```

<B5, OnMenyVisaAntalceller() och OnUpdateMenyVisaAntalceller(CCmdUI* pCmdUI)>.

Den första funktionen växlar variabelvärdet mellan true och false och visar/döljer motsvarande ruta eller knapp i programfönstret. Sen sparas de nya visa-inställningarna i filen startfil.txt med funktionen SkrivStartfil().

Den andra funktionen används för att uppdatera menyn så att rätt val är markerade. Den tar en pekare till ett menyobjekt som parameter, och använder SetCheck() för att göra uppdateringen. Den anropas med true eller false som parameter, beroende på vad funktionen innan satte variabeln till.

6.4.10 När är en editbox ändrad?

Först görs en läsning från flashminnet till programmet. De olika variablerna kommer upp i respektive editbox och combobox. Man kollar sen vilka värden som ska ändras på batteriladdaren. De värden som ska ändras skrivs till flashminnet. För att se vilka värden som ändrats gjordes först en funktion som använde det gamla värdet, sen inläsningen, och jämförde det med det värde som editboxen har vid skrivning. Men hittade sen en inbyggd funktion, OnChange som anropas när man skrivit något i en editbox. För Comboboxarna fanns den funktionen inte, utan istället kan man kolla när en combobox stängs med funktionen OnCloseup. För att markera de editboxar och comboboxar som ändrats gråmarkeras den text som hör till respektive box.

7. Diskussion

7.1 Problem

Vi hade inga tidigare erfarenheter av fönsterhantering i Visual C++, när vi började med detta arbete, utan vi hade bara programmerat textbaserade program. Därför började vi med att göra ett antal mindre program där vi testade olika delar som vi behövde i vårt huvudprogram. Skillnaden sen tidigare programmering var att vi nu fick ett tomt programkodsskal för fönsterhantering, som vi fyller med händelsestyrda funktioner.

Kommunikationen mellan PC och ST7 trodde vi först skulle kunna ske med hjälp av befintlig hårdvara i form av en så kallad ”dong”, som fungerar som interface. Men vi lyckades inte få fram den information som används för att förstå kommunikationen mellan PC och ”dongen”. Istället gjorde vi ett eget interface som vi kommunicerar med via parallellporten.

7.2 Resultat

Designen av programfönstret är ej slutgiltigt godkänt av Micropower, utan kommer att färdigställas efter testkörning.

Eftersom vi har valt att använda ytterligare en ST7:a för att programmera ST7:an i batteriladdaren och inte har haft tid att lära oss programmering av denna är interfacet inte helt färdigt. För att testa vårt huvudprogram har vi istället skapat ett program som simulerar gränssnittet och ST7:ans flashminne.

Referenslista

Monografier:

Skansholm, Jan (2000). C++ Direkt : Studentlitteratur

Chapman, Davis (2000). Lär dig Visual C++ 6 på tre veckor : Pagina

Horton's, Ivor (1998). Introduction to Microsoft Visual C++ 6.0 Standard edition : Wrox

Källor på Internet:

R1: Programming ST7 flash microcontrollers in remote ISP mode.

URL : <http://w.st.com/stoneline/books/toc/an/379.htm> (2002-05-28).

R2 : ST7 8 bit Microcontroller.

URL: <http://w.st.com/stoneline/books/pdf/docs/6815.pdf> (2002-05-28).

R3 : ST7 ICC Protocol Reference manual.

URL : <http://us.st.com/stoneline/books/pdf/docs/8227.pdf> (2002-05-28).

R4: Interfacing the Standard Parallel Port.

URL: <http://www.beyondlogic.org/spp/parallel.htm> (2002-05-28).

R5: Interfacing the Extended Capabilities Port

URL: <http://www.beyondlogic.org/ecp/ecp.htm> (2002-05-28).

R6: CParallelPort V1.1.

URL: <http://www.naughter.com/parallelport.html> (2002-05-28).

R7: Intel/Motorola Hex format (for EPROM programming).

URL: http://www.buchanan1.net/hex_format.html (2002-05-04).

Bilagor

- B1. Användarmanual
- B2. Kretsschema och layout för gränssnitt mellan PC och batteriladdare, version 1.
- B3. Kretsschema och layout för gränssnitt mellan PC och batteriladdare, version 2.
- B4. AdressST7.txt
- B5. ST7 benkonfiguration
- B6. Microsoft Visual C++ programkod.

B1. Användarmanual

ANVÄNDARMANUAL FÖR ISP AV ST7 med MICROSOFT VISUAL C++ PROGRAM

Programmet LaddningST7.exe används för att flashprogrammera microprocessorn ST7 och en fil skapas som definierar vad som inskrivet i flashen. ISP (In System Programmable) används, vilket betyder att ST7 - processorn är fast installerad i en batteriladdare.

Läsning och skrivning av följande parametrar kan göras till flash - minnet.

- huvudprogram
- laddningskurva
- serienummer
- hårdvaruversion
- amperetimmar
- antal celler
- grundladdartyp volt
- grundladdartyp ampere
- laddartyp volt
- laddartyp ampere
- kalibreringsdata

Innehållsförteckning

Avsnitt

1 . Installering	sida 3
2 . Användningsområde	sida 3
3 . Användarmanual	sida 4
3.1 Användarfall	sida 4
3.2 Parameterskrivning i de olika rutorna	sida 5
4 . Felmeddelandelista	sida 7
5 . Beskrivning av filen sokvag.txt	sida 10
6 . Beskrivning av filen AdressST7.txt	sida 11

1 . Installering.

- Kopiera LaddningST7.exe till lämplig mapp.
- Kopiera filen sokvag.txt till samma mapp.
- Kopiera mappen Laddningskurvor till angiven mapp som finns i filen sokvag.txt.
- Kopiera mappen Programfiler till angiven mapp som finns i filen sokvag.txt.
- Kopiera mappen Adresser till angiven mapp som finns i filen sokvag.txt.
- Anslut parallellporten på PC till parallellporten på ett kretskort (Interface).
- Anslut kretskortets (Interface) HE10 kontakt till batteriladdaren.

2 . Användningsområde.

Programmet LaddningST7.exe används för att flashprogrammera microprocessorn ST7. ISP (In - Suit Programming) används, vilket betyder att ST7 - processorn är fast installerad i en batteriladdare.

Läsning och skrivning av följande parametrar kan göras till flash - minnet.

- huvudprogram
- laddningskurva
- serienummer
- hårdvaruversion
- amperetimmar
- antal celler
- grundladdartyp volt
- grundladdartyp ampere
- laddartyp volt
- laddartyp ampere
- kalibreringsdata

3 . Användarmanual.

The screenshot shows a software window titled "LaddningST7" with a menu bar containing "Arkiv", "Visa", and "Hjälp". The window is divided into three main sections: "Anpassning", "Produktion", and "Tekniker".

Anpassning

- Laddningskurva: Laddningskurva.s19 (dropdown)
- Namn: LK-3
- Revision: 05.28
- Antal celler: 6
- Amperetimmar: 12
- Laddartyp: 12
- Volt: 6
- Ampere: (empty)

Produktion

- Program: Program.s19 (dropdown)
- Namn: Styrprg .
- Revsion: 12.34
- Hårdvaruversion: 2
- Serienummer: 02051234
- Kalibreringsdata: 3
- Grundladdartyp: 15
- Volt: 10
- Ampere: (empty)

Tekniker

- Adress: E000
- Data 0-7: 00, 02, 80, 00, 00, 64, 00, 20
- Data 8-15: 05, 04, 00, 20, 00, 44, FF, 01
- Buttons: Läs 16 Byte, Skriv 16 Byte

Meddelande

Skrivning lyckades!

Buttons at the bottom: Läs, Skriv, Skriv och Spara loggfil

Bild 1.Programfönster.

Programfönstret ser ut som bild 1.

Första gången programmet körs finns inga sparade fönsterinställningar.

De olika alternativ som finns väljs i menyn Visa. Se bild 2.

3.1 . Användarfall

- Anpassning
 - Val av följande parametrar ska kunna skrivas och läsas.
 - Laddningskurva
 - Antal celler
 - Amperetimmar
 - Laddartyp
 - Loggfil sparas automatiskt

I visamenyn väljer man default anpassning för att anpassa fönstret till enbart dessa funktioner. Går även att välja de olika alternativen som tillhör anpassningen enskilt.

- Produktion
Alla parametrar ska kunna skrivas och läsas.
Val om loggfil ska sparas eller inte.

I visamenyn väljer man default produktion för att anpassa fönstret till enbart dessa funktioner. Går även att välja de olika alternativen som tillhör produktion enskilt.

- Tekniker
Alla parametrar ska kunna skrivas och läsas.
Läsa eller skriva 16 byte från en viss adress
Val om loggfil ska sparas eller inte

I visamenyn väljer man default tekniker för att anpassa fönstret till enbart dessa funktioner. Går även att välja de olika alternativen som tillhör tekniker enskilt.

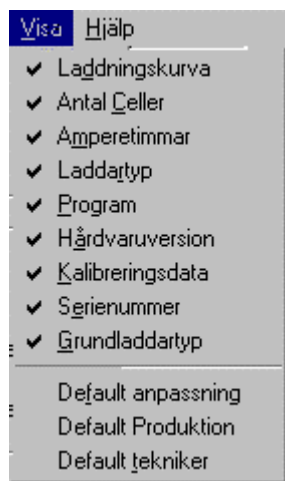


Bild 2. Menyn Visa.

3.2 . Parameterskrivning i de olika rutorna.

Det finns två lister i fönstret där man rullar fram de olika alternativen för Laddningskurva och Program. Det går endast välja av de fördefinierade namnen. Man får ej skriva in egna namn. Vid val av ett namn bekräftas valet genom att namn och revision kommer upp i resp. ruta. De andra rutorna är av typ där man själv skriver in vilket värde man vill ha. Parametervärdena kontrolleras enligt följande, innan något skrivs in i flashminnet på ST7. Serienummer, 8 siffror (År Mån Löpnr. ex 02051234). Hårdvaruversion, max värde 10. Amperetimmars, max värde 255. Antal celler max värde 60. Grundladdartyp volt max värde 120. Grundladdartyp ampere max värde 255. Laddartyp volt max värde 120.

Laddartyp ampere max värde 255.
Kalibreringsdata max värde ej definierat.
Felmeddelande enligt felmeddelandelista kommer upp i rutan för resp. fel.

Första åtgärd när man kopplat in en laddare är att klicka på knappen Läs för att få info om vad som finns i flashminnet innan man skriver in något nytt.
Läsning lyckades! Kommer upp i meddelanderutan, annars kommer ett felmeddelande (se felmeddelandelista).

Skriv och Spara loggfil - knapp. De parametervärden som är valda kommer att skrivas in på resp. adress. Skrivning lyckades! Kommer upp i meddelanderutan, annars kommer ett felmeddelande (se felmeddelandelista).
En loggfil sparas i mappen som finns deklarerad i filen sokvag.txt. De parametrar man har valt kommer att sparas i en textfil.

Skriv utan Spara loggfil - knapp. De parametervärden som är valda kommer att skrivas in på resp. adress. Skrivning lyckades! Kommer upp i meddelanderutan, annars kommer ett felmeddelande (se felmeddelandelista).

Skriv 16 Byte - knapp. De data som är valda kommer att skrivas in på adressen. Skrivning lyckades! Kommer upp i meddelanderutan, annars kommer ett felmeddelande (se felmeddelandelista).

Läs 16 Byte - knapp. De data som finns på adressen som angivits i Adressrutan kommer att presenteras i dataruta 0 - 15. Läsning lyckades! Kommer upp i meddelanderutan, annars kommer ett felmeddelande (se felmeddelandelista).

I menyn Arkiv finns följande val (bild 3).

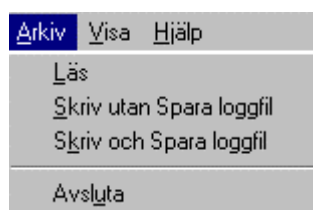


Bild 3. Arkivmeny.

4 . Felmeddelandelista

Fel nr	Beskrivning	Åtgärd
1	Felaktig katalog eller tom katalog för laddningskurva.	Kontrollera mappen Laddningskurvor. Kontrollera innehållet i filen sokvag.txt.
2	Felaktig katalog eller tom katalog för program.	Kontrollera mappen Programfiler. Kontrollera innehållet i filen sokvag.txt .
3	Fel på katalog eller tom katalog för adresser.	Kontrollera mappen Adresser. Innehåller den korrekta adresser till parametrarna? Se avsnitt 6. Kontrollera innehållet i filen sokvag.txt.
4	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaData ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Amperetimmar och Antal Celler.
5	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen Se Amperetimmar och Antal Celler .
6	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen Se Amperetimmar och Antal Celler .
7	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaData().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddartyp Volt och Laddartyp Ampere.
8	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddartyp Volt och Laddartyp Ampere.
9	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddartyp Volt och Laddartyp Ampere.
10	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaData().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Grundladdartyp Volt och Grundladdartyp Ampere.
11	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Grundladdartyp Volt och Grundladdartyp Ampere.
12	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Grundladdartyp Volt och Grundladdartyp Ampere.
13	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaData().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Hårdvaruversion.
14	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Hårdvaruversion.

15	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Hårdvaruversion.
16	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaByte().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Serienummer.
17	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaByte().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Serienummer.
18	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Serienummer.
19	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Serienummer.
20	Felaktigt serienummer!	Kontrollera att serienummer = 8 siffror
21	Laddningskurvafilläsning misslyckad.	Kontrollera mappen Laddningskurva i filen sokvag som ligger i samma katalog som .exe filen. Innehåller den korrekta Laddningskurvor?
22	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor.
23	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor.
24	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor.
25	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor.
26	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor.
27	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Laddningskurvor..
28	Laddningskurvafilläsning misslyckad.	Kontrollera mappen Programfiler i filen sokvag som ligger i samma katalog som .exe filen. Innehåller den korrekta Programfiler?
29	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
30	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
31	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.

		Se Programfiler.
32	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
33	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
34	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
35	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaKommando ().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen. Se Programfiler.
36	Felaktigt värde i Antal celler.	Kontrollera värdet på parametern.
37	Felaktigt värde i Amperetimmar.	Kontrollera värdet på parametern.
38	Felaktigt värde i Hårdvaruversion.	Kontrollera värdet på parametern.
39	Felaktigt värde i Kalibreringsdata	Kontrollera värdet på parametern.
40	Felaktigt värde i Serienummer.	Kontrollera värdet på parametern.
41	Felaktigt värde i Laddartyp Volt.	Kontrollera värdet på parametern.
42	Felaktigt värde i Laddartyp Ampere.	Kontrollera värdet på parametern.
43	Felaktigt värde i Grundladdartyp Volt.	Kontrollera värdet på parametern.
44	Felaktigt värde i Grundladdartyp Ampere.	Kontrollera värdet på parametern.
45	Felaktigt värde i Laddningskurva.	Kontrollera värdet på parametern.
46	Felaktigt värde i Program.	Kontrollera värdet på parametern.
47	Program i felläge. Adresser till parametrar kan ej hittas.	Kontrollera mappen Adresser. Innehåller den korrekta adresser till parametrarna? Se avsnitt 6. Kontrollera innehållet i filen sokvag.txt.
48	Kontakten bröts, läsningen avslutades. Fel har uppstått vid funktionen SkickaKommando() eller funktionen LasvardefranAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
49	Kontakten bröts, läsningen avslutades. Väntar på att bit 0 i kontrollregistret ska gå hög.	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
50	Kontakten bröts, läsningen avslutades. Väntar på att bit 0 i kontrollregistret ska gå låg.	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
51	Kontakten bröts, läsningen avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
52	Kontakten bröts, läsningen avslutades. Fel uppstod i funktionen SkickaKommando().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
53	Felaktigt tecken i adressruta.	Kontrollera att tecken = 0 - 9 eller A - F.
54	Felaktigt adressområde i adressruta.	Kontrollera att tecken nr 1 = E eller F.
55	Felaktig adress.	Kontrollera att tecken nr 4 = 0.
56	Felaktigt antal tecken i adressruta.	Kontrollera att antalet = 4.
57	Kontakten bröts, skrivning avslutades. Fel uppstod i funktionen SkickaAdress().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
58	Kontakten bröts, läsningen avslutades. Fel	Fel har uppstått vid överföringen.

	uppstod i funktionen SkickaKommando().	Kontrollera anslutningen och försök igen..
59	Kontakten bröts, läsningen avslutades. Fel uppstod i funktionen SkickaKommando().	Fel har uppstått vid överföringen. Kontrollera anslutningen och försök igen.
60	Felaktigt antal tecken i dataruta.	Kontrollera att antalet = 2.
61	Felaktigt tecken i dataruta.	Kontrollera att tecken är 0 -9 eller A - F.
62	Felaktigt tecken i adressruta.	Kontrollera att tecken är 0 -9 eller A - F.
63	Felaktigt adressområde i adressruta.	Kontrollera att tecken nr 1 = E eller F
64	Felaktig adress.	Kontrollera att tecken nr 4= 0.
65	Felaktigt antal tecken i adressruta.	Kontrollera att antalet = 4.
66	Inget giltigt serienummer.	Kontrollera att antalet tecken = 8.
67	Program i felläge. Adresser till parametrar kan ej hittas.	Kontrollera Filen sokvag i samma mapp som .exe. Filen har ej hittats eller felaktigt innehåll i filen.
68	Laddningskurvafilläsning misslyckad. Fel uppstod i funktionen OnCloseupLaddningskurvaCombo().	Kontrollera mappen Laddningskurvor. Sökväg finns i filen sokvag som finns i samma mapp som .exe filen.
69	Välj Laddningskurva i listan.	Gör ett val.
70	Programfilläsning misslyckad. Fel uppstod i funktionen OnCloseupLaddningskurvaCombo().	Kontrollera mappen Programfiler. Sökväg finns i filen sokvag som finns i samma mapp som .exe filen.
71	Välj Program i listan.	Gör ett val.
72	Hittade ej alla adresser. Fel uppstod i funktionen LasvardefranAdress().	Försök igen.
73	Kontakten bröts, läsning avslutades. Fel har uppstått vid överföringen.	Kontrollera anslutningen och försök igen.
74	Fel vid läsning av katalogfil, hittar ej sökvägar. Programmet kommer inte fungera felfritt. Vill du fortsätta?	Klicka på Ja eller Nej.
75	Fel vid läsning av adressfil, hittar ej sökvägar. Programmet kommer inte fungera felfritt. Vill du fortsätta?	Klicka på Ja eller Nej.

5 . Beskrivning av filen sokvag.txt

Följande utseende krävs på filen.

INFORMATIONSFIL SÖKVÄG

LADDNINGSKURVA:

I:\Laddningskurvor\ //Här skrivs sökväg för katalog med laddningskurvor.

PROGRAM:

I:\Programfiler\ //Här skrivs sökväg för katalog med programfiler.

LOGGFIL:

I:\Loggfiler\ //Här skrivs sökväg för katalog med loggfiler.

ADRESSER:

I:\Adresser\ //Här skrivs sökväg för katalog med filen AdressST7.txt.

Inga tomma rader får förekomma och namnen LADDNINGSKURVA, PROGRAM och ADRESSER måste ligga raden ovanför på resp. sökväg.

6 . Beskrivning av filen AdressST7.txt.

Följande utseende krävs på filen.

ADRESSFIL ST7

PROGRAM:

E000-E008:Programnamn	//E000 = Startadress för programnamn
E009-E00E:Programrevision	//E009 = Startadress för revisionnamn
E010:Optionsbyte0	//E010 = Adress för optionbyte0
E011:Optionsbyte1	//E010 = Adress för optionbyte1

EBFF:Maxstorlek(7kByte)

LADDNINGSKURVA:

FC00-FC08:Laddningskurvanamn	//FC00 = Startadress för programnamn
FC09-FC0E:Laddningskurvarevision	//FC09 = Startadress för revisionnamn

FC7F:Maxstorlek(128Byte)

KALIBRERINGSDATA:

VAKANT

GRUNDLADDARTYP:

FC80-FC81:Volt	//FC80 = 00, FC81 = variabelvärde
FC82-FC83:Ampere	//FC82 = 00, FC83 = variabelvärde

LADDARTYP:

FC90-FC91:Volt	//FC90 = 00, FC91 = variabelvärde
FC92-FC93:Ampere	//FC92 = 00, FC93 = variabelvärde

ANTALCELLER:

FCA0-FCA1:Antal celler	//FCA0 = 00, FCA1 = variabelvärde
------------------------	-----------------------------------

AMPERTIMMAR:

FCA2-FCA3:Amperetimmars	//FCA2 = 00, FCA3 = variabelvärde
-------------------------	-----------------------------------

FCA4-FCAF:Vakant

HÅRDVARUVERSION:

FCB0-FCB1:Hårdvaruversion	//FCB0 = 00, FCB1 = variabelvärde
---------------------------	-----------------------------------

FCB2-FCBF:Vakant

SERIENUMMER:

FCC0-FCC7:Serienummer	//FCC0 = Startadress för serienummer
-----------------------	--------------------------------------

FCC8-FCCF:Vakant

INTERUPTTABELL:	//FFE0 = Startadress för interupttabell
-----------------	---

FFE0-FFF0:32Byte

Inga tomma rader får förekomma och namnen PROGRAM, LADDNINGSKURVA, KALIBRERINGSDATA, GRUNDLADDARTYP, LADDARTYP, ANTALCELLER, AMPERETIMMAR, HÅRDVARUVERSION, SERIENUMMER och INTERUPTTABELL måste ligga raden ovanför resp. sökväg.

Adressen skrivs med 4 tecken och utan mellanslag. Exempelvis i raden FCBO-FCBF där det är de 4 första tecknen som används i sökningen. De 4 sista tecknen är för informationssyfte, där det framkommer hur många byte resp. parameter har till förfogande.

B2. Kretsschema och layout för gränssnitt mellan PC och batteriladdare, version 1.

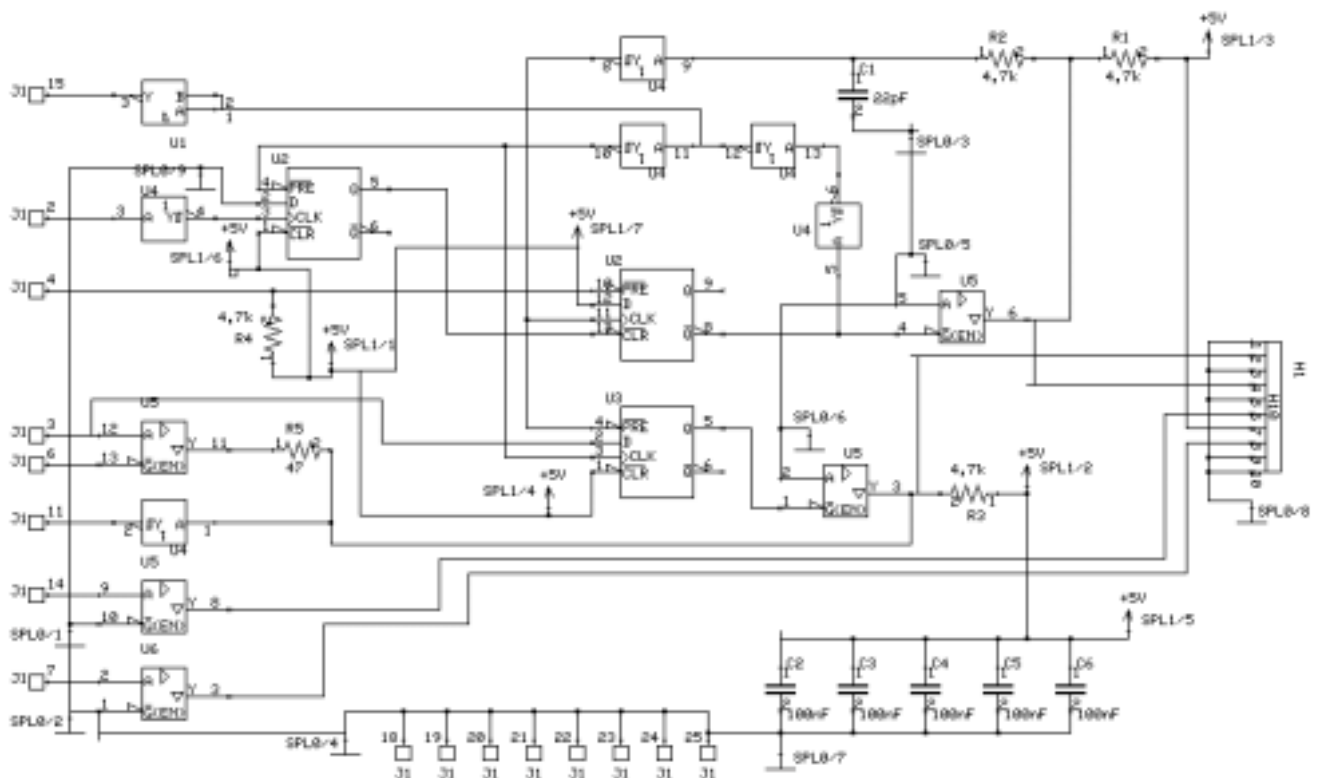


Bild 2.1 Kretsschema

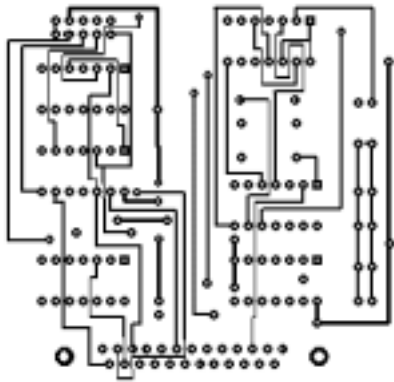


Bild 2.2 Layout för lödsida



Bild 2.3 Layout för komponentsida

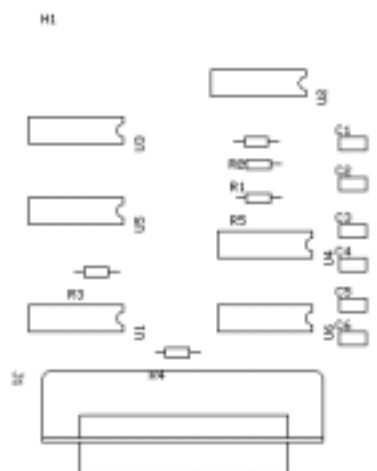


Bild 2.4 Komponentplacering

B3. Kretsschema och layout för gränssnitt mellan PC och batteriladdare, version 2.

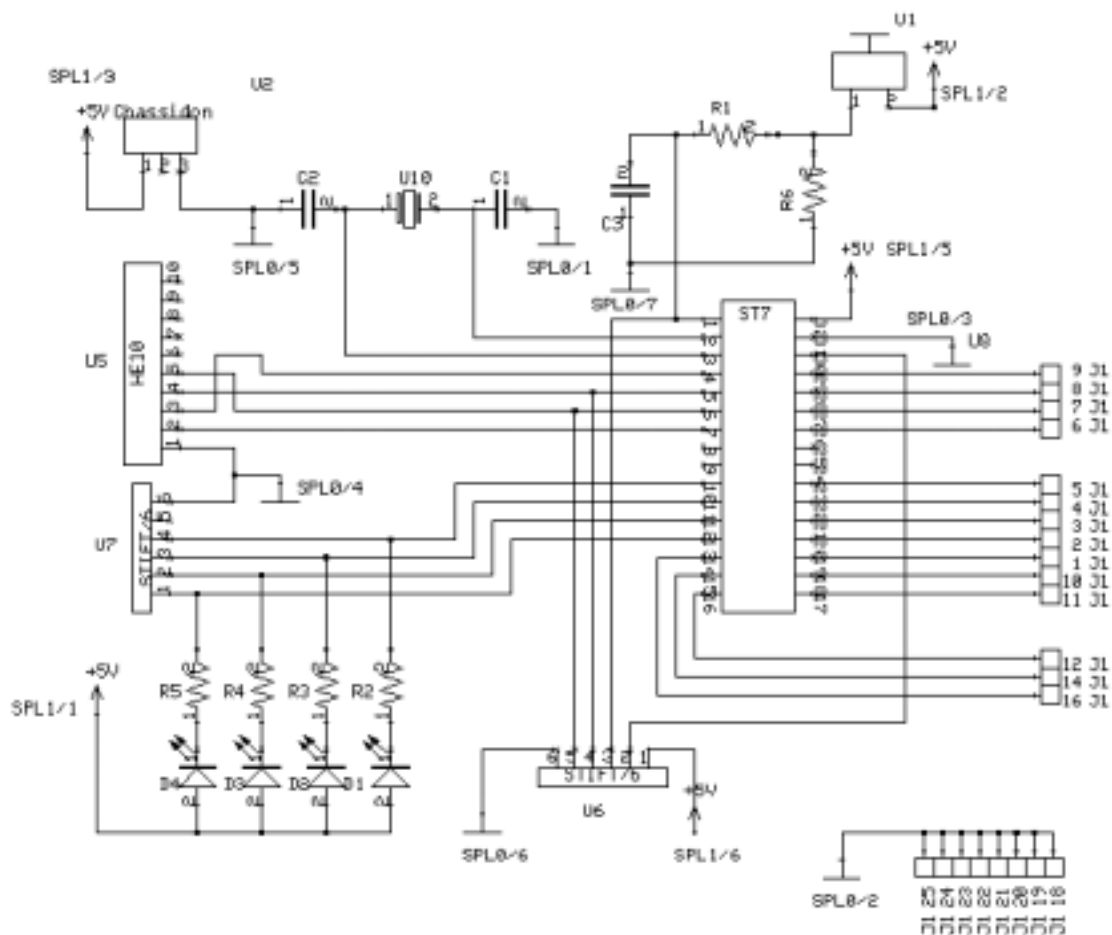


Bild 3.1 Kretsschema

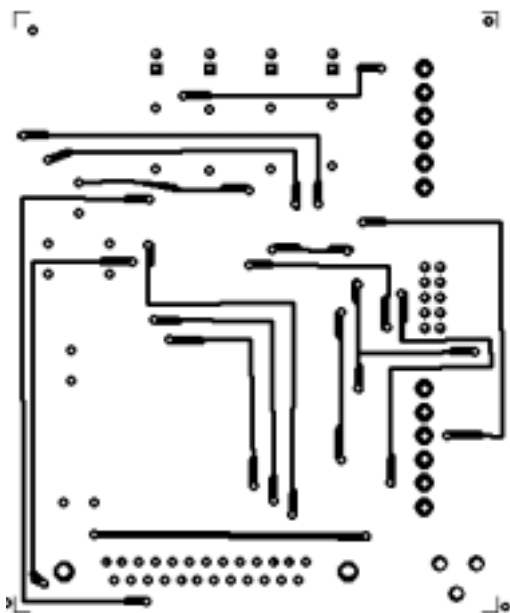


Bild 3.2 Layout för lödsida.

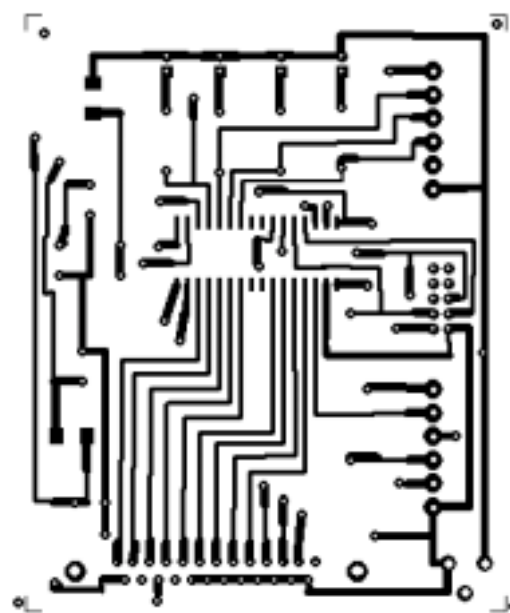


Bild 3.3 Layout för komponentsida.

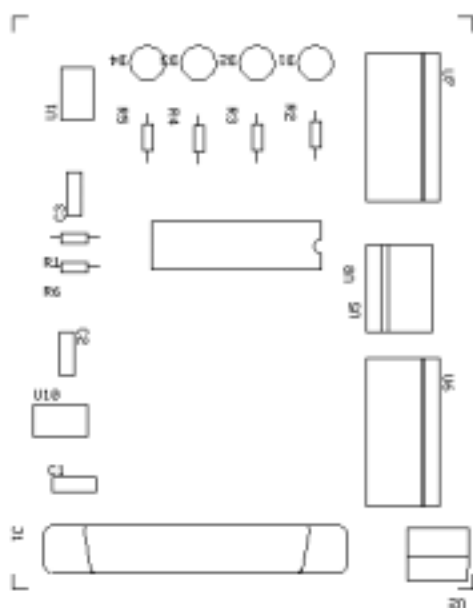


Bild 3.4 Komponentplacering

B4. AdressST7.txt

ADRESSFIL ST7

PROGRAM:

E000-E008:Programnamn

E009-E00F:Programrevision

E010:Optionsbyte0

E011:Optionsbyte1

EBFF:Maxstorlek(7kByte)

LADDNINGSKURVA:

FC00-FC08:Laddningskurvanamn

FC09-FC0F:Laddningskurvarevision

FC7F:Maxstorlek(128Byte)

KALIBRERINGSDATA:

FC

GRUNDLADDARTYP:

FC80-FC81:Volt

FC82-FC83:Ampere

LADDARTYP:

FC90-FC91:Volt

FC92-FC93:Ampere

ANTALCELLER:

FCA0-FCA1:Antal celler

AMPERTIMMAR:

FCA2-FCA3:Amperetimmar

FCA4-FCAF:Vakant

HÅRDVARUVERSION:

FCB0-FCB1:Hårdvaruversion

FCB2-FCBF:Vakant

SERIENUMMER:

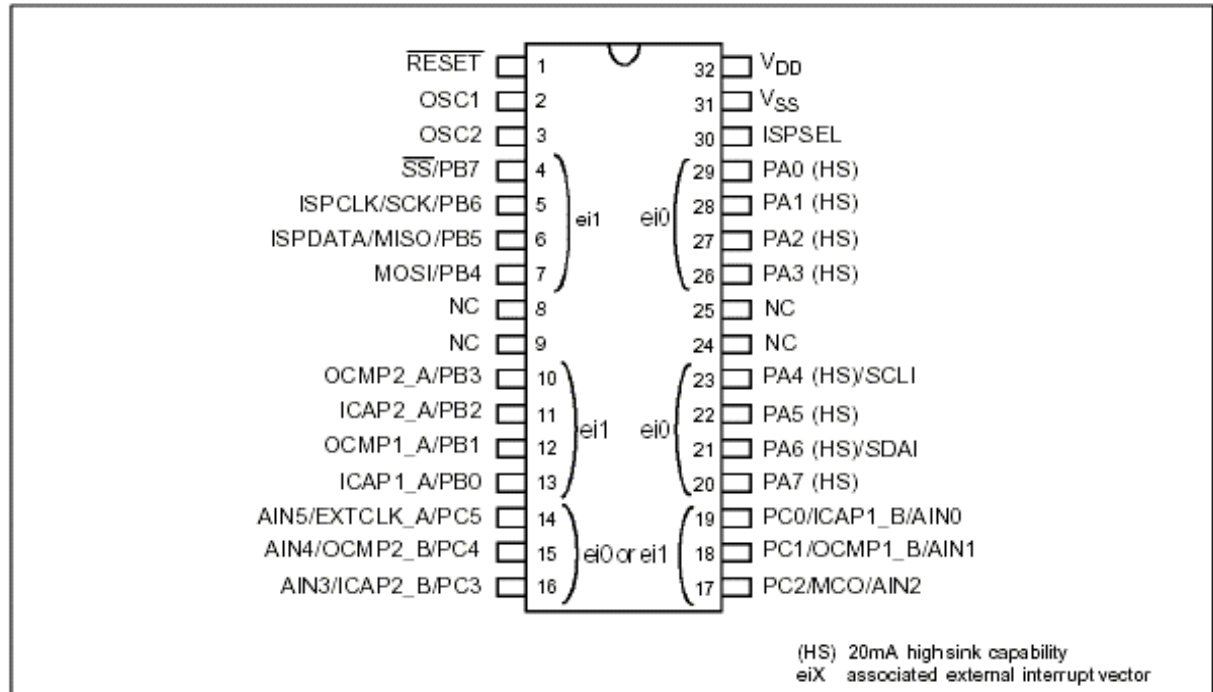
FCC0-FCC7:Serienummer

FCC8-FCCF:Vakant

INTERUPTTABELL:

FFE0-FFF0:32Byte

B5. ST7 benkonfiguration



B6. Microsoft Visual C++ programkod

```
// LaddningST7Dlg.cpp : implementation file

#include "stdafx.h"
#include "LaddningST7.h"
#include "LaddningST7Dlg.h"
#include <afxpriv.h>
#include "ParallelPort.h"
#include <iostream.h>
#include <fstream.h>
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

// CAboutDlg dialog used for App About
class CAboutDlg : public CDialog
{
public:
    CAboutDlg();
// Dialog Data
   //{{AFX_DATA(CAboutDlg)
    enum { IDD = IDD_ABOUTBOX };
    }}AFX_DATA
    // ClassWizard generated virtual function overrides
   //{{AFX_VIRTUAL(CAboutDlg)
protected:
    virtual void DoDataExchange(CDataExchange* pDX); // DDX/DDV support
    }}AFX_VIRTUAL
// Implementation
protected:
   //{{AFX_MSG(CAboutDlg)
    }}AFX_MSG
    DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
   //{{AFX_DATA_INIT(CAboutDlg)
    }}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
    CDialog::DoDataExchange(pDX);
   //{{AFX_DATA_MAP(CAboutDlg)
    }}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
   //{{AFX_MSG_MAP(CAboutDlg)
        // No message handlers
    }}AFX_MSG_MAP
END_MESSAGE_MAP()

// CLaddningST7Dlg dialog

CLaddningST7Dlg::CLaddningST7Dlg(CWnd* pParent /*=NULL*/)
{
}
```

```

: CDialog(CLaddningST7Dlg::IDD, pParent)
{
//{{AFX_DATA_INIT(CLaddningST7Dlg)
m_Amperetimmar = _T(""); //Startvärden i de olika edit-rutorna och combo-boxarna
m_Serienummer = _T("");
m_DataRuta0 = "00";
m_DataRuta1 = "02";
m_DataRuta10 = "00";
m_DataRuta11 = "20";
m_DataRuta12 = "00";
m_DataRuta13 = "44";
m_DataRuta14 = "FF";
m_DataRuta15 = "01";
m_DataRuta2 = "80";
m_DataRuta3 = "00";
m_DataRuta4 = "00";
m_DataRuta5 = "64";
m_DataRuta6 = "00";
m_DataRuta7 = "20";
m_DataRuta8 = "05";
m_DataRuta9 = "04";
m_Adress = _T("E000");
m_Antalceller = _T("");
m_GrundladdartypAmpere = _T("");
m_GrundladdartypVolt = _T("");
m_Hardvaruversion = _T("");
m_LaddartypAmpere = _T("");
m_LaddartypVolt = _T("");
m_tid = 0;
m_Kalibreringsdata = _T("");
m_Felmeddelande = _T("");
m_LaddningskurvaNamn = _T("");
m_LaddningskurvaRevision = _T("");
m_ProgramNamn = _T("");
m_ProgramRevision = _T("");
//}}AFX_DATA_INIT
// Note that LoadIcon does not require a subsequent DestroyIcon in Win32
m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);
}

```

```

void CLaddningST7Dlg::DoDataExchange(CDataExchange* pDX)
{
CDialog::DoDataExchange(pDX);
//{{AFX_DATA_MAP(CLaddningST7Dlg) Dataöverföring mellan minnet och rutorna
DDX_Control(pDX, IDC_ProgramCombo, m_ProgramCombo);
DDX_Control(pDX, IDC_LaddningskurvaCombo, m_Laddningskurva);
DDX_Text(pDX, IDC_AmperetimmarRuta, m_Amperetimmar);
DDV_MaxChars(pDX, m_Amperetimmar, 3);
DDX_Text(pDX, IDC_SerienummerRuta, m_Serienummer);
DDV_MaxChars(pDX, m_Serienummer, 8);
DDX_Text(pDX, IDC_DataRuta0, m_DataRuta0);
DDV_MaxChars(pDX, m_DataRuta0, 2);
DDX_Text(pDX, IDC_DataRuta1, m_DataRuta1);
DDV_MaxChars(pDX, m_DataRuta1, 2);
DDX_Text(pDX, IDC_DataRuta10, m_DataRuta10);
DDV_MaxChars(pDX, m_DataRuta10, 2);
DDX_Text(pDX, IDC_DataRuta11, m_DataRuta11);
DDV_MaxChars(pDX, m_DataRuta11, 2);
DDX_Text(pDX, IDC_DataRuta12, m_DataRuta12);
DDV_MaxChars(pDX, m_DataRuta12, 2);

```

```

DDX_Text(pDX, IDC_DataRuta13, m_DataRuta13);
DDV_MaxChars(pDX, m_DataRuta13, 2);
DDX_Text(pDX, IDC_DataRuta14, m_DataRuta14);
DDV_MaxChars(pDX, m_DataRuta14, 2);
DDX_Text(pDX, IDC_DataRuta15, m_DataRuta15);
DDV_MaxChars(pDX, m_DataRuta15, 2);
DDX_Text(pDX, IDC_DataRuta2, m_DataRuta2);
DDV_MaxChars(pDX, m_DataRuta2, 2);
DDX_Text(pDX, IDC_DataRuta3, m_DataRuta3);
DDV_MaxChars(pDX, m_DataRuta3, 2);
DDX_Text(pDX, IDC_DataRuta4, m_DataRuta4);
DDV_MaxChars(pDX, m_DataRuta4, 2);
DDX_Text(pDX, IDC_DataRuta5, m_DataRuta5);
DDV_MaxChars(pDX, m_DataRuta5, 2);
DDX_Text(pDX, IDC_DataRuta6, m_DataRuta6);
DDV_MaxChars(pDX, m_DataRuta6, 2);
DDX_Text(pDX, IDC_DataRuta7, m_DataRuta7);
DDV_MaxChars(pDX, m_DataRuta7, 2);
DDX_Text(pDX, IDC_DataRuta8, m_DataRuta8);
DDV_MaxChars(pDX, m_DataRuta8, 2);
DDX_Text(pDX, IDC_DataRuta9, m_DataRuta9);
DDV_MaxChars(pDX, m_DataRuta9, 2);
DDX_Text(pDX, IDC_AdressRuta, m_Adress);
DDV_MaxChars(pDX, m_Adress, 4);
DDX_Text(pDX, IDC_Antalcellerruta, m_Antalceller);
DDV_MaxChars(pDX, m_Antalceller, 3);
DDX_Text(pDX, IDC_GrundladdartypAmpereRuta, m_GrundladdartypAmpere);
DDV_MaxChars(pDX, m_GrundladdartypAmpere, 3);
DDX_Text(pDX, IDC_GrundladdartypVoltRuta, m_GrundladdartypVolt);
DDV_MaxChars(pDX, m_GrundladdartypVolt, 3);
DDX_Text(pDX, IDC_HardvaruversionRuta, m_Hardvaruversion);
DDV_MaxChars(pDX, m_Hardvaruversion, 3);
DDX_Text(pDX, IDC_LaddartypAmpereRuta, m_LaddartypAmpere);
DDV_MaxChars(pDX, m_LaddartypAmpere, 3);
DDX_Text(pDX, IDC_LaddartypVoltRuta, m_LaddartypVolt);
DDV_MaxChars(pDX, m_LaddartypVolt, 3);
DDX_DateTimeCtrl(pDX, IDC_DATETIMEPICKER1, m_tid);
DDX_Text(pDX, IDC_KalibreringsdataRuta, m_Kalibreringsdata);
DDV_MaxChars(pDX, m_Kalibreringsdata, 3);
DDX_Text(pDX, IDC_Felmeddelande, m_Felmeddelande);
DDX_Text(pDX, IDC_LaddningskurvaNamnRuta, m_LaddningskurvaNamn);
DDX_Text(pDX, IDC_LaddningskurvaRevisionRuta, m_LaddningskurvaRevision);
DDX_Text(pDX, IDC_ProgramNamnRuta, m_ProgramNamn);
DDX_Text(pDX, IDC_ProgramRevisionRuta, m_ProgramRevision);
//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CLaddningST7Dlg, CDialog)
//{{AFX_MSG_MAP(CLaddningST7Dlg) Hanterar olika mus-klickningar
ON_WM_SYSCOMMAND()
ON_WM_PAINT()
ON_WM_QUERYDRAGICON()
ON_BN_CLICKED(IDC_SkrivKnapp, OnSkrivKnapp)
ON_BN_CLICKED(IDC_LasKnapp, OnLasKnapp)
ON_COMMAND(MenyVisaAmperetimmer, OnMenyVisaAmperetimmer)
ON_UPDATE_COMMAND_UI(MenyVisaAmperetimmer, OnUpdateMenyVisaAmperetimmer)
ON_COMMAND(MenyVisaAntalceller, OnMenyVisaAntalceller)
ON_UPDATE_COMMAND_UI(MenyVisaAntalceller, OnUpdateMenyVisaAntalceller)
ON_COMMAND(MenyVisaGrundladdartyp, OnMenyVisaGrundladdartyp)
ON_UPDATE_COMMAND_UI(MenyVisaGrundladdartyp, OnUpdateMenyVisaGrundladdartyp)

```

```

ON_COMMAND(MenyVisaHardvaruversion, OnMenyVisaHardvaruversion)
ON_UPDATE_COMMAND_UI(MenyVisaHardvaruversion, OnUpdateMenyVisaHardvaruversion)
ON_COMMAND(MenyVisaKalibreringsdata, OnMenyVisaKalibreringsdata)
ON_UPDATE_COMMAND_UI(MenyVisaKalibreringsdata, OnUpdateMenyVisaKalibreringsdata)
ON_COMMAND(MenyVisaLaddartyp, OnMenyVisaLaddartyp)
ON_UPDATE_COMMAND_UI(MenyVisaLaddartyp, OnUpdateMenyVisaLaddartyp)
ON_COMMAND(MenyVisaLaddningskurva, OnMenyVisaLaddningskurva)
ON_UPDATE_COMMAND_UI(MenyVisaLaddningskurva, OnUpdateMenyVisaLaddningskurva)
ON_COMMAND(MenyVisaProgram, OnMenyVisaProgram)
ON_UPDATE_COMMAND_UI(MenyVisaProgram, OnUpdateMenyVisaProgram)
ON_COMMAND(MenyVisaSerienummer, OnMenyVisaSerienummer)
ON_UPDATE_COMMAND_UI(MenyVisaSerienummer, OnUpdateMenyVisaSerienummer)
ON_COMMAND(MenyVisaDefaultanpassning, OnMenyVisaDefaultanpassning)
ON_UPDATE_COMMAND_UI(MenyVisaDefaultanpassning, OnUpdateMenyVisaDefaultanpassning)
ON_COMMAND(MenyVisaDefaultTekniker, OnMenyVisaDefaultTekniker)
ON_UPDATE_COMMAND_UI(MenyVisaDefaultTekniker, OnUpdateMenyVisaDefaultTekniker)
ON_COMMAND(MenyArkivAvsluta, OnMenyArkivAvsluta)
ON_COMMAND(MenyHjalpOm, OnMenyHjalpOm)
ON_COMMAND(MenyArkivSkriv, OnMenyArkivSkriv)
ON_COMMAND(MenyArkivLas, OnMenyArkivLas)
ON_COMMAND(MenyVisaDefaultProduktion, OnMenyVisaDefaultProduktion)
ON_UPDATE_COMMAND_UI(MenyVisaDefaultProduktion, OnUpdateMenyVisaDefaultProduktion)
ON_BN_CLICKED(IDC_Skriv16byteKnapp, OnSkriv16byteKnapp)
ON_BN_CLICKED(IDC_SparaloggfilKnapp, OnSparaloggfilKnapp)
ON_EN_CHANGE(IDC_Antalcellerruta, OnChangeAntalcellerruta)
ON_EN_CHANGE(IDC_AmperetimmarRuta, OnChangeAmperetimmarRuta)
ON_CBN_CLOSEUP(IDC_LaddningskurvaCombo, OnCloseupLaddningskurvaCombo)
ON_CBN_CLOSEUP(IDC_ProgramCombo, OnCloseupProgramCombo)
ON_EN_CHANGE(IDC_HardvaruversionRuta, OnChangeHardvaruversionRuta)
ON_EN_CHANGE(IDC_KalibreringsdataRuta, OnChangeKalibreringsdataRuta)
ON_EN_CHANGE(IDC_SerienummerRuta, OnChangeSerienummerRuta)
ON_EN_CHANGE(IDC_LaddartypVoltRuta, OnChangeLaddartypVoltRuta)
ON_EN_CHANGE(IDC_LaddartypAmpereRuta, OnChangeLaddartypAmpereRuta)
ON_EN_CHANGE(IDC_GrundladdartypAmpereRuta, OnChangeGrundladdartypAmpereRuta)
ON_EN_CHANGE(IDC_GrundladdartypVoltRuta, OnChangeGrundladdartypVoltRuta)
ON_EN_CHANGE(IDC_AdressRuta, OnChangeAdressRuta)
ON_EN_CHANGE(IDC_DataRuta0, OnChangeDataRuta0)
ON_EN_CHANGE(IDC_DataRuta1, OnChangeDataRuta1)
ON_EN_CHANGE(IDC_DataRuta10, OnChangeDataRuta10)
ON_EN_CHANGE(IDC_DataRuta11, OnChangeDataRuta11)
ON_EN_CHANGE(IDC_DataRuta12, OnChangeDataRuta12)
ON_EN_CHANGE(IDC_DataRuta13, OnChangeDataRuta13)
ON_EN_CHANGE(IDC_DataRuta14, OnChangeDataRuta14)
ON_EN_CHANGE(IDC_DataRuta15, OnChangeDataRuta15)
ON_EN_CHANGE(IDC_DataRuta2, OnChangeDataRuta2)
ON_EN_CHANGE(IDC_DataRuta3, OnChangeDataRuta3)
ON_EN_CHANGE(IDC_DataRuta4, OnChangeDataRuta4)
ON_EN_CHANGE(IDC_DataRuta5, OnChangeDataRuta5)
ON_EN_CHANGE(IDC_DataRuta6, OnChangeDataRuta6)
ON_EN_CHANGE(IDC_DataRuta7, OnChangeDataRuta7)
ON_EN_CHANGE(IDC_DataRuta8, OnChangeDataRuta8)
ON_EN_CHANGE(IDC_DataRuta9, OnChangeDataRuta9)
ON_COMMAND(ID_ArkivSattKatalog, OnArkivSattKatalog)
ON_BN_CLICKED(IDC_Las16byte, OnLas16byte)
ON_COMMAND(MenyArkivSkrivochSpara, OnMenyArkivSkrivochSpara)
ON_WM_MOUSEMOVE()
//}}AFX_MSG_MAP
ON_MESSAGE(WM_KICKIDLE, OnKickIdle)
END_MESSAGE_MAP()

```

```

// CLaddingST7Dlg message handlers

BOOL CLaddingST7Dlg::OnInitDialog()
{
    CDialog::OnInitDialog(); // Initiering av programmet
    // Add "About..." menu item to system menu.
    // IDM_ABOUTBOX must be in the system command range.
    ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
    ASSERT(IDM_ABOUTBOX < 0xF000);
    CMenu* pSysMenu = GetSystemMenu(FALSE);
    if (pSysMenu != NULL)
    {
        CString strAboutMenu;
        strAboutMenu.LoadString(IDS_ABOUTBOX);
        if (!strAboutMenu.IsEmpty())
        {
            pSysMenu->AppendMenu(MF_SEPARATOR);
            pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX, strAboutMenu);
        }
    }
    // Set the icon for this dialog. The framework does this automatically
    // when the application's main window is not a dialog
    SetIcon(m_hIcon, TRUE); // Set big icon
    SetIcon(m_hIcon, FALSE); // Set small icon
    // TODO: Add extra initialization here
    m_felfil = FALSE;
    m_felfil2 = FALSE;
    m_adressfilfel1 = FALSE;
    m_adressfilfel2 = FALSE;

    HANDLE hFile;
    WIN32_FIND_DATA* lpFileData;
    lpFileData = new WIN32_FIND_DATA;

    //Initiering av Parallellporten
    IO1.Open(1);
    IO1.SetECPMode2(2);
    //Starta med senaste inställningarna för de olika fönsteralternativen.
    bool data;
    byte antal;
    ifstream startfil("startfil.txt", ios::in );
    for (antal=0; antal < 11; antal++){
        startfil.seekg(antal*sizeof(data));
        startfil.read((char *) &data , sizeof(data));
        switch(antal){
            case 0: m_LaddningskurvaCheck = data; break;
            case 1: m_AntalcellerCheck = data; break;
            case 2: m_AmperetimmarCheck = data; break;
            case 3: m_ProgramCheck = data; break;
            case 4: m_HardvaruversionCheck = data; break;
            case 5: m_KalibreringsdataCheck = data; break;
            case 6: m_SerienummerCheck = data; break;
            case 7: m_LaddartypCheck = data; break;
            case 8: m_GrundladdartypCheck = data; break;
            case 9: m_DataCheck = data; break;
            case 10: m_Sparautanloggfilcheck = data; break;
        } // for (antal=0; antal < 11; antal++){
        UpdateData(FALSE);
    }
    // Tända eller släcka resp. ruta eller knapp.

```



```

GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
GetDlgItem(IDC_ProgramCombo) -> EnableWindow(m_ProgramCheck);
GetDlgItem(IDC_LaddningskurvaCombo) -> EnableWindow(m_LaddningskurvaCheck);
GetDlgItem(IDC_LaddartypAmpereRuta) -> EnableWindow(m_LaddartypCheck);
GetDlgItem(IDC_LaddartypVoltRuta) -> EnableWindow(m_LaddartypCheck);
GetDlgItem(IDC_KalibreringsdataRuta) -> EnableWindow(m_KalibreringsdataCheck);
GetDlgItem(IDC_HardvaruversionRuta) -> EnableWindow(m_HardvaruversionCheck);
GetDlgItem(IDC_GrundladdartypAmpereRuta) -> EnableWindow(m_GrundladdartypCheck);
GetDlgItem(IDC_GrundladdartypVoltRuta) -> EnableWindow(m_GrundladdartypCheck);
GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
GetDlgItem(IDC_AdressRuta) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta0) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta1) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta2) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta3) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta4) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta5) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta6) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta7) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta8) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta9) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta10) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta11) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta12) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta13) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta14) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_DataRuta15) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_Skriv16byteKnapp) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_Las16byte) -> EnableWindow(m_DataCheck);
GetDlgItem(IDC_SkrivKnapp) -> EnableWindow(m_Sparautanloggfildcheck);
GetDlgItem(IDC_DATETIMEPICKER1) -> ShowWindow(FALSE);
//Starta med de senaste sökvägarna till program,laddningskurva och loggfil
//En textfil som ligger i samma katalog som .exe filen.
//Skapas i programmet anteckningar.
CStdioFile file;
BOOL fSuccess;
fSuccess = file.Open(
    _T("Sokvag.txt"),
    CFile::modeRead |
    CFile::shareDenyRead |
    CFile::typeText
);
if(!fSuccess){
    m_felfil = TRUE;
    m_felfil2 = TRUE;
}
CString Filtext;
m_HittatAdress = FALSE;
if (m_felfil == FALSE){
    while (file.ReadString(Filtext))
    {
        if((Filtext.GetAt(0) == 'L')&& (Filtext.GetAt(1) == 'A')){
            file.ReadString(Filtext);
            m_LaddningskurvaSokvag = Filtext;
        }
        if((Filtext.GetAt(0) == 'P')&& (Filtext.GetAt(1) == 'R')){
            file.ReadString(Filtext);
            m_ProgramSokvag = Filtext;
        }
    }
}

```

```

        UpdateData(FALSE);
    }
    if((Filtext.GetAt(0) == 'A') && (Filtext.GetAt(1) == 'D')){
        file.ReadString(Filtext);
        m_AdressSokvag = Filtext;
        UpdateData(FALSE);
    }
    if((Filtext.GetAt(0) == 'L') && (Filtext.GetAt(1) == 'O')){
        file.ReadString(Filtext);
        m_LoggfilSokvag = Filtext;
        UpdateData(FALSE);
    }
} //while (file.ReadString(Filtext))
//Uppdatering av combobox för Laddningsskurvor.
CString fil;
bool HittatFiler = FALSE;
fil = m_LaddningskurvaSokvag + ".*.*";
hFile = ::FindFirstFile(fil, lpFileData);
::FindNextFile(hFile, lpFileData);
while(::FindNextFile(hFile, lpFileData)){
    lpFileData->cFileName;
    m_Laddningskurva.AddString(lpFileData->cFileName);
    HittatFiler = TRUE;
}
if(HittatFiler == FALSE){
    m_Felmeddelande = " FEL NR: 1.Felaktig katalog eller tom katalog för laddningskurva";
    UpdateData(FALSE);
}
::FindClose(hFile);

//Uppdatering av combobox för Program.
HittatFiler = FALSE;
lpFileData = new WIN32_FIND_DATA;
fil = m_ProgramSokvag + ".*.*";
hFile = ::FindFirstFile(fil, lpFileData);
::FindNextFile(hFile, lpFileData);
while(::FindNextFile(hFile, lpFileData)){
    lpFileData->cFileName;
    m_ProgramCombo.AddString(lpFileData->cFileName);
    HittatFiler = TRUE;
}
if(HittatFiler == FALSE){
    m_Felmeddelande += "FEL NR: 2. Felaktig katalog eller tom katalog för program";
    UpdateData(FALSE);
}
::FindClose(hFile);

//Hämta in flashminnes-adresser från adressfil till de olika parametrarna
fil = m_AdressSokvag + "AdressST7.txt";
fSuccess = file.Open(
    _T(fil),
    CFile::modeRead |
    CFile::shareDenyRead |
    CFile::typeText
);
if(!fSuccess){
    m_adressfilfel1 = TRUE;
    m_adressfilfel2 = TRUE;
}
CString adress1, adress2;

```

```

if(m_adressfilfel1 == FALSE){
  while (file.ReadString(Filtext) ){
    if((Filtext.GetAt(0) == 'P')&& (Filtext.GetAt(1) == 'R') && (Filtext.GetAt(6) == 'M')){
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Programnamnstart = adress1;
      file.ReadString(Filtext);
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Optiosbyte = adress1;
      file.ReadString(Filtext);
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Programstorlek = adress1;
    }
    if((Filtext.GetAt(0) == 'L')&& (Filtext.GetAt(1) == 'A') && (Filtext.GetAt(9) == 'K')){
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Laddningskurvanamnstart = adress1;
      file.ReadString(Filtext);
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Laddningskurvastorlek = adress1;
    }
    if((Filtext.GetAt(0) == 'G')&& (Filtext.GetAt(1) == 'R') && (Filtext.GetAt(9) == 'A')){
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Grundladdartypstart = adress1;
    }
    if((Filtext.GetAt(0) == 'L')&& (Filtext.GetAt(1) == 'A') && (Filtext.GetAt(8) == 'P')){
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
      adress1 += Filtext.GetAt(3);
      m_Laddartypstart = adress1;
    }
    if((Filtext.GetAt(0) == 'A')&& (Filtext.GetAt(1) == 'N') && (Filtext.GetAt(8) == 'L')){
      file.ReadString(Filtext);
      adress1 = Filtext.GetAt(0);
      adress1 += Filtext.GetAt(1);
      adress1 += Filtext.GetAt(2);
    }
  }
}

```

```

        adress1 += Filtext.GetAt(3);
        m_Antalcellerstart = adress1;
    }
    if((Filtext.GetAt(0) == 'H') && (Filtext.GetAt(2) == 'R') && (Filtext.GetAt(8) == 'V')){
        file.ReadString(Filtext);
        adress1 = Filtext.GetAt(0);
        adress1 += Filtext.GetAt(1);
        adress1 += Filtext.GetAt(2);
        adress1 += Filtext.GetAt(3);
        m_Hardvaruversionstart = adress1;
    }
    if((Filtext.GetAt(0) == 'S') && (Filtext.GetAt(2) == 'R') && (Filtext.GetAt(8) == 'M')){
        file.ReadString(Filtext);
        adress1 = Filtext.GetAt(0);
        adress1 += Filtext.GetAt(1);
        adress1 += Filtext.GetAt(2);
        adress1 += Filtext.GetAt(3);
        m_Serienummerstart = adress1;
    }
    if((Filtext.GetAt(0) == 'T') && (Filtext.GetAt(2) == 'T') && (Filtext.GetAt(3) == 'E')){
        file.ReadString(Filtext);
        adress1 = Filtext.GetAt(0);
        adress1 += Filtext.GetAt(1);
        adress1 += Filtext.GetAt(2);
        adress1 += Filtext.GetAt(3);
        m_Interupptabellstart = adress1;
        adress2 = Filtext.GetAt(5);
        adress2 += Filtext.GetAt(6);
        adress2 += Filtext.GetAt(7);
        adress2 += Filtext.GetAt(8);
        m_Interupptabellstorlek = adress2;
    }
    } //while (file.ReadString(Filtext) ){
} //if(m_adressfilfel1 == FALSE){
else{
    m_Felmeddelande = "FEL NR: 3. Fel på katalog eller tom katalog för adresser.";
}
} //if(m_felfil == FALSE)
UpdateData(FALSE);
//Startvärde på att man inte har skrivit i en ruta
BOOL m_ProgramChanged = FALSE;
BOOL m_SerienummerChanged = FALSE;
BOOL m_KalibreringsdataChanged = FALSE;
BOOL m_LaddartypVoltChanged = FALSE;
BOOL m_LaddartypAmpereChanged = FALSE;
BOOL m_GrundladdartypAmpereChanged = FALSE;
BOOL m_GrundladdartypVoltChanged = FALSE;
BOOL m_HardvaruversionChanged = FALSE;
BOOL m_LaddningskurvaChanged = FALSE;
BOOL m_AmperetimmarChanged = FALSE;
BOOL m_AntalCellerChanged = FALSE;
return TRUE; // return TRUE unless you set the focus to a control
}

void CLaddningST7Dlg::OnSysCommand(UINT nID, LPARAM lParam)
{
    if ((nID & 0xFFF0) == IDM_ABOUTBOX)
    {
        CAboutDlg dlgAbout;
        dlgAbout.DoModal();
    }
}

```

```

    }
    else
    {
        CDialog::OnSysCommand(nID, lParam);
    }
}

// If you add a minimize button to your dialog, you will need the code below
// to draw the icon. For MFC applications using the document/view model,
// this is automatically done for you by the framework.
void CLaddningST7Dlg::OnPaint()
{
    if (IsIconic())
    {
        CPaintDC dc(this); // device context for painting
        SendMessage(WM_ICONERASEBKGND, (WPARAM) dc.GetSafeHdc(), 0);
        // Center icon in client rectangle
        int cxIcon = GetSystemMetrics(SM_CXICON);
        int cyIcon = GetSystemMetrics(SM_CYICON);
        CRect rect;
        GetClientRect(&rect);
        int x = (rect.Width() - cxIcon + 1) / 2;
        int y = (rect.Height() - cyIcon + 1) / 2;
        // Draw the icon
        dc.DrawIcon(x, y, m_hIcon);
    }
    else
    {
        CDialog::OnPaint();
    }
}

// The system calls this to obtain the cursor to display while the user drags
// the minimized window.
HCURSOR CLaddningST7Dlg::OnQueryDragIcon()
{
    return (HCURSOR) m_hIcon;
}

bool CLaddningST7Dlg::SkickaData(CString Data)
{
    //Omvandla från CString Data till byte
    byte high, low, temp;
    for(int i=0; i<32; i++){
        if (Data.GetAt(i) > 64)
            high = Data.GetAt(i) - 55;
        else
            high = Data.GetAt(i) - 48;
        i++;
        if (Data.GetAt(i) > 64)
            low = Data.GetAt(i) - 55;
        else
            low = Data.GetAt(i) - 48;

        high = high << 4;
        temp = high + low;
        if(SkickaByte(temp) == 0)
            return 0;
    }
    return 1;
}

```

```

}

bool CLaddningST7Dlgl::SkickaAdress(CString Adress)
{
    //Omvandla från CString Adress till byte.
    BYTE high,low,temp;
    if (Adress.GetAt(0) > 64)
        high = Adress.GetAt(0) - 55;
    else
        high = Adress.GetAt(0) - 48;

    if (Adress.GetAt(1) > 64)
        low = Adress.GetAt(1) - 55;
    else
        low = Adress.GetAt(1) - 48;

    high = high << 4;
    temp = high + low;
    if(SkickByte(temp) == 0)
        return 0;
    else{
        if (Adress.GetAt(2) > 64)
            high = Adress.GetAt(2) - 55;
        else
            high = Adress.GetAt(2) - 48;

        if (Adress.GetAt(3) > 64)
            low = Adress.GetAt(3) - 55;
        else
            low = Adress.GetAt(3) - 48;

        high = high << 4;
        temp = high + low;
        return SkickaByte(temp);
    } //if(SkickByte(temp) == 0)
}

bool CLaddningST7Dlgl::SkickaByte(BYTE temp)
{
    int raknare = 0;
    IO1.WriteData(temp); //Kommando för reset
    IO1.WriteControl(0+0+0+0+0+0+2+1); //Sätter autolinefeed hög (data). Kontrollregistret
    IO1.WriteControl(0+0+0+0+0+0+2+0); //Sätter strobe låg
    m_Status = IO1.ReadStatus(); //Statusregistret
    while(((m_Status & 0x80) >> 7) == 0 && raknare <= 10000){ //busy ska gå hög inom
        m_Status = IO1.ReadStatus(); // en viss tid

        raknare ++;
    }
    if(raknare < 10000){
        raknare = 0;
        IO1.WriteControl(0+0+0+0+0+0+2+1); //Sätter strobe hög. Kontrollregistret
        while(((m_Status & 0x80) >> 7) == 1 && raknare <= 10000) { //busy ska gå låg inom
            m_Status = IO1.ReadStatus(); //Statusregistret //en viss tid
            raknare ++;
        }
        if(raknare < 10000){
            IO1.WriteControl(0+0+0+0+0+0+2+1); //Sätter strobe hög. Kontrollregistret
        }
        else {

```

```

        return 0;
    }
}
else
    return 0;
return 1;
}

int CLaddningST7Dlg::Kontroll0till9(CString variabel)
{
    //Kontroll av att tecken i CString är mellan 0 till 9
    int langd = 0;
    langd = variabel.GetLength();
    for (int i = 0; i < langd; i++){
        char tecken;
        tecken = variabel.GetAt(i);
        if( tecken >='0' && tecken <='9');
        else
            return 0;
    }
    return 1;
}

int CLaddningST7Dlg::Kontroll0till9ellerAtillF(CString data)
{
    //Kontroll av att tecken i CString är mellan 0 till 9 eller A till F
    int langd = 0;
    langd = data.GetLength();
    for (int i = 0; i < langd; i++){
        char tecken;
        tecken = data.GetAt(i);
        if( (tecken >='0' && tecken <='9') || (tecken >='A' && tecken <='F'));
        else
            return 0;
    }
    return 1;
}

CString CLaddningST7Dlg::FranInttillHex(CString data)
{
    //Omvandla från Integer till hex
    BYTE hex, low, high, tal;

    if (data.GetAt(0) > 64)
        tal = data.GetAt(0) - 55;
    else
        tal = data.GetAt(0) - 48;
    hex = tal*100;
    if (data.GetAt(1) > 64)
        tal = data.GetAt(1) - 55;
    else
        tal = data.GetAt(1) - 48;
    hex += tal*10;
    if (data.GetAt(2) > 64)
        tal = data.GetAt(2) - 55;
    else
        tal = data.GetAt(2) - 48;
    hex += tal;
    high = hex;
    high = high >> 4;
}

```

```

low = hex;
low = (low & 0x0F);
if(high>9)
    high = high + 55;
else
    high = high + 48;
if(low>9)
    low = low + 55;
else
    low = low + 48;
data = high;
data += low;
return data;
}

bool CLaddingST7Dlg::OnSkrivKnapp()
{
    // TODO: Add your control notification handler code here
    if(m_felfil2 == FALSE && m_adressfil2 == FALSE){ //Inget fel i Initiering av sokvagfil
        UpdateData(TRUE); //eller adressfil
        m_Felmeddelande = "Skrivning lyckades!";
        UpdateData(FALSE);
        int test1=1,test2=1,test3=1,test4=1,test5=1,test6=1,test7=1,test8=1,test9=1,test10=1, test11=1, andrat=0;
        int test12=1,test13=1,test14=1,test15=1,test16=1,test17=1,test18=1,test19=1,test20=1;
        if (m_AntalCellerChanged == TRUE){ //Ändrat i editruta
            if(m_AntalCeller.GetLength() != 0)
                andrat++;
            else
                m_AntalCellerChanged = FALSE;
            test1 = Kontroll0till9(m_Antalceller); //test av värdet i rutan
            test12 = KontrollMaxvarde(m_Antalceller, 60); //test av maxvärde
        }
        if (m_AmperetimmarChanged == TRUE){
            if(m_Amperetimmar.GetLength() != 0)
                andrat++;
            else
                m_AmperetimmarChanged = FALSE;
            test2 = Kontroll0till9(m_Amperetimmar);
            test13 = KontrollMaxvarde(m_Amperetimmar, 255);
        }
        if (m_HardvaruversionChanged == TRUE){
            if(m_Hardvaruversion.GetLength() != 0)
                andrat++;
            else
                m_HardvaruversionChanged = FALSE;
            test3 = Kontroll0till9(m_Hardvaruversion);
            test14 = KontrollMaxvarde(m_Hardvaruversion, 10);
        }
        if (m_KalibreringsdataChanged == TRUE){
            if(m_Kalibreringsdata.GetLength() != 0)
                andrat++;
            else
                m_KalibreringsdataChanged = FALSE;
            test4 = Kontroll0till9(m_Kalibreringsdata);
            test15 = KontrollMaxvarde(m_Kalibreringsdata, 10);
        }
        if (m_SerienummerChanged == TRUE){
            if(m_Serienummer.GetLength() != 0)
                andrat++;
            else

```



```

        m_SerienummerChanged = FALSE;
        test5 = Kontroll0till9(m_Serienummer);
    }
    if (m_LaddartypVoltChanged == TRUE){
        if(m_LaddartypVolt.GetLength() != 0)
            andrat++;
        else
            m_LaddartypVoltChanged = FALSE;
        test6 = Kontroll0till9(m_LaddartypVolt);
        test16 = KontrollMaxvarde(m_LaddartypVolt, 120);
    }
    if (m_LaddartypAmpereChanged == TRUE){
        if(m_LaddartypAmpere.GetLength() != 0)
            andrat++;
        else
            m_LaddartypAmpereChanged = FALSE;
        test7 = Kontroll0till9(m_LaddartypAmpere);
        test17 = KontrollMaxvarde(m_LaddartypAmpere, 255);
    }
    if (m_GrundladdartypVoltChanged == TRUE){
        if(m_GrundladdartypVolt.GetLength() != 0)
            andrat++;
        else
            m_GrundladdartypVoltChanged = FALSE;
        test8 = Kontroll0till9(m_GrundladdartypVolt);
        test18 = KontrollMaxvarde(m_GrundladdartypVolt, 120);
    }
    if (m_GrundladdartypAmpereChanged == TRUE){
        if(m_GrundladdartypAmpere.GetLength() != 0)
            andrat++;
        else
            m_GrundladdartypAmpereChanged = FALSE;
        test9 = Kontroll0till9(m_GrundladdartypAmpere);
        test19 = KontrollMaxvarde(m_GrundladdartypAmpere, 255);
    }
}

if (m_LaddningskurvaChanged == TRUE){ //något valt i combobox
    andrat++;
    int h=m_Laddningskurva.GetCurSel();
    if (h == CB_ERR){
        test10 = 0;
        m_LaddningskurvaChanged = FALSE;
        GetDlgItem(IDC_TextLaddningskurva) -> EnableWindow(TRUE);
    }
    else
        test10 = 1;
}

if (m_ProgramChanged == TRUE){
    andrat++;
    int h=m_ProgramCombo.GetCurSel();
    if (h == CB_ERR){
        test11 = 0;
        m_ProgramChanged = FALSE;
        GetDlgItem(IDC_TextProgram) -> EnableWindow(TRUE);
    }
    else
        test11 = 1;
}

//test av att Laddartyp <= Grundladdartyp
int tal1=0, tal2=0, tal3=0, tal4=0, langd=0;

```

```

langd = m_LaddartypVolt.GetLength();
if(langd == 3)
    tal1 = (m_LaddartypVolt.GetAt(0) - 48) * 100 +
            (m_LaddartypVolt.GetAt(1) - 48) * 10 + (m_LaddartypVolt.GetAt(2) - 48);
if(langd == 2)
    tal1 = (m_LaddartypVolt.GetAt(0) - 48) * 10 + (m_LaddartypVolt.GetAt(1) - 48);
if(langd == 1)
    tal1 = (m_LaddartypVolt.GetAt(0) - 48);
langd = m_GrundladdartypVolt.GetLength();
if(langd == 3)
    tal2 = (m_GrundladdartypVolt.GetAt(0) - 48) * 100 +
            (m_GrundladdartypVolt.GetAt(1) - 48) * 10 + (m_GrundladdartypVolt.GetAt(2) - 48);
if(langd == 2)
    tal2 = (m_GrundladdartypVolt.GetAt(0) - 48) * 10 + (m_GrundladdartypVolt.GetAt(1) - 48);
if(langd == 1)
    tal2 = (m_GrundladdartypVolt.GetAt(0) - 48);
langd = m_LaddartypAmpere.GetLength();
if(langd == 3)
    tal3 = (m_LaddartypAmpere.GetAt(0) - 48) * 100 +
            (m_LaddartypAmpere.GetAt(1) - 48) * 10 + (m_LaddartypAmpere.GetAt(2) - 48);
if(langd == 2)
    tal3 = (m_LaddartypAmpere.GetAt(0) - 48) * 10 + (m_LaddartypAmpere.GetAt(1) - 48);
if(langd == 1)
    tal3 = (m_LaddartypAmpere.GetAt(0) - 48);
langd = m_GrundladdartypAmpere.GetLength();
if(langd == 3)
    tal4 = (m_GrundladdartypAmpere.GetAt(0) - 48) * 100 +
            (m_GrundladdartypAmpere.GetAt(1) - 48) * 10 + (m_GrundladdartypAmpere.GetAt(2) - 48);
if(langd == 2)
    tal4 = (m_GrundladdartypAmpere.GetAt(0) - 48) * 10 + (m_GrundladdartypAmpere.GetAt(1) - 48);
if(langd == 1)
    tal4 = (m_GrundladdartypAmpere.GetAt(0) - 48);
test20 = test1 + test2 + test3 + test4 + test5 + test6 + test7 + test8 + test9 + test10 + test11;
test20 += test12 + test13 + test14 + test15 + test16 + test17 + test18 + test19;
if (test20 == 19){ //alla tester är OK!
    if(Andrat != 0){ //något nytt att skicka
        if (tal1 <= tal2 && tal3 <= tal4){ //Laddartyp <= Grundladdartyp
            //Reset kommando
            if(SkickaKommando(1) == 1){
                //Adress och data
                CString buffer1, buffer2, buffer3;
                UpdateData(TRUE);
                if ((m_AmperetimmarChanged == TRUE) || (m_AntalCellerChanged == TRUE)){
                    buffer1 = Buffer(m_Antalceller);
                    buffer1 = FranInttillHex(buffer1);
                    buffer2 = Buffer(m_Amperetimmar);
                    buffer2 = FranInttillHex(buffer2);
                    buffer3 = TotBuffer(buffer1, buffer2, 2);
                    if(SkickaKommando(2) == 1){
                        if(SkickaAdress(m_Antalcellerstart) == 1){
                            if(SkickaData(buffer3) == 0){
                                m_Felmeddelande = "FEL NR: 4.Kontakten bröts, skrivning avslutades.";
                                UppdateraFonster();//SkickaData-fel
                                return 0;
                            }
                        }
                    }
                }
            }
            else{
                m_Felmeddelande = "FEL NR: 5. Kontakten bröts, skrivning avslutades.";
                UppdateraFonster();
                return 0;
            }
        }
    }
}

```

```

    }
}
else{
    m_Felmeddelande = "FEL NR: 6. Kontakten bröts, skrivning avslutades.";
    UppdateraFonster();
    return 0;
}
}
if ((m_LaddartypVoltChanged == TRUE) || (m_LaddartypAmpereChanged == TRUE)){
    buffer1 = Buffer(m_LaddartypVolt);
    buffer1 = FranInttillHex(buffer1);
    buffer2 = Buffer(m_LaddartypAmpere);
    buffer2 = FranInttillHex(buffer2);
    buffer3 = TotBuffer(buffer1, buffer2,2);
    if(SkickaKommando(2) == 1){
        if(SkickaAdress(m_Laddartypstart) == 1){
            if(SkickaData(buffer3) == 0){
                m_Felmeddelande = "FEL NR: 7. Kontakten bröts, skrivning avslutades.";
                UppdateraFonster();
                return 0;
            }
        }
        else{
            m_Felmeddelande = "FEL NR: 8. Kontakten bröts, skrivning avslutades.";
            UppdateraFonster();
            return 0;
        }
    }
}
else{
    m_Felmeddelande = "FEL NR: 9. Kontakten bröts, skrivning avslutades.";
    UppdateraFonster();
    return 0;
}
}
if ((m_GrundladdartypVoltChanged == TRUE)
|| (m_GrundladdartypAmpereChanged == TRUE)){
    buffer1 = Buffer(m_GrundladdartypVolt);
    buffer1 = FranInttillHex(buffer1);
    buffer2 = Buffer(m_GrundladdartypAmpere);
    buffer2 = FranInttillHex(buffer2);
    buffer3 = TotBuffer(buffer1, buffer2,2);
    if(SkickaKommando(2) == 1){
        if(SkickaAdress(m_Grundladdartypstart) == 1){
            if(SkickaData(buffer3) == 0){
                m_Felmeddelande = "FEL NR: 10. Kontakten bröts, skrivning avslutades.";
                UppdateraFonster();
                return 0;
            }
        }
        else{
            m_Felmeddelande = "FEL NR: 11. Kontakten bröts, skrivning avslutades.";
            UppdateraFonster();
            return 0;
        }
    }
}
else{
    m_Felmeddelande = "FEL NR: 12. Kontakten bröts, skrivning avslutades.";
    UppdateraFonster();
    return 0;
}
}

```

```

    }

    if (m_HardvaruversionChanged == TRUE){
        buffer1 = Buffer(m_Hardvaruversion);
        buffer1 = FranInttillHex(buffer1);
        buffer3 = TotBuffer(buffer1, buffer1,1);
        if(SkickaKommando(2) == 1){
            if(SkickaAdress(m_Hardvaruversionstart) == 1){
                if(SkickaData(buffer3) == 0){
                    m_Felmeddelande = "FEL NR: 13. Kontakten bröts, skrivning avslutades.";
                    UppdateraFonster();
                    return 0;
                }
            }
        }
        else{
            m_Felmeddelande = "FEL NR: 14. Kontakten bröts, skrivning avslutades.";
            UppdateraFonster();
            return 0;
        }
    }
    else{
        m_Felmeddelande = "FEL NR: 15. Kontakten bröts, skrivning avslutades.";
        UppdateraFonster();
        return 0;
    }
}

if (m_SerienummerChanged == TRUE){
    int byte;
    int h = m_Serienummer.GetLength();
    if (h == 8){ //test av serienummerlängd
        if(SkickaKommando(2) == 1){
            if(SkickaAdress(m_Serienummerstart) == 1){
                for(int i=0; i<8; i++){
                    byte = m_Serienummer.GetAt(i);
                    if(SkickaByte(byte) == 0){
                        m_Felmeddelande = "FEL NR: 16. Kontakten bröts, skrivning avslutades.";
                        UppdateraFonster();
                        return 0;
                    }
                }
                //for(int i=0; i<8; i++){
                byte = 255;
                for(i=0; i<8;i++){
                    if(SkickaByte(byte) == 0){
                        m_Felmeddelande = "FEL NR: 17. Kontakten bröts, skrivning avslutades.";
                        UppdateraFonster();
                        return 0;
                    }
                }
            }
        } //if(SkickaAdress(m_Serienummerstart) == 1){
        else{
            m_Felmeddelande = "FEL NR: 18. Kontakten bröts, skrivning avslutades.";
            UppdateraFonster();
            return 0;
        }
    }
} //if(SkickaKommando(2) == 1){
else{
    m_Felmeddelande = "FEL NR: 19. Kontakten bröts, skrivning avslutades.";
    UppdateraFonster();
}

```

```

        return 0;
    }
} //if (h == 8){
else
    m_Felmeddelande = "FEL NR: 20. Felaktigt serienummer!";
}

/*if (m_KalibreringsdataChanged == TRUE){
    SkickaAdress("DD");
    SkickaAdress("DD");
    SkickaData(m_Kalibreringsdata);
}*/

if (m_LaddningskurvaChanged == TRUE){
    int h = m_Laddningskurva.GetCurSel();
    if(h != CB_ERR){
        CString filnamn,tempfilnamn;
        CString adressdata;
        int antal = 0;
        BOOL fSuccess;
        tempfilnamn =m_LaddningskurvaSokvag;
        m_Laddningskurva.GetLBText(h,filnamn);
        tempfilnamn += filnamn;
        CStdioFile file;
        fSuccess = file.Open(
            _T(tempfilnamn),
            CFile::modeRead |
            CFile::shareDenyRead |
            CFile::typeText
        );

        if(!fSuccess){
            m_Felmeddelande = "FEL NR: 21. Laddningskurvafilläsning misslyckad";
        }
        CString Filtext;
        while (file.ReadString(Filtext)) //tills filslut eller tom rad
        {
            if(SkickaKommando(2) == 1){
                Filtext.Delete(0);
                Filtext.Delete(0);
                Filtext.Delete(0);
                Filtext.Delete(0);
                Filtext.Delete(Filtext.GetLength()-1);
                Filtext.Delete(Filtext.GetLength()-1);

                antal = Filtext.GetLength();
                int ggr, j=0;
                for (ggr=0; antal>3; ggr++){
                    antal=antal-4;
                }
                for (int i = 0; i < ggr; i++){
                    adressdata = Filtext.GetAt(j);
                    j++;
                    adressdata += Filtext.GetAt(j);
                    j++;
                    adressdata += Filtext.GetAt(j);
                    j++;
                    adressdata += Filtext.GetAt(j);
                    j++;
                    if(SkickaAdress(adressdata) == 0){
                        m_Felmeddelande = "FEL NR: 22. Kontakten bröts, skrivning

```

```

        avslutades.";
        UppdateraFonster();
        return 0;
    }
}
if(antal==1){
    adressdata = Filtext.GetAt(j);
    adressdata += "FFF";
    if(SkickaAdress(adressdata) == 0){
        m_Felmeddelande = "FEL NR: 23. Kontakten bröts, skrivning
        avslutades.";
        UppdateraFonster();
        return 0;
    }
    ggr++;
}
if(antal==2){
    adressdata = Filtext.GetAt(j);
    j++;
    adressdata += Filtext.GetAt(j);
    j++;
    adressdata += "FF";
    if(SkickaAdress(adressdata) == 0){
        m_Felmeddelande = "FEL NR: 24. Kontakten bröts, skrivning
        avslutades.";
        UppdateraFonster();
        return 0;
    }
    ggr++;
}
if(antal==3){
    adressdata = Filtext.GetAt(j);
    j++;
    adressdata += Filtext.GetAt(j);
    j++;
    adressdata += Filtext.GetAt(j);
    j++;
    adressdata += 'F';
    if(SkickaAdress(adressdata) == 0){
        m_Felmeddelande = "FEL NR: 25. Kontakten bröts, skrivning
        avslutades.";
        UppdateraFonster();
        return 0;
    }
    ggr++;
}
antal = (36 - 4*ggr)/4;
for(i=0; i<antal; i++){
    adressdata = "FFFF";
    if(SkickaAdress(adressdata) == 0){
        m_Felmeddelande = "FEL NR:26. Kontakten bröts, skrivning
        avslutades.";
        UppdateraFonster();
        return 0;
    }
}
} //if(SkickaKommando(2) == 1){
else{
    m_Felmeddelande = "FEL NR: 27. Kontakten bröts, skrivning avslutades.";
    UpdateData(FALSE);
}

```

```

        return 0;
    }
    } /// while (file.ReadString(Filtext))
    } //if(h != CB_ERR){
} //if (m_LaddningskurvaChanged == TRUE){
if (m_ProgramChanged == TRUE){
    int h = m_ProgramCombo.GetCurSel();
    if(h != CB_ERR){
        CString filnamn,tempfilnamn;
        CString adressdata;
        int antal = 0;
        BOOL fSuccess;
        tempfilnamn = m_ProgramSokvag;
        m_ProgramCombo.GetLBText(h,filnamn);
        tempfilnamn += filnamn;
        CStdioFile file;
        fSuccess = file.Open(
            _T(tempfilnamn),
            CFile::modeRead |
            CFile::shareDenyRead |
            CFile::typeText
        );

        if(!fSuccess){
            m_Felmeddelande = "FEL NR: 28. Programfilläsning misslyckad";
        }
        else { //fSuccess
            CString Filtext;
            while (file.ReadString(Filtext))
            {
                if(SkickaKommando(2) == 1){
                    Filtext.Delete(0);
                    Filtext.Delete(0);
                    Filtext.Delete(0);
                    Filtext.Delete(0);
                    Filtext.Delete(Filtext.GetLength()-1);
                    Filtext.Delete(Filtext.GetLength()-1);

                    antal = Filtext.GetLength();
                    int ggr, j=0;
                    for (ggr=0; antal>3; ggr++){
                        antal=antal-4;
                    }
                    for (int i = 0; i < ggr; i++){
                        adressdata = Filtext.GetAt(j);
                        j++;
                        adressdata += Filtext.GetAt(j);
                        j++;
                        adressdata += Filtext.GetAt(j);
                        j++;
                        adressdata += Filtext.GetAt(j);
                        j++;
                        if(SkickaAdress(adressdata) == 0){
                            m_Felmeddelande = "FEL NR: 29. Kontakten bröts, skrivning avslutades.";
                            UppdateraFonster();
                            return 0;
                        }
                    }
                }
            }
            if(antal==1){
                adressdata = Filtext.GetAt(j);

```

```

        adressdata += "FFF";
        if(SkickaAdress(adressdata) == 0){
            m_Felmeddelande = "FEL NR: 30. Kontakten bröts, skrivning
            avslutades.";
            UppdateraFonster();
            return 0;
        }
        ggr++;
    }
    if(antal==2){
        adressdata = Filtext.GetAt(j);
        j++;
        adressdata += Filtext.GetAt(j);
        j++;
        adressdata += "FF";
        if(SkickaAdress(adressdata) == 0){
            m_Felmeddelande = "FEL NR: 31. Kontakten bröts, skrivning
            avslutades.";
            UppdateraFonster();
            return 0;
        }
        ggr++;
    }
    if(antal==3){
        adressdata = Filtext.GetAt(j);
        j++;
        adressdata += Filtext.GetAt(j);
        j++;
        adressdata += Filtext.GetAt(j);
        j++;
        adressdata += 'F';
        if(SkickaAdress(adressdata) == 0){
            m_Felmeddelande = "FEL NR: 32. Kontakten bröts, skrivning
            avslutades.";
            UppdateraFonster();
            return 0;
        }
        ggr++;
    }
    antal = (36 - 4*ggr)/4;
    for(i=0; i<antal; i++){
        adressdata = "FFFF";
        if(SkickaAdress(adressdata) == 0){
            m_Felmeddelande = "FEL NR: 33. Kontakten bröts, skrivning
            avslutades.";
            UppdateraFonster();
            return 0;
        }
    }
} //if(SkickaKommando(2) == 1){
else{
    m_Felmeddelande = "FEL NR: 34. Kontakten bröts, skrivning
    avslutades.";
    UpdateData(FALSE);
    return 0;
}
} //while (file.ReadString(Filtext))
} //else { //fSuccess
} //if CB_ERR vid GetCurSel
} // if något changed

```



```

        UppdateraFonster();
        return 1;
    } //if(SkickaKommando(1) == 1){
    else{
        m_Felmeddelande = "FEL NR: 35. Kontakten bröts, skrivning avslutades.";
        UpdateData(FALSE);
        return 0;
    }
}
else //if tal1<tal2
    m_Felmeddelande = "Laddartyp > Grundladdartyp.";
}
else //if ändrat==0
    m_Felmeddelande = "Inget nytt att skriva.";
} //test20 == 19
else {
    m_Felmeddelande = "";
    Beep(100,100);
    if (test1 == 0 || test12 == 0)
        m_Felmeddelande += "FEL NR: 36. Felaktigt värde i Antal celler.";
    if (test2 == 0 || test13 == 0)
        m_Felmeddelande += "FEL NR: 37. Felaktigt värde i Amperetimmrar.";
    if (test3 == 0 || test14 == 0)
        m_Felmeddelande += "FEL NR: 38. Felaktigt värde i Hårdvaruversion.";
    if (test4 == 0 || test15 == 0)
        m_Felmeddelande += "FEL NR: 39. Felaktigt värde i Kalibreringsdata.";
    if (test5 == 0)
        m_Felmeddelande += "FEL NR: 40. Felaktigt värde i Serienummer.";
    if (test6 == 0 || test16 == 0)
        m_Felmeddelande += "FEL NR: 41. Felaktigt värde i Laddartyp Volt.";
    if (test7 == 0 || test17 == 0)
        m_Felmeddelande += "FEL NR: 42. Felaktigt värde i Laddartyp Ampere.";
    if (test8 == 0 || test18 == 0)
        m_Felmeddelande += "FEL NR: 43. Felaktigt värde i Grundladdartyp Volt.";
    if (test9 == 0 || test19 == 0)
        m_Felmeddelande += "FEL NR: 44. Felaktigt värde i Grundladdartyp Ampere.";
    if (test10 == 0)
        m_Felmeddelande += "FEL NR: 45. Felaktigt värde i Laddningskurva.";
    if (test11 == 0)
        m_Felmeddelande += "FEL NR: 46. Felaktigt värde i Program.";
} //test20 != 19
} //if(m_felfil2 == FALSE && m_adressfilfel2 == FALSE){
else
    m_Felmeddelande = "FEL NR: 47. Program i felläge. Adresser till parametrar kan ej hittas";
UpdateData(FALSE);
return 0;
} //OnSkrivKnapp

```

```

LRESULT CLaddningST7Dlg::OnKickIdle(WPARAM, LPARAM)
{
    CMenu* pMainMenu = GetMenu();
    CCmdUI cmdUI;
    for (UINT n = 0; n < pMainMenu->GetMenuItemCount(); ++n)
    {
        CMenu* pSubMenu = pMainMenu->GetSubMenu(n);
        cmdUI.m_nIndexMax = pSubMenu->GetMenuItemCount();
        for (UINT i = 0; i < cmdUI.m_nIndexMax; ++i)
        {
            cmdUI.m_nIndex = i;
            cmdUI.m_nID = pSubMenu->GetMenuItemID(i);

```

```

    cmdUI.m_pMenu = pSubMenu;
    cmdUI.DoUpdate(this, FALSE);
}
}
return TRUE;
}

//Val av olika alternativ i menyn. Sparas i SkrivStartfil()
////////////////////////////////////
void CLaddningST7Dlg::OnMenyVisaAmperetimmer()
{
    // TODO: Add your command handler code here
    m_AmperetimmerCheck = !m_AmperetimmerCheck;
    GetDlgItem(IDC_AmperetimmerRuta) -> EnableWindow(m_AmperetimmerCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaAmperetimmer(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_AmperetimmerCheck);
}

void CLaddningST7Dlg::OnMenyVisaAntalceller()
{
    // TODO: Add your command handler code here
    m_AntalcellerCheck = !m_AntalcellerCheck;
    GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaAntalceller(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_AntalcellerCheck);
}

void CLaddningST7Dlg::OnMenyVisaGrundladdartyp()
{
    // TODO: Add your command handler code here
    m_GrundladdartypCheck = !m_GrundladdartypCheck;
    GetDlgItem(IDC_GrundladdartypAmpereRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_GrundladdartypVoltRuta) -> EnableWindow(m_GrundladdartypCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaGrundladdartyp(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_GrundladdartypCheck);
}

void CLaddningST7Dlg::OnMenyVisaHardvaruversion()
{
    // TODO: Add your command handler code here
    m_HardvaruversionCheck = !m_HardvaruversionCheck;
    GetDlgItem(IDC_HardvaruversionRuta) -> EnableWindow(m_HardvaruversionCheck);
    SkrivStartfil();
}

```

```

void CLaddningST7Dlg::OnUpdateMenyVisaHardvaruversion(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_HardvaruversionCheck);
}

void CLaddningST7Dlg::OnMenyVisaKalibreringsdata()
{
    // TODO: Add your command handler code here
    m_KalibreringsdataCheck = !m_KalibreringsdataCheck;
    GetDlgItem(IDC_KalibreringsdataRuta) -> EnableWindow(m_KalibreringsdataCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaKalibreringsdata(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_KalibreringsdataCheck);
}

void CLaddningST7Dlg::OnMenyVisaLaddartyp()
{
    // TODO: Add your command handler code here
    m_LaddartypCheck = !m_LaddartypCheck;
    GetDlgItem(IDC_LaddartypAmpereRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_LaddartypVoltRuta) -> EnableWindow(m_LaddartypCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaLaddartyp(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_LaddartypCheck);
}

void CLaddningST7Dlg::OnMenyVisaLaddningskurva()
{
    // TODO: Add your command handler code here
    m_LaddningskurvaCheck = !m_LaddningskurvaCheck;
    GetDlgItem(IDC_LaddningskurvaCombo) -> EnableWindow(m_LaddningskurvaCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaLaddningskurva(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_LaddningskurvaCheck);
}

void CLaddningST7Dlg::OnMenyVisaProgram()
{
    // TODO: Add your command handler code here
    m_ProgramCheck = !m_ProgramCheck;
    GetDlgItem(IDC_ProgramCombo) -> EnableWindow(m_ProgramCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaProgram(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here

```

```

        pCmdUI->SetCheck(m_ProgramCheck);
    }
void CLaddningST7Dlg::OnMenyVisaSerienummer()
{
    // TODO: Add your command handler code here
    m_SerienummerCheck = !m_SerienummerCheck;
    GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
    SkrivStartfil();
}

void CLaddningST7Dlg::OnUpdateMenyVisaSerienummer(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    pCmdUI->SetCheck(m_SerienummerCheck);
}

void CLaddningST7Dlg::OnMenyVisaDefaultanpassning()
{
    // TODO: Add your command handler code here
    //Visa följande fönster vid anpassningen (defaultvärde)
    m_DefaultAnpassningCheck = TRUE;
    m_AmperetimmarCheck = TRUE;
    m_AntalcellerCheck = TRUE;
    m_GrundladdartypCheck = FALSE;
    m_HardvaruversionCheck = FALSE;
    m_KalibreringsdataCheck = FALSE;
    m_LaddartypCheck = TRUE;
    m_LaddningskurvaCheck = TRUE;
    m_ProgramCheck = FALSE;
    m_SerienummerCheck = FALSE;
    m_DataCheck = FALSE;
    m_Sparautanloggfilcheck = FALSE;
    GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
    GetDlgItem(IDC_ProgramCombo) -> EnableWindow(m_ProgramCheck);
    GetDlgItem(IDC_LaddningskurvaCombo) -> EnableWindow(m_LaddningskurvaCheck);
    GetDlgItem(IDC_LaddartypAmpereRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_LaddartypVoltRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_KalibreringsdataRuta) -> EnableWindow(m_KalibreringsdataCheck);
    GetDlgItem(IDC_HardvaruversionRuta) -> EnableWindow(m_HardvaruversionCheck);
    GetDlgItem(IDC_GrundladdartypAmpereRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_GrundladdartypVoltRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_AdressRuta) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta0) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta1) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta2) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta3) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta4) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta5) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta6) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta7) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta8) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta9) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta10) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta11) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta12) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta13) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta14) -> EnableWindow(m_DataCheck);
}

```

```

        GetDlgItem(IDC_DataRuta15) -> EnableWindow(m_DataCheck);
        GetDlgItem(IDC_Skriv16byteKnapp) -> EnableWindow(m_DataCheck);
        GetDlgItem(IDC_Las16byte) -> EnableWindow(m_DataCheck);
        GetDlgItem(IDC_SkrivKnapp) -> EnableWindow(m_Sparautanloggfilcheck);
        SkrivStartfil();
    }

void CLaddningST7Dlg::OnUpdateMenyVisaDefaultanpassning(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    //pCmdUI->SetCheck(m_DefaultAnpassningCheck);
}

void CLaddningST7Dlg::OnMenyVisaDefaultProduktion()
{
    // TODO: Add your command handler code here
    //Visa följande fönster vid produktionen (defaultvärde)
    m_DefaultAnpassningCheck = FALSE;
    m_AmperetimmarCheck = TRUE;
    m_AntalcellerCheck = TRUE;
    m_GrundladdartypCheck = TRUE;
    m_HardvaruversionCheck = TRUE;
    m_KalibreringsdataCheck = TRUE;
    m_LaddartypCheck = TRUE;
    m_LaddningskurvaCheck = TRUE;
    m_ProgramCheck = TRUE;
    m_SerienummerCheck = TRUE;
    m_DataCheck = FALSE;
    m_Sparautanloggfilcheck = TRUE;
    GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
    GetDlgItem(IDC_ProgramCombo) -> EnableWindow(m_ProgramCheck);
    GetDlgItem(IDC_LaddningskurvaCombo) -> EnableWindow(m_LaddningskurvaCheck);
    GetDlgItem(IDC_LaddartypAmpereRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_LaddartypVoltRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_KalibreringsdataRuta) -> EnableWindow(m_KalibreringsdataCheck);
    GetDlgItem(IDC_HardvaruversionRuta) -> EnableWindow(m_HardvaruversionCheck);
    GetDlgItem(IDC_GrundladdartypAmpereRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_GrundladdartypVoltRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_AdressRuta) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta0) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta1) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta2) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta3) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta4) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta5) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta6) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta7) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta8) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta9) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta10) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta11) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta12) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta13) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta14) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta15) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_Skriv16byteKnapp) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_Las16byte) -> EnableWindow(m_DataCheck);
}

```

```

        GetDlgItem(IDC_SkrivKnapp) -> EnableWindow(m_Sparautanloggfilcheck);
        SkrivStartfil();
    }

void CLaddningST7Dlg::OnUpdateMenyVisaDefaultProduktion(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    //pCmdUI->SetCheck(m_AmperetimmarCheck);
}

void CLaddningST7Dlg::OnMenyVisaDefaultTekniker()
{
    // TODO: Add your command handler code here
    //Visa följande fönster vid tekniker (defaultvärde)
    m_DefaultAnpassningCheck = FALSE;
    m_DataCheck = TRUE;
    m_AmperetimmarCheck = TRUE;
    m_AntalcellerCheck = TRUE;
    m_GrundladdartypCheck = TRUE;
    m_HardvaruversionCheck = TRUE;
    m_KalibreringsdataCheck = TRUE;
    m_LaddartypCheck = TRUE;
    m_LaddningskurvaCheck = TRUE;
    m_ProgramCheck = TRUE;
    m_SerienummerCheck = TRUE;
    m_Sparautanloggfilcheck = TRUE;
    GetDlgItem(IDC_SerienummerRuta) -> EnableWindow(m_SerienummerCheck);
    GetDlgItem(IDC_ProgramCombo) -> EnableWindow(m_ProgramCheck);
    GetDlgItem(IDC_LaddningskurvaCombo) -> EnableWindow(m_LaddningskurvaCheck);
    GetDlgItem(IDC_LaddartypAmpereRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_LaddartypVoltRuta) -> EnableWindow(m_LaddartypCheck);
    GetDlgItem(IDC_KalibreringsdataRuta) -> EnableWindow(m_KalibreringsdataCheck);
    GetDlgItem(IDC_HardvaruversionRuta) -> EnableWindow(m_HardvaruversionCheck);
    GetDlgItem(IDC_GrundladdartypAmpereRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_GrundladdartypVoltRuta) -> EnableWindow(m_GrundladdartypCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_Antalcellerruta) -> EnableWindow(m_AntalcellerCheck);
    GetDlgItem(IDC_AmperetimmarRuta) -> EnableWindow(m_AmperetimmarCheck);
    GetDlgItem(IDC_AdressRuta) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta0) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta1) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta2) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta3) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta4) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta5) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta6) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta7) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta8) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta9) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta10) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta11) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta12) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta13) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta14) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_DataRuta15) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_Skriv16byteKnapp) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_Las16byte) -> EnableWindow(m_DataCheck);
    GetDlgItem(IDC_SkrivKnapp) -> EnableWindow(m_Sparautanloggfilcheck);
    SkrivStartfil();
}

```

```

void CLaddningST7Dlg::OnUpdateMenyVisaDefaultTekniker(CCmdUI* pCmdUI)
{
    // TODO: Add your command update UI handler code here
    //pCmdUI->SetCheck(m_DataCheck);
}

void CLaddningST7Dlg::OnMenyArkivAvsluta()
{
    // TODO: Add your command handler code here
    IO1.Close();
    OnOK();    //Avsluta programmet
}

void CLaddningST7Dlg::OnMenyHjalpOm()
{
    // TODO: Add your command handler code here
    CAboutDlg dlgAbout;
    dlgAbout.DoModal();    //Visa fönstret about
}

void CLaddningST7Dlg::OnMenyArkivLas()
{
    // TODO: Add your command handler code here
    //(Läser in alla värden) och sparar de i temp variabler
    OnLasKnapp();
}

void CLaddningST7Dlg::OnMenyArkivSkriv()
{
    // TODO: Add your command handler code here
    OnSkrivKnapp();
}

void CLaddningST7Dlg::OnLasKnapp()
{
    //Läs in värden från flashen till följande parametrar
    // TODO: Add your control notification handler code here
    //Kommando
    UpdateData(TRUE);
    bool fel = FALSE;
    m_Felmeddelande = "Läsningen lyckades";
    if(SkickaKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Antalcellerstart,1) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(SkickaKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Laddartypstart,2) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(SkickaKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Grundladdartypstart,3) == 0)
            fel = TRUE;
    }
}

```

```

    }
    else
        fel = TRUE;
    if(SkickKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Serienummerstart,4) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(SkickKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Hardvaruversionstart,5) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(SkickKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Laddningskurvanamnstart,6) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(SkickKommando(3) == 1 && fel == FALSE){
        if(LasvardefranAdress(m_Programnamnstart,7) == 0)
            fel = TRUE;
    }
    else
        fel = TRUE;
    if(fel == TRUE){
        m_Felmeddelande = "FEL NR: 48. Kontakten bröts, skrivning avslutades.";
        UpdateData(FALSE);
    }
}

```

```

void CLaddningST7Dlg::OnLas16byte()
{
    //Läs in följande adress från flashen + 16 byte
    // TODO: Add your control notification handler code here
    UpdateData(TRUE);
    m_Felmeddelande = "Läsning av 16 byte lyckades!";
    //Adress
    CString tempAdress;
    if (m_Adress.GetLength() == 4){
        tempAdress = m_Adress.GetAt(0);
        tempAdress += m_Adress.GetAt(1);
        tempAdress += m_Adress.GetAt(2);
        tempAdress += m_Adress.GetAt(3);
        if(m_Adress.GetAt(3) == '0'){
            int rattecken = 1;
            rattecken = Kontroll0till9ellerAtilF(tempAdress);
            if (m_Adress.GetAt(0) == 'E' || m_Adress.GetAt(0) == 'F'){
                if(rattecken == 1){
                    //Kommando
                    if(SkickKommando(3) == 1){
                        if(SkickAdress(tempAdress) == 1){
                            for(int i = 0; i < 18; i++){
                                byte high, low;
                                int raknare = 0;
                                byte m_readstatusruta;
                                m_readstatusruta = IO1.ReadStatus();

```



```

while(((m_readstatusruta & 0x40) >> 6) == 1 && raknare <= 10000){
    m_readstatusruta = IO1.ReadStatus();    //Väntar på att strobe ska gå låg.
    raknare ++;
}
if(raknare < 10000){
    raknare = 0;
    IO1.WriteControl(0+0+0+0+0+4+0+0);        //Ack på valid data
    m_readstatusruta = IO1.ReadStatus();
    while(((m_readstatusruta & 0x40) >> 6) == 0 && raknare <= 10000){
        m_readstatusruta = IO1.ReadStatus(); //Väntar på att strobe ska gå hög
        raknare ++;
    }
    if(raknare < 10000)
        IO1.WriteControl(0+0+32+0+0+4+0+0);    //Bi-directional port
    else
        m_Felmeddelande = "FEL NR: 49. Kontakten bröts, läsningen avslutades.";
} //if(raknare < 10000)
else
    m_Felmeddelande = "FEL NR: 50. Kontakten bröts, läsningen avslutades.";
m_TaEmot = IO1.ReadData();                    //Läser data och sparar på fil
m_readstatusruta = IO1.ReadStatus();
if(((m_readstatusruta & 0x20) >> 5) == 1) { //Kollar att autofeed är hög.
    high = m_TaEmot;
    low = m_TaEmot;
    high = (high >> 4);
    low = (low & 0x0F);
    if (high>9)
        high = high + 55;
    else
        high = high + 48;
    if (low>9)
        low = low + 55;
    else
        low = low + 48;
    CString temp,tempadress1,tempadress2;
    temp = high;
    temp += low;
    switch (i){
        case 0: tempadress1 = temp;break;
        case 1: tempadress2 = temp;break;
        case 2: m_DataRuta0 = temp;break;
        case 3: m_DataRuta1 = temp;break;
        case 4: m_DataRuta2 = temp;break;
        case 5: m_DataRuta3 = temp;break;
        case 6: m_DataRuta4 = temp;break;
        case 7: m_DataRuta5 = temp;break;
        case 8: m_DataRuta6 = temp;break;
        case 9: m_DataRuta7 = temp;break;
        case 10: m_DataRuta8 = temp;break;
        case 11: m_DataRuta9 = temp;break;
        case 12: m_DataRuta10 = temp;break;
        case 13: m_DataRuta11 = temp;break;
        case 14: m_DataRuta12 = temp;break;
        case 15: m_DataRuta13 = temp;break;
        case 16: m_DataRuta14 = temp;break;
        case 17: m_DataRuta15 = temp;break;
        m_Adress = tempadress1 + tempadress2;
    } //switch (i)
    UpdateData(FALSE);
    GetDlgItem(IDC_TextAdress) -> EnableWindow(TRUE);
}

```

```

        GetDlgItem(IDC_TextData7till0) -> EnableWindow(TRUE);
        GetDlgItem(IDC_TextData15till8) -> EnableWindow(TRUE);
    } // if(((m_readstatusruta & 0x20) >> 5) == 1) { //Kollar att autofeed är hög.

        IO1.WriteControl(0+0+0+0+0+0+0+0); //Klar med läsning på porten
    } //for(int i = 0; i < 18; i++){
    } // if SkickaAdress() == 1)
    else
        m_Felmeddelande = "FEL NR: 51. Kontakten bröts, läsningen avslutades.";
    } // if(kommando (3) == 1)
    else
        m_Felmeddelande = "FEL NR: 52. Kontakten bröts, läsningen avslutades.";
    } //if(rattecken == 1){
    else
        m_Felmeddelande = "FEL NR: 53. Felaktigt tecken i adressruta";
    }
    else
        m_Felmeddelande = "FEL NR: 54. Felaktigt adressområde i adressruta";
    } //if(m_Adress.GetAt(3) == '0'){
    else
        m_Felmeddelande = "FEL NR: 55. Felaktig adress";
    } //if (m_Adress.GetLength()==4){
    else
        m_Felmeddelande = "FEL NR: 56. Felaktigt antal tecken i adressruta";
    UpdateData(FALSE);
}

int CLaddningST7Dlg::KollaLangd(CString data)
{
    int langd = data.GetLength();
    if(langd==2)
        return 1;
    else
        return 0;
}

void CLaddningST7Dlg::OnSkriv16byteKnapp()
{
    // TODO: Add your control notification handler code here
    UpdateData(TRUE);
    m_Felmeddelande = "Skrivning av 16 byte lyckades!";
    UpdateData(FALSE);
    //Adress
    CString tempAdress;
    if(m_Adress.GetLength()==4){
        tempAdress = m_Adress.GetAt(0);
        tempAdress += m_Adress.GetAt(1);
        tempAdress += m_Adress.GetAt(2);
        tempAdress += m_Adress.GetAt(3);
        if(m_Adress.GetAt(3) == '0'){
            int rattecken = 1, allarattecken = 0;
            rattecken = Kontroll0till9ellerAtillF(tempAdress);
            if (m_Adress.GetAt(0) == 'E' || m_Adress.GetAt(0) == 'F'){
                if (rattecken==1){
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta0);if(rattecken == 1)allarattecken++;
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta1);if(rattecken == 1)allarattecken++;
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta2);if(rattecken == 1)allarattecken++;
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta3);if(rattecken == 1)allarattecken++;
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta4);if(rattecken == 1)allarattecken++;
                    rattecken = Kontroll0till9ellerAtillF(m_DataRuta5);if(rattecken == 1)allarattecken++;
                }
            }
        }
    }
}

```

```

rattecken = Kontroll0till9ellerAtillF(m_DataRuta6);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta7);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta8);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta9);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta10);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta11);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta12);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta13);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta14);if(rattecken == 1)allarattecken++;
rattecken = Kontroll0till9ellerAtillF(m_DataRuta15);if(rattecken == 1)allarattecken++;
if(allarattecken == 16){
    int rattlangd = 0, allarattlangd = 0;
    rattlangd = KollaLangd(m_DataRuta0);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta1);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta2);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta3);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta4);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta5);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta6);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta7);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta8);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta9);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta10);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta11);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta12);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta13);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta14);if(rattlangd == 1)allarattlangd++;
    rattlangd = KollaLangd(m_DataRuta15);if(rattlangd == 1)allarattlangd++;

    if(allarattlangd == 16){

        //Kommando
        if (SkickaKommando(1) == 1){
            if (SkickaKommando(2) == 1){
                if (SkickaAdress(tempAdress) == 1){
                    BYTE temp, high, low;
                    //Data 16 byte
                    CString skickadata;
                    for (int i=0; i<16; i++){
                        if (i == 0)
                            skickadata = m_DataRuta0;
                        if (i == 1)
                            skickadata = m_DataRuta1;
                        if (i == 2)
                            skickadata = m_DataRuta2;
                        if (i == 3)
                            skickadata = m_DataRuta3;
                        if (i == 4)
                            skickadata = m_DataRuta4;
                        if (i == 5)
                            skickadata = m_DataRuta5;
                        if (i == 6)
                            skickadata = m_DataRuta6;
                        if (i == 7)
                            skickadata = m_DataRuta7;
                        if (i == 8)
                            skickadata = m_DataRuta8;
                        if (i == 9)
                            skickadata = m_DataRuta9;
                        if (i == 10)

```

```

        skickadata = m_DataRuta10;
    if (i == 11)
        skickadata = m_DataRuta11;
    if (i == 12)
        skickadata = m_DataRuta12;
    if (i == 13)
        skickadata = m_DataRuta13;
    if (i == 14)
        skickadata = m_DataRuta14;
    if (i == 15)
        skickadata = m_DataRuta15;

    if (skickadata.GetAt(0) > 64)
        high = skickadata.GetAt(0) - 55;
    else
        high = skickadata.GetAt(0) - 48;

    if (skickadata.GetAt(1) > 64)
        low = skickadata.GetAt(1) - 55;
    else
        low = skickadata.GetAt(1) - 48;

    high = high << 4;
    temp = high + low;

    SkickaByte(temp);
} //for
} //Skicka adress
else{
    m_Felmeddelande = "FEL NR: 57. Kontakten bröts, skrivning avslutades.";
    UpdateData(FALSE);
}
} //Skicka kommando 2
else{
    m_Felmeddelande = "FEL NR: 58. Kontakten bröts, skrivning avslutades.";
    UpdateData(FALSE);
}
} //Skicka kommando 1
else {
    m_Felmeddelande = "FEL NR: 59. Kontakten bröts, skrivning avslutades.";
    UpdateData(FALSE);
}
GetDlgItem(IDC_TextAdress) -> EnableWindow(TRUE);
GetDlgItem(IDC_TextData7till0) -> EnableWindow(TRUE);
GetDlgItem(IDC_TextData15till8) -> EnableWindow(TRUE);
} //if(allarattlangd == 16){
else
    m_Felmeddelande = "FEL NR: 60.Felaktigt antal tecken i dataruta";
} //if(allarattecken == 16){
else
    m_Felmeddelande = "FEL NR: 61 Felaktigt tecken i dataruta";
} //if/rattecken == 1
else
    m_Felmeddelande = "FEL NR 62. Felaktigt tecken i adressruta";
} // if((m_Adress.GetAt(0) == 'E' || m_Adress.GetAt(0) == 'F'){
else
    m_Felmeddelande = "FEL NR 63 Felaktigt adressområde i adressruta";
} //if(m_Adress.GetAt(3) == '0'){
else
    m_Felmeddelande = "FEL NR 64. Felaktig adress";

```

```

    } //if(m_Address.GetLength()==4){
    else
        m_Felmeddelande = "FEL NR: 65. Felaktigt antal tecken i adressruta";
    UpdateData(FALSE);
}

void CLaddningST7Dlg::SkrivStartfil()
{
    //Spara startvärden för fönsterutseende i startfil
    byte data;
    ofstream startfil("startfil.txt",ios::out );
    for (int i=0; i< 11; i++){
        startfil.seekp(i*sizeof(data));
        switch(i){
            case 0: data = m_LaddningskurvaCheck; break;
            case 1: data = m_AntalcellerCheck; break;
            case 2: data = m_AmperetimmarCheck; break;
            case 3: data = m_ProgramCheck; break;
            case 4: data = m_HardvaruversionCheck; break;
            case 5: data = m_KalibreringsdataCheck; break;
            case 6: data = m_SerienummerCheck; break;
            case 7: data = m_LaddartypCheck; break;
            case 8: data = m_GrundladdartypCheck; break;
            case 9: data = m_DataCheck; break;
            case 10: data = m_Sparautanloggfilcheck; break;
        }
        startfil.write((char *) &data , sizeof(data));
    }
}

void CLaddningST7Dlg::OnSparaloggfilKnapp()
{
    // TODO: Add your control notification handler code here
    UpdateData(TRUE);
    if (m_felfil2 == FALSE && m_adressfilfel2 == FALSE){
        if (m_Serienummer.GetLength() == 8){ //test av 8 tecken
            int test = Kontroll0till9(m_Serienummer); //test av bara siffror som tecken
            if (test == 1){
                if (OnSkrivKnapp() == 1){ //om lyckad skrivning till flash, spara loggfil
                    UpdateData(TRUE);
                    char Inledning[8] = "LOGGFIL";
                    char Amperetimmar[16] = "Amperetimmar = ";
                    char Hardvaruversion[19] = "Hårdvaruversion = ";
                    char Program[11] = "Program = ";
                    char Laddningskurva[18] = "Laddningskurva = ";
                    char Antalceller[16] = "Antal celler = ";
                    char Kalibreringsdata[20] = "Kalibreringsdata = ";
                    char Serienummer[15] = "Serienummer = ";
                    char Laddartyp[13] = "Laddartyp = ";
                    char Grundladdartyp[18] = "Grundladdartyp = ";
                    char Ampere[8] = " Ampere";
                    char Volt[8] = " Volt ";
                    char Tiden[9] = "Datum = ";
                    char streck[2] = "-";
                    char kolon[2] = ":";
                    char mellanslag[2] = " ";
                    int NyRad = 10;

                    CString filnamn;
                    filnamn = m_LoggfilSokvag;
                }
            }
        }
    }
}

```

```

filnamn += m_Serienummer;
filnamn += ".txt";
ofstream loggfil(filnamn,ios::out);

char attskriva;
int langd;
loggfil.write((char *) &NyRad, sizeof(NyRad));
loggfil.write((char *) &Inledning , sizeof(Inledning));
loggfil.write((char *) &NyRad, sizeof(NyRad));
loggfil.write((char *) &NyRad, sizeof(NyRad));

m_tid= m_tid.GetCurrentTime();
int tid;
tid = m_tid.GetYear();
int tusental = 0;
int hundratal = 0;
int tiotal=0;
int ental=0;
char tio,en,tusen,hundra;
while (tid >999)
{
    tid=tid-1000;
    tusental++;
}
while (tid >99)
{
    tid=tid-100;
    hundratal++;
}
while (tid >9)
{
    tid=tid-10;
    tiotal++;
}
while (tid >0)
{
    tid=tid-1;
    ental++;
}
tio = tiotal + 48;
en= ental + 48;
tusen = tusental + 48;
hundra = hundratal + 48;
loggfil.write((char *) &Tiden, sizeof(Tiden));
loggfil.write((char *) &tusen, sizeof(char));
loggfil.write((char *) &hundra, sizeof(char));
loggfil.write((char *) &tio, sizeof(char));
loggfil.write((char *) &en, sizeof(char));
loggfil.write((char *) &streck, sizeof(streck));

tid = m_tid.GetMonth();
tiotal=0;
ental=0;
while (tid >9)
{
    tid=tid-10;
    tiotal++;
}
while (tid >0)
{

```

```

        tid=tid-1;
        ental++;
    }
    tio=tio+48;
    en= ental+48;
    loggfil.write((char *) &tio, sizeof(char));
    loggfil.write((char *) &en, sizeof(char));
    loggfil.write((char *) &streck, sizeof(streck));

    tid = m_tid.GetDay();
    tio=0;
    ental=0;
    while (tid >9)
    {
        tid=tid-10;
        tio++;
    }
    while (tid >0)
    {
        tid=tid-1;
        ental++;
    }
    tio=tio+48;
    en= ental+48;
    loggfil.write((char *) &tio, sizeof(char));
    loggfil.write((char *) &en, sizeof(char));
    loggfil.write((char *) &mellanslag, sizeof(mellanslag));

    tid = m_tid.GetHour();
    tio=0;
    ental=0;
    while (tid >9)
    {
        tid=tid-10;
        tio++;
    }
    while (tid >0)
    {
        tid=tid-1;
        ental++;
    }
    tio=tio+48;
    en= ental+48;
    loggfil.write((char *) &tio, sizeof(char));
    loggfil.write((char *) &en, sizeof(char));
    loggfil.write((char *) &kolon, sizeof(kolon));

    tid = m_tid.GetMinute();
    tio=0;
    ental=0;
    while (tid >9)
    {
        tid=tid-10;
        tio++;
    }
    while (tid >0)
    {
        tid=tid-1;
        ental++;
    }

```

```

tio=tio+48;
en= en+48;
loggfil.write((char *) &tio, sizeof(char));
loggfil.write((char *) &en, sizeof(char));
loggfil.write((char *) &kolon, sizeof(kolon));

tid = m_tid.GetSecond();
tio=0;
en=0;
while (tid >9)
{
    tid=tid-10;
    tio++;
}
while (tid >0)
{
    tid=tid-1;
    en++;
}
tio=tio+48;
en= en+48;
loggfil.write((char *) &tio, sizeof(char));
loggfil.write((char *) &en, sizeof(char));
loggfil.write((char *) &NyRad, sizeof(NyRad));
loggfil.write((char *) &Laddningskurva, sizeof(Laddningskurva));
CString m_ProgramTemp;

int h=m_Laddningskurva.GetCurSel();
if ( h != CB_ERR){
    m_Laddningskurva.GetLBText(h,m_ProgramTemp);
    langd = m_ProgramTemp.GetLength();
    for (int i=0; i<langd; i++){
        attskriva = m_ProgramTemp.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
}
loggfil.write((char *) &NyRad, sizeof(NyRad));

loggfil.write((char *) &Antalceller, sizeof(Antalceller));
langd = m_Antalceller.GetLength();
for (int i=0; i<langd; i++){
    attskriva =m_Antalceller.GetAt(i);
    loggfil.write((char *) &attskriva , sizeof(char));
}
loggfil.write((char *) &NyRad, sizeof(NyRad));

loggfil.write((char *) &Amperetimmr , sizeof(Amperetimmr));
langd = m_Amperetimmr.GetLength();
for ( i=0; i<langd; i++){
    attskriva = m_Amperetimmr.GetAt(i);
    loggfil.write((char *) &attskriva , sizeof(char));
}
loggfil.write((char *) &NyRad, sizeof(NyRad));

loggfil.write((char *) &Program, sizeof(Program));

h=m_ProgramCombo.GetCurSel();
if (h != CB_ERR){
    m_ProgramCombo.GetLBText(h,m_ProgramTemp);
    langd = m_ProgramTemp.GetLength();

```



```

        for (i=0; i<langd; i++){
            attskriva = m_ProgramTemp.GetAt(i);
            loggfil.write((char *) &attskriva , sizeof(char));
        }
    }
    loggfil.write((char *) &NyRad, sizeof(NyRad));

    loggfil.write((char *) &Hardvaruversion , sizeof(Hardvaruversion));
    langd = m_Hardvaruversion.GetLength();
    for (i=0; i<langd; i++){
        attskriva = m_Hardvaruversion.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &NyRad, sizeof(NyRad));

    loggfil.write((char *) &Kalibreringsdata , sizeof(Kalibreringsdata));
    langd = m_Kalibreringsdata.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_Kalibreringsdata.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &NyRad, sizeof(NyRad));

    loggfil.write((char *) &Serienummer, sizeof(Serienummer));
    langd = m_Serienummer.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_Serienummer.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &NyRad, sizeof(NyRad));

    loggfil.write((char *) &Laddartyp, sizeof(Laddartyp));
    langd = m_LaddartypVolt.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_LaddartypVolt.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &Volt, sizeof(Volt));
    langd = m_LaddartypAmpere.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_LaddartypAmpere.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &Ampere, sizeof(Ampere));
    loggfil.write((char *) &NyRad, sizeof(NyRad));

    loggfil.write((char *) &Grundladdartyp, sizeof(Grundladdartyp));
    langd = m_GrundladdartypVolt.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_GrundladdartypVolt.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &Volt, sizeof(Volt));
    langd = m_GrundladdartypAmpere.GetLength();
    for (i=0; i<langd; i++){
        attskriva =m_GrundladdartypAmpere.GetAt(i);
        loggfil.write((char *) &attskriva , sizeof(char));
    }
    loggfil.write((char *) &Ampere, sizeof(Ampere));

```

```

        loggfil.write((char *) &NyRad, sizeof(NyRad));
    } //if (OnSkrivKnapp() == 1){
    } //if(test == 1)
} //if (m_Serienummer.GetLength() == 8){
else
    m_Felmeddelande = "FEL NR: 66. Inget giltigt serienummer";
}
else
    m_Felmeddelande = "FEL NR: 67. Program i felläge. Adresser till variabler kan ej hittas.";
UpdateData(FALSE);
}

//Kontroll om något värde ändrat i en editbox eller combobox
void CLaddningST7Dlg::OnChangeAntalcellerruta()
{
    // TODO: If this is a RICHEDIT control, the control will not
    // send this notification unless you override the CDialog::OnInitDialog()
    // function and call CRichEditCtrl().SetEventMask()
    // with the ENM_CHANGE flag ORed into the mask.

    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextAntalceller) -> EnableWindow(FALSE);
    m_AntalCellerChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeAmperetimmarRuta()
{
    // TODO: If this is a RICHEDIT control, the control will not
    // send this notification unless you override the CDialog::OnInitDialog()
    // function and call CRichEditCtrl().SetEventMask()
    // with the ENM_CHANGE flag ORed into the mask.

    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextAmperetimmar) -> EnableWindow(FALSE);
    m_AmperetimmarChanged = TRUE;
}

void CLaddningST7Dlg::OnCloseupLaddningskurvaCombo()
{
    //Håller reda på valt objekt i combobox
    //Namn och Revision läses in från flachen vid valt objekt
    // TODO: Add your control notification handler code here
    UpdateData(TRUE);
    bool Filfel = FALSE;
    int h = m_Laddningskurva.GetCurSel();
    if (h != CB_ERR){
        GetDlgItem(IDC_TextLaddningskurva) -> EnableWindow(FALSE);
        m_LaddningskurvaChanged = TRUE;
        CString filnamn,tempfilnamn;
        BOOL fSuccess;
        tempfilnamn =m_LaddningskurvaSokvag;
        m_Laddningskurva.GetLBText(h,filnamn);
        tempfilnamn += filnamn;
        CStdioFile file;
        fSuccess = file.Open(
            _T(tempfilnamn),
            CFile::modeRead |
            CFile::shareDenyRead |
            CFile::typeText
        );
    }
}

```

```

    if(!fSuccess){
        Filfel = TRUE;
    }
    if(Filfel == FALSE){
        CString Filtext;
        m_HittatAdress = FALSE;
        while (file.ReadString(Filtext) && m_HittatAdress == FALSE){
            byte ascii;
            CString high;
            Filtext.Delete(0);
            Filtext.Delete(0);
            Filtext.Delete(0);
            Filtext.Delete(0);
            if((Filtext.GetAt(0) == 'F') && (Filtext.GetAt(1) == 'C') && (Filtext.GetAt(2) == '0')
            && (Filtext.GetAt(3) == '0')){
                m_LaddningskurvaNamn = "";
                m_LaddningskurvaRevision = "";
                for(int i=4; i<22; i++){
                    high = "";
                    high = Filtext.GetAt(i);
                    high += Filtext.GetAt(i+1);
                    ascii = OmvandlaHex tillAscii(high);
                    m_LaddningskurvaNamn += ascii;
                    i++;
                }
                for( i=22; i<32; i++){
                    high = "";
                    high = Filtext.GetAt(i);
                    high += Filtext.GetAt(i+1);
                    ascii = OmvandlaHex tillAscii(high);
                    m_LaddningskurvaRevision += ascii;
                    i++;
                }
                UpdateData(FALSE);
                m_HittatAdress = TRUE;
            } //if((Filtext.GetAt(0) == 'F') && (Filtext.GetAt(
            m_Felmeddelande = "";
        } //while (file.ReadString(Filtext) && m_HittatAdress == FALSE){
    } //if(Filfel == FALSE){
    else
        m_Felmeddelande = "FEL NR: 68. Laddningskurvafilläsning misslyckad";
} //if (h != CB_ERR){
else
    m_Felmeddelande = "FEL NR: 69. Välj Laddningskurva i listan";
UpdateData(FALSE);
}

void CLaddningST7Dlg::OnCloseupProgramCombo()
{
    //Håller reda på valt objekt i combobox
    //Namn och Revision läses in från flachen vid valt objekt
    // TODO: Add your control notification handler code here
    bool Filfel = FALSE;
    UpdateData(TRUE);
    int h = m_ProgramCombo.GetCurSel();
    if (h != CB_ERR){
        GetDlgItem(IDC_TextProgram) -> EnableWindow(FALSE);
        m_ProgramChanged = TRUE;
        CString filnamn,tempfilnamn;
        BOOL fSuccess;

```

```

tempfilnamn =m_ProgramSokvag;
m_ProgramCombo.GetLBText(h,filnamn);
tempfilnamn += filnamn;
CStdioFile file;
    fSuccess = file.Open(
        _T(tempfilnamn),
        CFile::modeRead |
        CFile::shareDenyRead |
        CFile::typeText
    );
if(!fSuccess){
    Filfel = TRUE;
}
CString Filtext;
m_HittatAddress = FALSE;
if(Filfel == FALSE){
    while (file.ReadString(Filtext) && m_HittatAddress == FALSE)
    {
        byte ascii;
        CString high;
        Filtext.Delete(0);
        Filtext.Delete(0);
        Filtext.Delete(0);
        Filtext.Delete(0);
        if((Filtext.GetAt(0) == 'E') && (Filtext.GetAt(1) == '0')
        && (Filtext.GetAt(2) == '0') && (Filtext.GetAt(3) == '0')){
            m_ProgramNamn = "";
            m_ProgramRevision = "";
            for(int i=4; i<22; i++){
                high = "";
                high = Filtext.GetAt(i);
                high += Filtext.GetAt(i+1);
                ascii = OmvandlaHex tillAscii(high);
                m_ProgramNamn += ascii;
                i++;
            }
            for( i=22; i<32; i++){
                high = "";
                high = Filtext.GetAt(i);
                high += Filtext.GetAt(i+1);
                ascii = OmvandlaHex tillAscii(high);
                m_ProgramRevision += ascii;
                i++;
            }
            UpdateData(FALSE);
            m_HittatAddress = TRUE;
        }
    }
    m_Felmeddelande = "";
}
else{
    m_Felmeddelande = "FEL NR: 70. Programfilläsning misslyckad";
}
}
else
    m_Felmeddelande = "FEL NR: 71. Välj Program i listan";
UpdateData(FALSE);
}

void CLaddningST7Dlgs::OnChangeHardvaruversionRuta()

```

```

{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextHardvaruversion) -> EnableWindow(FALSE);
    m_HardvaruversionChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeKalibreringsdataRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextKalibreringsdata) -> EnableWindow(FALSE);
    m_KalibreringsdataChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeSerienummerRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextSerienummer) -> EnableWindow(FALSE);
    m_SerienummerChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeLaddartypVoltRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextLaddartyp) -> EnableWindow(FALSE);
    m_LaddartypVoltChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeLaddartypAmpereRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextLaddartyp) -> EnableWindow(FALSE);
    m_LaddartypAmpereChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeGrundladdartypAmpereRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextGrundladdartyp) -> EnableWindow(FALSE);
    m_GrundladdartypAmpereChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeGrundladdartypVoltRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextGrundladdartyp) -> EnableWindow(FALSE);
    m_GrundladdartypVoltChanged = TRUE;
}

void CLaddningST7Dlg::OnChangeAdressRuta()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextAdress) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta0()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

```

```

void CLaddningST7Dlg::OnChangeDataRuta1()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta10()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta11()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta12()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta13()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta14()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta15()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta2()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta3()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta4()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

```

```

void CLaddningST7Dlg::OnChangeDataRuta5()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta6()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta7()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData7till0) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta8()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

void CLaddningST7Dlg::OnChangeDataRuta9()
{
    // TODO: Add your control notification handler code here
    GetDlgItem(IDC_TextData15till8) -> EnableWindow(FALSE);
}

byte CLaddningST7Dlg::OmvandlaHex tillAscii(CString hex)
{
    // TODO: Add your control notification handler code here
    byte high, total;
    if(hex.GetAt(0) >= 65)
        high = hex.GetAt(0) - 55;
    else
        high = hex.GetAt(0) - 48;
    total = high * 16;
    if(hex.GetAt(1) >= 65)
        high = hex.GetAt(1) - 55;
    else
        high = hex.GetAt(1) - 48;
    total += high;
    return total;
}

CString CLaddningST7Dlg::Buffer(CString data)
{
    CString buffertemp;
    int langd;
    langd = data.GetLength();
    if (langd == 1){
        buffertemp = "0";
        buffertemp += "0";
        buffertemp += data.GetAt(0);
    }
    if (langd == 2){
        buffertemp = "0";
        buffertemp += data.GetAt(0);
    }
}

```

```

        buffertemp += data.GetAt(1);
    }
    if (langd == 3){
        buffertemp += data.GetAt(0);
        buffertemp += data.GetAt(1);
        buffertemp += data.GetAt(2);
    }
    return buffertemp;
}

```

CString CLaddningST7Dlg::TotBuffer(CString data1, CString data2, int antal)

```

{
    //Fyller på med ett antal "FF", för att få 16 byte i varje sändning av data
    CString buffertemp;
    if (antal==1){
        buffertemp = "00";
        buffertemp += data1;
        for(int i=0; i<14;i++)
            buffertemp += "FF";
    }
    if (antal==2){
        buffertemp = "00";
        buffertemp += data1;
        buffertemp += "00";
        buffertemp += data2;
        for(int i=0; i<12;i++)
            buffertemp += "FF";
    }
    else{
        for(int i=0; i<16;i++)
            buffertemp += "FF";
    }
    return buffertemp;
}

```

bool CLaddningST7Dlg::SkickaKommando(int kommando)

```

{
    int raknare = 0;
    IO1.WriteData(kommando);           //Kommando för reset
    IO1.WriteControl(0+0+0+0+0+0+0+1); //Sätter autolinefeed låg (kommando). Kontrollregistret

    IO1.WriteControl(0+0+0+0+0+0+0+0); //Sätter strobe låg
    m_Status = IO1.ReadStatus();        //Statusregistret
    while(((m_Status & 0x80) >> 7) == 0 && raknare <= 10000){ //busy ska gå hög inom
        m_Status = IO1.ReadStatus();    // en viss tid
        raknare ++;
    }
    if(raknare < 10000){
        raknare = 0;
        IO1.WriteControl(0+0+0+0+0+0+0+1); //Sätter strobe hög.
        while(((m_Status & 0x80) >> 7) == 1 && raknare <= 10000) { //busy ska gå låg inom
            m_Status = IO1.ReadStatus(); //Statusregistret //en viss tid
            raknare ++;
        }
        if(raknare < 10000){
            IO1.WriteControl(0+0+0+0+0+0+0+1); //Sätter strobe hög. Kontrollregistret
        }
        else {
            return 0;
        }
    }
}

```



```

    }
}
else
    return 0;
return 1;
}

bool CLaddningST7Dlg::LasvardefranAdress(CString adress, int val)
{
    if(SkickaAdress(adress) == 1){
        int raknare = 0;
        for(int i = 0; i < 18; i++){ //2 BYTE ADDRESS + 16 BYTE DATA
            byte m_readstatusruta;
            m_readstatusruta = IO1.ReadStatus();
            while(((m_readstatusruta & 0x40) >> 6) == 1 && raknare < 10000) { //Väntar på att strobe ska gå låg.
                m_readstatusruta = IO1.ReadStatus(); //inom en viss tid
                raknare ++;
            }
            if(raknare < 10000){
                raknare = 0;
                IO1.WriteControl(0+0+0+0+0+4+0+0); //Ack på valid data

                m_readstatusruta = IO1.ReadStatus();
                while(((m_readstatusruta & 0x40) >> 6) == 0 && raknare < 10000){ //Väntar på att strobe ska gå hög
                    m_readstatusruta = IO1.ReadStatus(); //inom en viss tid
                    raknare ++;
                }
                if(raknare < 10000){
                    IO1.WriteControl(0+0+32+0+0+4+0+0); //Bi-directional port
                    m_TaEmot = IO1.ReadData(); //Läser data och sparar på fil
                    m_readstatusruta = IO1.ReadStatus();
                }
                else
                    return 0;
            }
        }
        else
            return 0;
        if(((m_readstatusruta & 0x20) >> 5) == 1) { //Kollar att autofeed är hög.Data
            CString hundra,tio,en,tal,tempadress;
            if (val == 1 || val == 2 || val == 3 || val == 5){ //refererar till parametrarna

                int j = 0, temp = 0;;
                for (j = 0; m_TaEmot > 99; j++)
                    m_TaEmot = m_TaEmot - 100;
                temp = j + 48;
                hundra = temp;
                for (j = 0; m_TaEmot > 9; j++)
                    m_TaEmot = m_TaEmot - 10;
                temp = j + 48;
                tio = temp;
                temp = m_TaEmot + 48;
                en = temp;
                if( hundra == '0'){
                    if( tio == '0')
                        tal = en;
                    else
                        tal = tio + en;
                }
            }
            else

```

```

        tal = hundra + tio + en;

    } //if (val == 1 || val == 2 || val == 3 || val == 5)
    byte high,low,temp;
    if(val == 4 || val == 6 || val == 7){ //refererar till parametrarna
        high = m_TaEmot;
        low = m_TaEmot;
        high = high >> 4;
        low = low & 0x0F;
        temp = high*16 + low;
        tal = temp;
    }
    high = m_TaEmot;
    low = m_TaEmot;
    high = high >> 4;
    low = low & 0x0F;
    if (high > 9)
        high = high + 55;
    else
        high = high + 48;
    if (low > 9)
        low = low + 55;
    else
        low = low + 48;

    switch (i){
        case 0: m_Adress = high;
                m_Adress += low;
                break;
        case 1: m_Adress += high;
                m_Adress += low;
                break;
    }
    UpdateData(FALSE);
    if (m_Adress != "FFFF"){
        switch (i){
            case 2: if (val == 4)
                    m_Serienummer = tal;
                    if (val == 6)
                        m_LaddningskurvaNamn = tal;
                    if (val == 7)
                        m_ProgramNamn = tal;
                    break;
            case 3: if (val == 1)
                    m_Antalceller = tal;
                    if (val == 2)
                        m_LaddartypVolt = tal;
                    if (val == 3)
                        m_GrundladdartypVolt = tal;
                    if (val == 4)
                        m_Serienummer += tal;
                    if (val == 5)
                        m_Hardvaruversion = tal;
                    if (val == 6)
                        m_LaddningskurvaNamn += tal;
                    if (val == 7)
                        m_ProgramNamn += tal;
                    break;
            case 4: if (val == 4)
                    m_Serienummer += tal;

```

```

        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 5: if (val == 1)
        m_Amperetimmar = tal;
        if (val == 2)
            m_LaddartypAmpere = tal;
        if (val == 3)
            m_GrundladdartypAmpere = tal;
        if (val == 4)
            m_Serienummer += tal;
        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 6: if (val == 4)
        m_Serienummer += tal;
        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 7: if (val == 4)
        m_Serienummer += tal;
        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 8: if (val == 4)
        m_Serienummer += tal;
        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 9: if (val == 4)
        m_Serienummer += tal;
        if (val == 6)
            m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 10: if (val == 6)
        m_LaddningskurvaNamn += tal;
        if (val == 7)
            m_ProgramNamn += tal;
        break;
case 11: if (val == 6)
        m_LaddningskurvaRevision = tal;
        if (val == 7)
            m_ProgramRevision = tal;
        break;
case 12: if (val == 6)
        m_LaddningskurvaRevision += tal;
        if (val == 7)
            m_ProgramRevision += tal;

```

```

        break;
    case 13: if (val == 6)
        m_LaddningskurvaRevision += tal;
        if (val == 7)
            m_ProgramRevision += tal;
        break;
    case 14: if (val == 6)
        m_LaddningskurvaRevision += tal;
        if (val == 7)
            m_ProgramRevision += tal;
        break;
    case 15: if (val == 6)
        m_LaddningskurvaRevision += tal;
        if (val == 7)
            m_ProgramRevision += tal;
        break;
    case 16: if (val == 6)
        m_LaddningskurvaRevision += tal;
        if (val == 7)
            m_ProgramRevision += tal;
        break;

    } //switch (i)
    } //if (m_Adress != "FFFF"){
    else
        m_Felmeddelande = "FEL NR: 72. Hittade ej alla adresser.";
    } //if(((m_readstatusruta & 0x20) >> 5) == 1) { //Kollar att autofeed är hög.Data

    IO1.WriteControl(0+0+0+0+0+0+0+0); //Klar med läsning på porten
    }
} //if skicka adress
else{
    m_Felmeddelande = "FEL NR 73. Kontakten bröts, läsning avslutades.";
    UpdateData(FALSE);
    return 0;
}
return 1;
UpdateData(FALSE);
}

void CLaddningST7Dlg::OnMenyArkivSkrivochSpara()
{
    // TODO: Add your command handler code here
    OnSparaloggfilKnapp();
}

void CLaddningST7Dlg::OnMouseMove(UINT nFlags, CPoint point)
{
    // TODO: Add your message handler code here and/or call default
    int iResult;
    if(m_felfil==TRUE){
        iResult = MessageBox("FEL NR: 74. Fel vid läsning av katalogfil, hittar ej sökvagar. Programmet kommer inte fungera felfritt. Vill du fortsätta?", "", MB_YESNO | MB_ICONQUESTION);
        if(iResult == IDNO)
            OnOK();
        else
            m_felfil = FALSE;
    }
    if(m_adressfilfel1==TRUE){
        iResult = MessageBox("FEL NR: 75. Fel vid läsning av adressfil, hittar ej sökvagar. Programmet kommer

```

```

        inte fungera felfritt. Vill du fortsätta?", "", MB_YESNO | MB_ICONQUESTION);
        if(iResult == IDNO)
            OnOK();
        else
            m_adressfilfel1 = FALSE;
    }
    CDialog::OnMouseMove(nFlags, point);
}

int CLaddningST7Dlg::KontrollMaxvarde(CString variabel, int max)
{
    int tal=0, langd=0;
    langd = variabel.GetLength();
    if(langd == 3)
        tal = (variabel.GetAt(0) - 48) * 100 + (variabel.GetAt(1) - 48) * 10 + (variabel.GetAt(2) - 48);
    if(langd == 2)
        tal = (variabel.GetAt(0) - 48) * 10 + (variabel.GetAt(1) - 48);
    if(langd == 1)
        tal = (variabel.GetAt(0) - 48);
    if (tal >= max)
        return 0;
    else
        return 1;
}

void CLaddningST7Dlg::UppdateraFonster()
{
    m_ProgramChanged = FALSE;
    GetDlgItem(IDC_TextProgram) -> EnableWindow(TRUE);
    m_HardvaruversionChanged = FALSE;
    GetDlgItem(IDC_TextHardvaruversion) -> EnableWindow(TRUE);
    m_GrundladdartypVoltChanged = FALSE;
    m_GrundladdartypAmpereChanged = FALSE;
    GetDlgItem(IDC_TextGrundladdartyp) -> EnableWindow(TRUE);
    m_LaddartypVoltChanged = FALSE;
    m_LaddartypAmpereChanged = FALSE;
    GetDlgItem(IDC_TextLaddartyp) -> EnableWindow(TRUE);
    m_AntalCellerChanged = FALSE;
    GetDlgItem(IDC_TextAntalceller) -> EnableWindow(TRUE);
    m_AmperetimmarChanged = FALSE;
    GetDlgItem(IDC_TextAmperetimmar) -> EnableWindow(TRUE);
    m_LaddningskurvaChanged = FALSE;
    GetDlgItem(IDC_TextLaddningskurva) -> EnableWindow(TRUE);
    m_KalibreringsdataChanged = FALSE;
    GetDlgItem(IDC_TextKalibreringsdata) -> EnableWindow(TRUE);
    m_SerienummerChanged = FALSE;
    GetDlgItem(IDC_TextSerienummer) -> EnableWindow(TRUE);
    UpdateData(FALSE);
}

```

```

// LaddningST7Dlg.h : header file

#ifndef __AFX_LADDNINGST7DLG_H__A449CF26_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_
#define __AFX_LADDNINGST7DLG_H__A449CF26_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "ParallelPort.h"

// CLaddningST7Dlg dialog

class CLaddningST7Dlg : public CDialog
{
// Construction
public:
    bool m_Sparautanloggfilcheck;
    void UppdateraFonster();
    bool m_adressfilfel2;
    bool m_adressfilfel1;
    int KontrollMaxvarde(CString variabel, int max);
    bool m_felfil2;
    bool m_felfil;
    int Kontroll0till9ellerAtillF(CString data);
    int KollaLangd(CString data);
    bool LasvardefranAdress(CString adress,int val);
    bool SkickaKommando(int kommando);
    CString TotBuffer(CString data1, CString data2, int antal);
    CString Buffer(CString data);
    CString FranIntillHex(CString data);
    CString m_Interupttabellstorlek;
    CString m_Interupttabellstart;
    CString m_Serienummerstart;
    CString m_Hardvaruversionstart;
    CString m_Antalcellerstart;
    CString m_Laddartypstart;
    CString m_Grudladdartypstart;
    CString m_Laddningskurvastorlek;
    CString m_Laddningskurvanamnstart;
    CString m_Programstorlek;
    CString m_Optiosbyte;
    CString m_Programnamnstart;
    CString m_ProgramSokvag;
    CString m_LoggfilSokvag;
    CString m_AdressSokvag;
    CString m_LaddningskurvaSokvag;
    BOOL m_HittatAdress;
    byte OmvandlaHex tillAscii(CString hex);
    byte m_TaEmot;
    int Kontroll0till9(CString variabel);
    BOOL m_ProgramChanged;
    BOOL m_SerienummerChanged;
    BOOL m_KalibreringsdataChanged;
    BOOL m_LaddartypVoltChanged;
    BOOL m_LaddartypAmpereChanged;
    BOOL m_GrudladdartypAmpereChanged;
    BOOL m_GrudladdartypVoltChanged;
    BOOL m_HardvaruversionChanged;
    BOOL m_LaddningskurvaChanged;

```

```

    BOOL m_AmperetimmerChanged;
    bool SkickaByte(BYTE temp);
    bool SkickaData(CString Data);
    BOOL m_AntalCellerChanged;
    bool SkickaAdress(CString Address);
    void SkrivStartfil();
    byte m_Status;
    BOOL m_DataCheck;
    BOOL m_DefaultAnpassningCheck;
    BOOL m_SerienummerCheck;
    BOOL m_ProgramCheck;
    BOOL m_LaddningskurvaCheck;
    BOOL m_LaddartypCheck;
    BOOL m_KalibreringsdataCheck;
    BOOL m_HardvaruversionCheck;
    BOOL m_GrundladdartypCheck;
    BOOL m_AntalcellerCheck;
    BOOL m_AmperetimmerCheck;
    LRESULT OnKickIdle(WPARAM, LPARAM);
    CLaddningST7Dlg(CWnd* pParent = NULL); // standard constructor

// Dialog Data
//{{AFX_DATA(CLaddningST7Dlg)
enum { IDD = IDD_LADDNINGST7_DIALOG };
    CComboBox m_ProgramCombo;
    CComboBox m_Laddningskurva;
    CString m_Amperetimmer;
    CString m_Serienummer;
    CString m_DataRuta0;
    CString m_DataRuta1;
    CString m_DataRuta10;
    CString m_DataRuta11;
    CString m_DataRuta12;
    CString m_DataRuta13;
    CString m_DataRuta14;
    CString m_DataRuta15;
    CString m_DataRuta2;
    CString m_DataRuta3;
    CString m_DataRuta4;
    CString m_DataRuta5;
    CString m_DataRuta6;
    CString m_DataRuta7;
    CString m_DataRuta8;
    CString m_DataRuta9;
    CString m_Adress;
    CString m_Antalceller;
    CString m_GrundladdartypAmpere;
    CString m_GrundladdartypVolt;
    CString m_Hardvaruversion;
    CString m_LaddartypAmpere;
    CString m_LaddartypVolt;
    CTime m_tid;
    CString m_Kalibreringsdata;
    CString m_Felmeddelande;
    CString m_LaddningskurvaNamn;
    CString m_LaddningskurvaRevision;
    CString m_ProgramNamn;
    CString m_ProgramRevision;
//}}AFX_DATA

```

```

// ClassWizard generated virtual function overrides
//{{AFX_VIRTUAL(CLaddningST7Dlg)
protected:
virtual void DoDataExchange(CDataExchange* pDX); // DDX/DDV support
//}}AFX_VIRTUAL

// Implementation
protected:
    HICON m_hIcon;

    // Generated message map functions
   //{{AFX_MSG(CLaddningST7Dlg)
virtual BOOL OnInitDialog();
afx_msg void OnSysCommand(UINT nID, LPARAM lParam);
afx_msg void OnPaint();
afx_msg HCURSOR OnQueryDragIcon();
afx_msg bool OnSkrivKnapp();
afx_msg void OnLasKnapp();
afx_msg void OnMenyVisaAmperetimmar();
afx_msg void OnUpdateMenyVisaAmperetimmar(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaAntalceller();
afx_msg void OnUpdateMenyVisaAntalceller(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaGrundladdartyp();
afx_msg void OnUpdateMenyVisaGrundladdartyp(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaHardvaruversion();
afx_msg void OnUpdateMenyVisaHardvaruversion(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaKalibreringsdata();
afx_msg void OnUpdateMenyVisaKalibreringsdata(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaLaddartyp();
afx_msg void OnUpdateMenyVisaLaddartyp(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaLaddningskurva();
afx_msg void OnUpdateMenyVisaLaddningskurva(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaProgram();
afx_msg void OnUpdateMenyVisaProgram(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaSerienummer();
afx_msg void OnUpdateMenyVisaSerienummer(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaDefaultanpassning();
afx_msg void OnUpdateMenyVisaDefaultanpassning(CCmdUI* pCmdUI);
afx_msg void OnMenyVisaDefaultTekniker();
afx_msg void OnUpdateMenyVisaDefaultTekniker(CCmdUI* pCmdUI);
afx_msg void OnMenyArkivAvsluta();
afx_msg void OnMenyHjalpOm();
afx_msg void OnMenyArkivSkriv();
afx_msg void OnMenyArkivLas();
afx_msg void OnMenyVisaDefaultProduktion();
afx_msg void OnUpdateMenyVisaDefaultProduktion(CCmdUI* pCmdUI);
afx_msg void OnSkriv16byteKnapp();
afx_msg void OnSparaloggfilKnapp();
afx_msg void OnChangeAntalcellerruta();
afx_msg void OnChangeAmperetimmarRuta();
afx_msg void OnCloseupLaddningskurvaCombo();
afx_msg void OnCloseupProgramCombo();
afx_msg void OnChangeHardvaruversionRuta();
afx_msg void OnChangeKalibreringsdataRuta();
afx_msg void OnChangeSerienummerRuta();
afx_msg void OnChangeLaddartypVoltRuta();
afx_msg void OnChangeLaddartypAmpereRuta();
afx_msg void OnChangeGrundladdartypAmpereRuta();
afx_msg void OnChangeGrundladdartypVoltRuta();
afx_msg void OnChangeAdressRuta();

```



```

afx_msg void OnChangeDataRuta0();
afx_msg void OnChangeDataRuta1();
afx_msg void OnChangeDataRuta10();
afx_msg void OnChangeDataRuta11();
afx_msg void OnChangeDataRuta12();
afx_msg void OnChangeDataRuta13();
afx_msg void OnChangeDataRuta14();
afx_msg void OnChangeDataRuta15();
afx_msg void OnChangeDataRuta2();
afx_msg void OnChangeDataRuta3();
afx_msg void OnChangeDataRuta4();
afx_msg void OnChangeDataRuta5();
afx_msg void OnChangeDataRuta6();
afx_msg void OnChangeDataRuta7();
afx_msg void OnChangeDataRuta8();
afx_msg void OnChangeDataRuta9();
afx_msg void OnArkivSattKatalog();
afx_msg void OnLas16byte();
afx_msg void OnMenyArkivSkrivochSpara();
afx_msg void OnMouseMove(UINT nFlags, CPoint point);
//}}AFX_MSG
//afx_msg LRESULT OnKickIdle(WPARAM, LPARAM);
DECLARE_MESSAGE_MAP()
private:
    CParallelPort IO1;
};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_LADDNINGST7DLG_H__A449CF26_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_)

```

```

// LaddningST7.cpp : Defines the class behaviors for the application.

#include "stdafx.h"
#include "LaddningST7.h"
#include "LaddningST7Dlg.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

// CLaddningST7App

BEGIN_MESSAGE_MAP(CLaddningST7App, CWinApp)
   //{{AFX_MSG_MAP(CLaddningST7App)
    // NOTE - the ClassWizard will add and remove mapping macros here.
    // DO NOT EDIT what you see in these blocks of generated code!
   //}}AFX_MSG
    ON_COMMAND(ID_HELP, CWinApp::OnHelp)
END_MESSAGE_MAP()

// CLaddningST7App construction

CLaddningST7App::CLaddningST7App()
{
    // TODO: add construction code here,
    // Place all significant initialization in InitInstance
}

// The one and only CLaddningST7App object

CLaddningST7App theApp;

// CLaddningST7App initialization

BOOL CLaddningST7App::InitInstance()
{
    AfxEnableControlContainer();

    // Standard initialization
    // If you are not using these features and wish to reduce the size
    // of your final executable, you should remove from the following
    // the specific initialization routines you do not need.

#ifdef _AFXDLL
    Enable3dControls();      // Call this when using MFC in a shared DLL
#else
    Enable3dControlsStatic(); // Call this when linking to MFC statically
#endif

    CLaddningST7Dlg dlg;
    m_pMainWnd = &dlg;
    int nResponse = dlg.DoModal();
    if (nResponse == IDOK)
    {
        // TODO: Place code here to handle when the dialog is
        // dismissed with OK
    }
    else if (nResponse == IDCANCEL)
    {

```

```
        // TODO: Place code here to handle when the dialog is
        // dismissed with Cancel
    }

    // Since the dialog has been closed, return FALSE so that we exit the
    // application, rather than start the application's message pump.
    return FALSE;
}
```

```

// LaddningST7.h : main header file for the LADDNINGST7 application

#ifndef !defined(AFX_LADDNINGST7_H__A449CF24_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_)
#define AFX_LADDNINGST7_H__A449CF24_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
    #error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"      // main symbols

// CLaddningST7App:
// See LaddningST7.cpp for the implementation of this class
//

class CLaddningST7App : public CWinApp
{
public:
    CLaddningST7App();

// Overrides
    // ClassWizard generated virtual function overrides
    //{{AFX_VIRTUAL(CLaddningST7App)
public:
    virtual BOOL InitInstance();
    //}}AFX_VIRTUAL

// Implementation

    //{{AFX_MSG(CLaddningST7App)
    // NOTE - the ClassWizard will add and remove member functions here.
    //  DO NOT EDIT what you see in these blocks of generated code !
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_LADDNINGST7_H__A449CF24_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_)

```

```

/*
Module : PARALLEL.CPP
Purpose: Implementation for an MFC class to encapsulate parallel ports
Created: PJN / 28-12-1999
History: PJN / 06-03-2000 Code now throws an exception if no Win32 parallel ports are available
        PJN / 04-04-2000 Code handles the case where it is run on NT / Windows 2000 by throwing
                        an exception
        PJN / 19-07-2000 1. Now works on NT / Windows 2000 thanks to DriverLINX Port I/O Driver
                        2. Made all typedefs and enums local to the appropriate class rather
                        than polluting the global namespace

Copyright (c) 1999 - 2000 by PJ Naughter.
All rights reserved.
*/

//////////////////// Includes //////////////////////
#include "stdafx.h"
#include "ParallelPort.h"
#include <winerror.h>
#include <conio.h>
#include <afxpriv.h>

//////////////////// Defines / Statics //////////////////////
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

int CParallelPort::sm_nRefCount = 0;
BOOL CParallelPort::sm_bRunningOnNT = FALSE;
HINSTANCE CParallelPort::sm_hDLINX = NULL;
CParallelPort::LPDLPORTREADPORTUCHAR CParallelPort::sm_lpfnDlPortReadUchar = NULL;
CParallelPort::LPDLPORTWRITEPORTUCHAR CParallelPort::sm_lpfnDlPortWriteUchar = NULL;
CArray<CParallelPortSettings, CParallelPortSettings&> CParallelPort::sm_Ports;

//////////////////// Implementation //////////////////////
////////// Exception handling code

void AfxThrowParallelException(DWORD dwError /*= 0 */)
{
    if (dwError == 0)
        dwError = ::GetLastError();

    CParallelException* pException = new CParallelException(dwError);

    TRACE(_T("Warning: throwing CParallelException for error %d\n"), dwError);
    THROW(pException);
}

BOOL CParallelException::GetErrorMessage(LPTSTR pstrError, UINT nMaxError, PUINT pnHelpContext)
{
    ASSERT(pstrError != NULL && AfxIsValidString(pstrError, nMaxError));

    if (pnHelpContext != NULL)
        *pnHelpContext = 0;

    LPTSTR lpBuffer;

```

```

        BOOL bRet = FormatMessage(FORMAT_MESSAGE_ALLOCATE_BUFFER |
FORMAT_MESSAGE_FROM_SYSTEM,
                                NULL, m_dwError, MAKELANGID(LANG_NEUTRAL,
SUBLANG_SYS_DEFAULT),
                                (LPTSTR) &lpBuffer, 0, NULL);

        if (bRet == FALSE)
            *pstrError = '\0';
        else
        {
            lstrcpyn(pstrError, lpBuffer, nMaxError);
            bRet = TRUE;

            LocalFree(lpBuffer);
        }

        return bRet;
    }

CString CParallelException::GetErrorMessage()
{
    CString rVal;
    LPTSTR pstrError = rVal.GetBuffer(4096);
    GetErrorMessage(pstrError, 4096, NULL);
    rVal.ReleaseBuffer();
    return rVal;
}

CParallelException::CParallelException(DWORD dwError)
{
    m_dwError = dwError;
}

CParallelException::~CParallelException()
{
}

IMPLEMENT_DYNAMIC(CParallelException, CException)

#ifdef _DEBUG
void CParallelException::Dump(CDumpContext& dc) const
{
    CObject::Dump(dc);

    dc << "m_dwError = " << m_dwError;
}
#endif

////////// Settings class
CParallelPortSettings::CParallelPortSettings()
{
    m_nBaseAddress = 0;
    m_Type = ParallelTypeUndefined;
    m_ECPMode = ECPModeUndefined;
}

CParallelPortSettings::CParallelPortSettings(const CParallelPortSettings& state)
{
    *this = state;
}

```

```

CParallelPortSettings& CParallelPortSettings::operator=(const CParallelPortSettings& state)
{
    m_nBaseAddress = state.m_nBaseAddress;
    m_Type          = state.m_Type;
    m_ECPMode       = state.m_ECPMode;

    return *this;
}

////////// The actual parallel port class

CParallelPort::CParallelPort()
{
    m_hPort = INVALID_HANDLE_VALUE;
    m_nPortIndex = -1;
    m_nBaseAddress = 0;
    m_dwTimeout = 1000; //Default timeout is 1 second

    //Test for presence of parallel ports at the standard addresses of 0x3BC, 0x378 and 0x278
    if (sm_nRefCount == 0)
    {
        ++sm_nRefCount;

        //Initialize the DriverLINX driver if on NT / Windows 2000
        sm_bRunningOnNT = RunningOnNT();
        if (sm_bRunningOnNT)
        {
            if (!InitializeDriverLINX())
            {
                TRACE(_T("Running on Windows NT / Windows 2000 and the DriverLINX PORTIO driver is not
installed, aborting !!\n"));
                AfxThrowParallelException();
            }
        }

        //Open LPT1 - LPT3 as a precaution against other processes trying to write
        //to the port while we are detecting them
        HANDLE hPort1 = CreateFile(_T("\\\\.\\LPT1"), GENERIC_READ | GENERIC_WRITE, 0, NULL,
OPEN_EXISTING, 0, NULL);
        HANDLE hPort2 = CreateFile(_T("\\\\.\\LPT2"), GENERIC_READ | GENERIC_WRITE, 0, NULL,
OPEN_EXISTING, 0, NULL);
        HANDLE hPort3 = CreateFile(_T("\\\\.\\LPT3"), GENERIC_READ | GENERIC_WRITE, 0, NULL,
OPEN_EXISTING, 0, NULL);

        //Must have at least one port available in Win32 to continue to attempt
        //detection of the ports
        int nWin32Ports = 0;
        if (hPort1 != INVALID_HANDLE_VALUE)
            ++nWin32Ports;
        if (hPort2 != INVALID_HANDLE_VALUE)
            ++nWin32Ports;
        if (hPort3 != INVALID_HANDLE_VALUE)
            ++nWin32Ports;
        if (nWin32Ports == 0)
        {
            TRACE(_T("No parallel ports are available to Win32, aborting !!\n"));
            AfxThrowParallelException(ERROR_DEV_NOT_EXIST);
        }
    }
}

```

```

//Try to detect the details of the 3 standard ports
CParallelPortSettings settings;
if (GetPort(0x3BC, settings))
    sm_Ports.Add(settings);

if (sm_Ports.GetSize() < nWin32Ports)
    if (GetPort(0x378, settings))
        sm_Ports.Add(settings);

if (sm_Ports.GetSize() < nWin32Ports)
    if (GetPort(0x278, settings))
        sm_Ports.Add(settings);

if (sm_Ports.GetSize() == 0)
    TRACE(_T("Could not detect any parallel ports on this machine\n"));

//Don't forget to close the 3 SDK handles we had open
CloseHandle(hPort3);
CloseHandle(hPort2);
CloseHandle(hPort1);
}
}
CParallelPort::~CParallelPort()
{
    //decrement the reference count and
    //free the DriverLINX pointers if necessary
    --sm_nRefCount;
    if (sm_nRefCount == 0)
        DeInitializeDriverLINX();

    Close();
}
IMPLEMENT_DYNAMIC(CParallelPort, CObject)

#ifdef _DEBUG
void CParallelPort::Dump(CDumpContext& dc) const
{
    CObject::Dump(dc);

    dc << _T("m_hPort = ") << m_hPort << _T("\n");
}
#endif

void CParallelPort::Open(int nPort)
{
    //Call Close just in case we already have the port open
    Close();

    m_nPortIndex = nPort - 1;
    if (m_nPortIndex < sm_Ports.GetSize())
    {
        //Cache the base address of the port for performance reasons
        m_nBaseAddress = sm_Ports.ElementAt(m_nPortIndex).m_nBaseAddress;

        //Call CreateFile to open up the parallel port. This prevents other apps
        //causing problems when we do the Port IO directly.
        CString sPort;
        sPort.Format(_T("\\\\.\\LPT%d"), nPort);
        m_hPort = CreateFile(sPort, GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING, 0,
        NULL);
    }
}

```



```

if (m_hPort == INVALID_HANDLE_VALUE)
{
    TRACE(_T("Failed to open parallel port, LPT%d\n"), nPort);
    Close();
    AfxThrowParallelException();
}
}
else
{
    TRACE(_T("Could not find the parallel port, LPT%d\n"), nPort);
    Close();
    AfxThrowParallelException(ERROR_FILE_NOT_FOUND);
}
}

BOOL CParallelPort::IsOpen() const
{
    return (m_hPort != INVALID_HANDLE_VALUE);
}

void CParallelPort::Close()
{
    if (IsOpen())
    {
        BOOL bSuccess = CloseHandle(m_hPort);
        m_hPort = INVALID_HANDLE_VALUE;
        m_nPortIndex = -1;
        m_nBaseAddress = 0;
        if (!bSuccess)
            TRACE(_T("Failed to close the parallel port, GetLastError:%d\n"), GetLastError());
    }
}

void CParallelPort::SetECPPMode(CParallelPortSettings::ECPPortMode mode)
{
    ASSERT(IsOpen()); //Port must be open
    CParallelPortSettings& settings = sm_Ports.ElementAt(m_nPortIndex);
    ASSERT(settings.m_Type == CParallelPortSettings::ParallelTypeECP); //Must be an ECP port
    ASSERT(mode != CParallelPortSettings::ECPPortModeUndefined);

    unsigned short nEcrAddress = (unsigned short)(m_nBaseAddress + 0x402);

    //Read the ECR & clear bits 5, 6 & 7
    int nEcrData = _inp(nEcrAddress) & 0x1F;

    //Write the selected value to bits 5, 6 & 7
    switch (mode)
    {
        case CParallelPortSettings::ECPPortModeSPP:      nEcrData |= (0 << 5); break;
        case CParallelPortSettings::ECPPortModePS2:      nEcrData |= (1 << 5); break;
        case CParallelPortSettings::ECPPortModeFastCentronics: nEcrData |= (2 << 5); break;
        case CParallelPortSettings::ECPPortModeECP:      nEcrData |= (3 << 5); break;
        case CParallelPortSettings::ECPPortModeEPP:      nEcrData |= (4 << 5); break;
        case CParallelPortSettings::ECPPortModeTest:     nEcrData |= (6 << 5); break;
        case CParallelPortSettings::ECPPortModeConfiguration: nEcrData |= (7 << 5); break;
        default: ASSERT(FALSE);                          break;
    }
    _outp(nEcrAddress, nEcrData);

    //Update the value in our cached array

```

```

    settings.m_ECPMode = mode;
}

CParallelPortSettings::ECPPortMode CParallelPort::GetECPMode()
{
    ASSERT(IsOpen()); //Port must be open
    CParallelPortSettings& settings = sm_Ports.ElementAt(m_nPortIndex);
    ASSERT(settings.m_Type == CParallelPortSettings::ParallelTypeECP); //Must be an ECP port

    CParallelPortSettings::ECPPortMode t = ReadECPMode(settings.m_nBaseAddress);
    ASSERT(t == settings.m_ECPMode);

    return settings.m_ECPMode;
}

CParallelPortSettings::ECPPortMode CParallelPort::ReadECPMode(unsigned short nBaseAddress)
{
    CParallelPortSettings::ECPPortMode mode = CParallelPortSettings::ECPPortModeUndefined;
    int nEcrData = _inp((unsigned short)(nBaseAddress+0x402));
    nEcrData = (nEcrData & 0xE0) >> 5;
    switch (nEcrData)
    {
        case 0: mode = CParallelPortSettings::ECPPortModeSPP;          break;
        case 1: mode = CParallelPortSettings::ECPPortModePS2;        break;
        case 2: mode = CParallelPortSettings::ECPPortModeFastCentronics; break;
        case 3: mode = CParallelPortSettings::ECPPortModeECP;        break;
        case 4: mode = CParallelPortSettings::ECPPortModeEPP;        break;
        case 6: mode = CParallelPortSettings::ECPPortModeTest;        break;
        case 7: mode = CParallelPortSettings::ECPPortModeConfiguration; break;
        default: break;
    }
    return mode;
}

BOOL CParallelPort::GetPort(unsigned short nBaseAddress, CParallelPortSettings& settings)
{
    BOOL bSuccess = FALSE;

    //First try to detect an ECP port
    if (GetECPPort(nBaseAddress))
    {
        settings.m_nBaseAddress = nBaseAddress;
        settings.m_Type = CParallelPortSettings::ParallelTypeECP;
        settings.m_ECPMode = ReadECPMode(nBaseAddress);
        bSuccess = TRUE;
    }
    else
    {
        //If its not an ECP, look for an EPP.
        //If the baseaddress is 3BCh, skip the EPP test.
        //EPPs aren't allowed at 3BCh due to possible conflicts
        //with video memory
        BOOL bFoundEPP = FALSE;

        if (nBaseAddress != 0x3BC)
        {
            bFoundEPP = GetEPPPort(nBaseAddress);
            if (bFoundEPP)
            {
                settings.m_nBaseAddress = nBaseAddress;
            }
        }
    }
}

```

```

        settings.m_Type = CParallelPortSettings::ParallelTypeEPP;
        settings.m_ECPMode = CParallelPortSettings::ECPModeUndefined;
        bSuccess = TRUE;
    }
}

if (!bFoundEPP)
{
    //If its not an EPP, look for an SPP.
    if (GetSPPPort(nBaseAddress))
    {
        //Test for a PS/2 port only if the SPP exists.
        //because if the port doesn't exist, it will
        //pass the PS/2 test!
        if (GetPS2Port(nBaseAddress))
            settings.m_Type = CParallelPortSettings::ParallelTypePS2;
        else
            settings.m_Type = CParallelPortSettings::ParallelTypeSPP;

        settings.m_nBaseAddress = nBaseAddress;
        settings.m_ECPMode = CParallelPortSettings::ECPModeUndefined;
        bSuccess = TRUE;
    }
}

if (!bSuccess)
{
    settings.m_nBaseAddress = 0;
    settings.m_Type = CParallelPortSettings::ParallelTypeUndefined;
    settings.m_ECPMode = CParallelPortSettings::ECPModeUndefined;
}

return bSuccess;
}

BOOL CParallelPort::GetECPPort(unsigned short nBaseAddress)
{
    //If the ECP is idle and the FIFO empty,
    //in the ECP's Ecp (at base address+402h),
    //bit 1 (Fifo full)=0, and bit 0 (Fifo empty)=1.
    //The first test is to see if these bits differ from the
    //corresponding bits in the control port (at base address+2).
    //If so a further test is to write 34h to the Ecr,
    //then read it back. Bit 1 is read/write and bit 0 is read-only.
    //If the value read is 35h, the port is an ECP.
    BOOL bSuccess = FALSE;

    unsigned short nEcrAddress = (unsigned short)(nBaseAddress+0x402);
    int nEcrData = _inp(nEcrAddress);

    //Read bits 0 and 1 and control port bit 1
    int nEcrBit0 = nEcrData & 0x1;
    int nEcrBit1 = (nEcrData & 0x2) >> 1;
    int nControlBit1 = (ReadControl(nBaseAddress) & 0x2) >> 1;

    if (nEcrBit0 == 1 && nEcrBit1 == 0)
    {
        //Compare control bit 1 to ECR bit 1
        //Toggle the control bit if necessary
    }
}

```

```

//to be sure the two registers are different.
if (nControlBit1 == 0)
{
    WriteControl(nBaseAddress, 0xF);
    nControlBit1 = (ReadControl(nBaseAddress) & 0x2) >> 1;
}

if (nEcrBit1 != nControlBit1)
{
    int nOriginalEcrData = nEcrData;
    _outp(nEcrAddress, 0x34);
    if (_inp(nEcrAddress) == 0x35)
        bSuccess = TRUE;

    //Restore the ECR to its original value
    _outp(nEcrAddress, nOriginalEcrData);
}
}

return bSuccess;
}

BOOL CParallelPort::GetEPPPort(unsigned short nBaseAddress)
{
    //Write to an EPP register, then read it back.
    //If the read matches the write, it's probably an EPP
    BOOL bSuccess = FALSE;

    //Use nEppAddressPort for testing, SPP's, ECP's and PS/2 ports
    //do not have this register
    unsigned short nEppAddressPort = (unsigned short) (nBaseAddress+3);
    _outp(nEppAddressPort, 0x55);

    //Clear the timeout bit after each EPP operation
    int nTimeoutBit = GetEPPTimeoutBit(nBaseAddress);
    int nRead = _inp(nEppAddressPort);
    nTimeoutBit = GetEPPTimeoutBit(nBaseAddress);
    if (nRead == 0x55)
    {
        _outp(nEppAddressPort, 0xAA);
        nTimeoutBit = GetEPPTimeoutBit(nBaseAddress);
        nRead = _inp(nEppAddressPort);
        nTimeoutBit = GetEPPTimeoutBit(nBaseAddress);
        if (nRead == 0xAA)
            bSuccess = TRUE;
    }

    return bSuccess;
}

int CParallelPort::GetEPPTimeoutBit(unsigned short nBaseAddress)
{
    //Reads and clears the EPP timeout bit (Status port bit 0)
    //Should be done after each EPP operation.
    //The method for clearing the bit varies, so try 3 ways
    //1. Write 1 to status port bit 0
    //2. Write 0 to status port bit 1
    //3. Read the status port again
    int nReturn = (ReadStatus(nBaseAddress) & 0x1);
    unsigned short nStatusPortAddress = (unsigned short)(nBaseAddress+1);

```

```

    _outp(nStatusPortAddress, 1);
    _outp(nStatusPortAddress, 0);
    nReturn = (ReadStatus(nBaseAddress) & 0x1);
    return nReturn;
}

```

```

BOOL CParallelPort::GetPS2Port(unsigned short nBaseAddress)
{
    //First try to tristate (disable) the data outputs by
    //setting bit 5 of the control port. Then write 2 values
    //to the data port and read each one back. If the values
    //match, the data outputs are not disabled and the port
    //is not bidirectional. If the values don't match, the data
    //outputs are disabled and the port is bidirectional
    BOOL bSuccess = FALSE;

    //Set control port bit 5
    WriteControl(nBaseAddress, 0x2F);

    //Write the first byte and read it back
    WriteData(nBaseAddress, 0x55);

    //If it doesn't match, the port is bidirectional
    if (ReadData(nBaseAddress) == 0x55)
    {
        WriteData(nBaseAddress, 0xAA);

        //If it doesn't match, the port is bidirectional
        if (ReadData(nBaseAddress) != 0xAA)
            bSuccess = TRUE;
    }
    else
        bSuccess = TRUE;

    //Reset control port bit 5
    WriteControl(nBaseAddress, 0xF);

    return bSuccess;
}

```

```

BOOL CParallelPort::GetSPPPort(unsigned short nBaseAddress)
{
    //Write two bytes and read them back. If the
    //reads matches the writes, the port exists
    BOOL bSuccess = FALSE;

    //Be sure that control port bit 5 = 0 (Data outputs enabled)
    WriteControl(nBaseAddress, 0xF);

    //Perform the first write
    WriteData(nBaseAddress, 0x55);
    if (ReadData(nBaseAddress) == 0x55)
    {
        WriteData(nBaseAddress, 0xAA);
        bSuccess = (ReadData(nBaseAddress) == 0xAA);
    }
    return bSuccess;
}

```

```

void CParallelPort::WriteControl(unsigned short nBaseAddress, int nData)

```

```

{
    //The control port is at offset nBaseAddress + 2.
    //Bits 0, 1 & 3 need to be inverted
    _outp((unsigned short)(nBaseAddress+2), nData ^ 0xB);
}

int CParallelPort::ReadControl(unsigned short nBaseAddress)
{
    //The control port is at offset nBaseAddress + 2.
    //Bits 0, 1 & 3 need to be inverted
    return (_inp((unsigned short)(nBaseAddress+2)) ^ 0xB);
}

void CParallelPort::WriteData(unsigned short nBaseAddress, int nData)
{
    //The data port is at offset nBaseAddress.
    _outp(nBaseAddress, nData);
}

int CParallelPort::ReadData(unsigned short nBaseAddress)
{
    //The data port is at offset nBaseAddress.
    return _inp(nBaseAddress);
}

int CParallelPort::ReadStatus(unsigned short nBaseAddress)
{
    //The status port is at offset nBaseAddress + 1.
    //Bit 7 need to be inverted
    return (_inp((unsigned short)(nBaseAddress+1)) ^ 0x80);
}

void CParallelPort::WriteControl(int nData)
{
    ASSERT(IsOpen()); //Port must be open
    WriteControl(m_nBaseAddress, nData);
}

int CParallelPort::ReadControl()
{
    ASSERT(IsOpen()); //Port must be open
    return ReadControl(m_nBaseAddress);
}

void CParallelPort::WriteData(int nData)
{
    ASSERT(IsOpen()); //Port must be open
    WriteData(m_nBaseAddress, nData);
}

int CParallelPort::ReadData()
{
    ASSERT(IsOpen()); //Port must be open
    return ReadData(m_nBaseAddress);
}

int CParallelPort::ReadStatus()
{
    ASSERT(IsOpen()); //Port must be open
    return ReadStatus(m_nBaseAddress);
}

```

```

}

BOOL CParallelPort::ReadByteUsingNibbleMode(BYTE& byData)
{
    ASSERT(IsOpen()); //Port must be open

    //Read a byte of data at the status port, in 2 nibbles

    //When S6 = 0, set D3 to 0
    int S6;
    DWORD dwStartTicks = GetTickCount();
    do
    {
        S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 1);
    WriteData(m_nBaseAddress, 0);

    //When the peripheral responds by setting S6=1, Set D3=1
    //nLowNibble then holds 4 bits of data
    int nLowNibble;
    do
    {
        nLowNibble = ReadStatus(m_nBaseAddress);
        S6 = (nLowNibble & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 0);
    WriteData(m_nBaseAddress, 8);

    //When S6 = 0, set D3 to 0
    do
    {
        S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 1);
    WriteData(m_nBaseAddress, 0);

    //When the peripheral responds by setting S6=1, Set D3=1
    //nHighNibble then holds 4 bits of data
    int nHighNibble;
    do
    {
        nHighNibble = ReadStatus(m_nBaseAddress);
        S6 = (nHighNibble & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 0);
    WriteData(m_nBaseAddress, 0);
}

```

```

    }
    while (S6 == 0);
    WriteData(m_nBaseAddress, 8);

    //Recombine the two nibbles back into the byte
    byData = (BYTE) (((nLowNibble & 0x8) >> 3) +
        ((nLowNibble & 0x10) >> 3) +
        ((nLowNibble & 0x20) >> 3) +
        ((nLowNibble & 0x80) >> 4) +
        ((nHighNibble & 0x8) << 1) +
        ((nHighNibble & 0x10) << 1) +
        ((nHighNibble & 0x20) << 1) +
        ((nHighNibble & 0x80)));

    return TRUE;
}

BOOL CParallelPort::WriteByteUsingNibbleMode(BYTE byData)
{
    ASSERT(IsOpen()); //Port must be open

    //Write a byte to the data port, in 2 nibbles.
    //The remote system reads the data at its status port.
    //The data bits are D0, D1, D2 and D4.
    //D3 is the strobe
    int nLowNibble = byData & 0x07;
    nLowNibble |= (byData & 0x08) << 1;
    int nHighNibble = (byData & 0x70) >> 4;
    nHighNibble |= (byData & 0x80) >> 3;

    //When S6=1 (not busy), write the low nibble and set D3=0.
    int S6;
    DWORD dwStartTicks = GetTickCount();
    do
    {
        S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 0);
    WriteData(m_nBaseAddress, nLowNibble);

    //When the peripheral responds by setting S6=0, Set D3=1.
    do
    {
        S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

        //Check the timeout has not elapsed
        if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
            return FALSE;
    }
    while (S6 == 1);
    WriteData(m_nBaseAddress, nLowNibble | 0x8);

    //When S6=1, write the high nibble and set D3=0
    do
    {
        S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

```



```

    //Check the timeout has not elapsed
    if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
        return FALSE;
}
while (S6 == 0);
WriteData(m_nBaseAddress, nHighNibble);

//When the peripheral responds by setting S6=0, Set D3=1.
do
{
    S6 = (ReadStatus(m_nBaseAddress) & 0x40) >> 6;

    //Check the timeout has not elapsed
    if ((GetTickCount() - dwStartTicks) > m_dwTimeout)
        return FALSE;
}
while (S6 == 1);
WriteData(m_nBaseAddress, nHighNibble | 0x8);

return TRUE;
}

BOOL CParallelPort::ReadUsingNibbleMode(void* lpBuf, DWORD dwBytes)
{
    //Validate the parameters
    ASSERT(lpBuf);

    //Read each byte into lpBuf
    BYTE* lpByteBuf = (BYTE*) lpBuf;
    for (DWORD i=0; i<dwBytes; i++)
    {
        if (!ReadByteUsingNibbleMode(lpByteBuf[i]))
            return FALSE;
    }
    return TRUE;
}

BOOL CParallelPort::WriteUsingNibbleMode(const void* lpBuf, DWORD dwBytes)
{
    //Validate the parameters
    ASSERT(lpBuf);

    //Write each byte from lpBuf
    const BYTE* lpByteBuf = (const BYTE*) lpBuf;
    for (DWORD i=0; i<dwBytes; i++)
    {
        if (!WriteByteUsingNibbleMode(lpByteBuf[i]))
            return FALSE;
    }
    return TRUE;
}

int CParallelPort::_inp(unsigned short port)
{
    if (sm_bRunningOnNT)
    {
        ASSERT(sm_lpfnDIPortReadUchar);
        return sm_lpfnDIPortReadUchar(port);
    }
}

```

```

    else
        return ::_inp(port);
}

int CParallelPort::_outp(unsigned short port, int databyte)
{
    if (sm_bRunningOnNT)
    {
        ASSERT(sm_lpfndlPortWriteUchar);
        sm_lpfndlPortWriteUchar(port, (UCHAR) databyte);
        return databyte;
    }
    else
        return ::_outp(port, databyte);
}

BOOL CParallelPort::InitializeDriverLINX()
{
    //Load up the DriverLINX dll and get the function pointers we are interested in
    ASSERT(sm_hDLINX == NULL);
    BOOL bSuccess = FALSE;
    sm_hDLINX = LoadLibrary(_T("DLPORTIO.DLL"));
    if (sm_hDLINX)
    {
        //Get the function pointers
        sm_lpfndlPortReadUchar = (LPDLPORTREADPORTUCHAR) GetProcAddress(sm_hDLINX,
            "DlPortReadPortUchar");
        sm_lpfndlPortWriteUchar = (LPDLPORTWRITEPORTUCHAR) GetProcAddress(sm_hDLINX,
            "DlPortWritePortUchar");

        //If any of the functions are not installed then fail the load
        if (sm_lpfndlPortReadUchar == NULL ||
            sm_lpfndlPortWriteUchar == NULL)
        {
            TRACE(_T("Failed to get one of the function pointer`s in the DriverLINX dll\n"));
            DeInitializeDriverLINX();
        }
        else
            bSuccess = TRUE;
    }
    else
        TRACE(_T("Could not find the load the DriverLINX dll, please ensure DriverLINX driver is installed\n"));

    return bSuccess;
}

void CParallelPort::DeInitializeDriverLINX()
{
    if (sm_hDLINX)
    {
        //Unload the dll and reset the function pointers to NULL
        FreeLibrary(sm_hDLINX);
        sm_hDLINX = NULL;
        sm_lpfndlPortReadUchar = NULL;
        sm_lpfndlPortWriteUchar = NULL;
    }
}

BOOL CParallelPort::PortPresent(int nPort)
{

```

```

    BOOL bSuccess = FALSE;

    //Try to open the port
    CString sPort;
    sPort.Format(_T("\\\\.\\LPT%d"), nPort);
    HANDLE hPort = ::CreateFile(sPort, GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING,
        0, NULL);
    if (hPort != INVALID_HANDLE_VALUE)
    {
        bSuccess = TRUE;

        //Close the port, now that we don't need it anymore
        ::CloseHandle(hPort);
    }
    else
    {
        //Determine if the port exists based on the error code.
        DWORD dwError = GetLastError();

        //Check to see if the error was because some other app had the port open or a general failure
        if (dwError == ERROR_ACCESS_DENIED || dwError == ERROR_GEN_FAILURE)
            bSuccess = TRUE;
    }

    return bSuccess;
}

BOOL CParallelPort::RunningOnNT()
{
    OSVERSIONINFO osv;
    memset(&osv, 0, sizeof(OSVERSIONINFO));
    osv.dwOSVersionInfoSize = sizeof(OSVERSIONINFO);
    VERIFY(GetVersionEx(&osv));
    return (osv.dwPlatformId == VER_PLATFORM_WIN32_NT);
}

CParallelPortSettings CParallelPort::GetSettings() const
{
    ASSERT(IsOpen()); //Port must be open
    return sm_Ports.GetAt(m_nPortIndex);
}

////////// The file transfer class
CParallelPortFileTransfer::CParallelPortFileTransfer(CParallelPort* pPort)
{
    m_pPort = NULL;
    ASSERT(pPort != NULL);
    ASSERT(pPort->IsOpen());

    //Ensure the Port is set to SPP mode if it is an ECP port
    CParallelPortSettings settings = pPort->GetSettings();
    if (settings.m_Type == CParallelPortSettings::ParallelTypeECP)
        pPort->SetECPMode(CParallelPortSettings::ECPModeSPP);

    //Initialize D3 (strobe) to 1
    pPort->WriteData(0x8);

    //Wait for the opposite end to set D3=1 (not busy)
    int S6;
    DWORD dwStartTicks = GetTickCount();

```

```

do
{
    S6 = (pPort->ReadStatus() & 0x40) >> 6;

    //Check the timeout has not elapsed
    if ((GetTickCount() - dwStartTicks) > pPort->GetTimeout())
    {
        TRACE(_T("Could not setup file transfer, other end busy\n"));
        AfxThrowParallelException(ERROR_TIMEOUT);
    }
}
while (S6 == 0);

//Store away the port pointer for use in other class methods
m_pPort = pPort;
}

CParallelPortFileTransfer::~CParallelPortFileTransfer()
{
    m_pPort = NULL;
}

void CParallelPortFileTransfer::SendFile(const CString& sLocalFile)
{
    ASSERT(m_pPort); //Initialize failed

    //For correct operation of the T2A macro, see MFC Tech Note 59
    USES_CONVERSION;

    //First try to open the file to send
    CFile file;
    if (file.Open(sLocalFile, CFile::modeRead | CFile::shareDenyWrite))
    {
        //first send the filename (excluding the path) (size if _MAX_PATH)
        TCHAR pszPath[_MAX_PATH];
        TCHAR pszFname[_MAX_FNAME];
        TCHAR pszExt[_MAX_EXT];
        _tsplitpath(sLocalFile, NULL, NULL, pszFname, pszExt);
        _tmakepath(pszPath, NULL, NULL, pszFname, pszExt);

        //Now need to convert the filename to ASCII so that a UNICODE
        //client can take to an ASCII server
        char pszAsciiPath[_MAX_PATH];
        char* pszTemp = T2A(pszPath);
        ZeroMemory(pszAsciiPath, _MAX_PATH);
        strcpy(pszAsciiPath, pszTemp);

        //Do the actual send of the filename
        if (!m_pPort->WriteUsingNibbleMode(pszAsciiPath, _MAX_PATH))
        {
            TRACE(_T("Failed to send the filename\n"));
            AfxThrowParallelException(ERROR_TIMEOUT);
        }

        try
        {
            //Then send the size of the file (as 4 bytes).
            DWORD dwLength = file.GetLength();
            if (!m_pPort->WriteUsingNibbleMode(&dwLength, sizeof(DWORD)))
            {

```

```

        TRACE(_T("Failed to send the file length\n"));
        AfxThrowParallelException(ERROR_TIMEOUT);
    }

    //Then the actual file contents ifself (4k at a time)
    BYTE byData[4096];
    int nBytesRead;
    do
    {
        nBytesRead = file.Read(byData, 4096);
        if (!m_pPort->WriteUsingNibbleMode(byData, nBytesRead))
        {
            TRACE(_T("Failed to send the file contents\n"));
            AfxThrowParallelException(ERROR_TIMEOUT);
        }
    }
    while (nBytesRead);
}
catch(CFileException* pEx)
{
    TRACE(_T("An error occurred reading from the file to send\n"));
    delete pEx;
    AfxThrowParallelException();
}
}
else
{
    TRACE(_T("Could not open the file to send, %s\n"), sLocalFile);
    AfxThrowParallelException();
}
}

void CParallelPortFileTransfer::ReceiveFile(const CString& sLocalFile)
{
    //For correct operation of the T2A macro, see MFC Tech Note 59
    USES_CONVERSION;

    //First try to open the file to send
    CFile file;
    if (file.Open(sLocalFile, CFile::modeCreate | CFile::modeWrite | CFile::shareDenyWrite))
    {
        //Receive the filename
        char pszAsciiPath[_MAX_PATH];
        if (!m_pPort->ReadUsingNibbleMode(pszAsciiPath, _MAX_PATH))
        {
            TRACE(_T("Failed to receive the filename\n"));

            //Delete the local file
            file.Close();
            DeleteFile(sLocalFile);

            //Throw the exception
            AfxThrowParallelException(ERROR_TIMEOUT);
        }
    }

    try
    {
        //Receive the size of the file (as 4 bytes).
        DWORD dwLength;
        if (!m_pPort->ReadUsingNibbleMode(&dwLength, sizeof(DWORD)))
    
```

```

{
    TRACE(_T("Failed to receive the file length\n"));

    //Delete the local file
    file.Close();
    DeleteFile(sLocalFile);

    //Throw the exception
    AfxThrowParallelException(ERROR_TIMEOUT);
}

//Then the actual file contents ifself (4k at a time)
BYTE byData[4096];
DWORD dwBytesLeft = dwLength;
do
{
    DWORD dwBytesToRead;
    if (dwBytesLeft > 4096)
        dwBytesToRead = 4096;
    else
        dwBytesToRead = dwBytesLeft;

    if (!m_pPort->ReadUsingNibbleMode(byData, dwBytesToRead))
    {
        TRACE(_T("Failed to receive the file contents\n"));

        //Delete the local file
        file.Close();
        DeleteFile(sLocalFile);

        //Throw the exception
        AfxThrowParallelException(ERROR_TIMEOUT);
    }

    file.Write(byData, dwBytesToRead);
    dwBytesLeft -= dwBytesToRead;
}
while (dwBytesLeft);
}
catch(CFileException* pEx)
{
    TRACE(_T("An error occurred writing the file which was being received\n"));
    delete pEx;
    AfxThrowParallelException();
}
}
else
{
    TRACE(_T("Could not open the file to receive into, %s\n"), sLocalFile);
    AfxThrowParallelException();
}
}

```

```

void CParallelPort::SetECPPMode2(int val)
{
    CParallelPortSettings::ECPPortMode mode;
    switch(val){
    case 1:
        mode = CParallelPortSettings::ECPPModeECP;

```

```

        break;
    case 2:
        mode = CParallelPortSettings::ECPModePS2;
        break;
    case 3:
        mode = CParallelPortSettings::ECPModeSPP;
        break;
    }
    ASSERT(IsOpen()); //Port must be open
    CParallelPortSettings& settings = sm_Ports.ElementAt(m_nPortIndex);
    ASSERT(settings.m_Type == CParallelPortSettings::ParallelTypeECP); //Must be an ECP port
    ASSERT(mode != CParallelPortSettings::ECPModeUndefined);

    unsigned short nEcrAddress = (unsigned short)(m_nBaseAddress + 0x402);

    //Read the ECR & clear bits 5, 6 & 7
    int nEcrData = _inp(nEcrAddress) & 0x1F;

    //Write the selected value to bits 5, 6 & 7
    switch (mode)
    {
        case CParallelPortSettings::ECPModeSPP:         nEcrData |= (0 << 5); break;
        case CParallelPortSettings::ECPModePS2:         nEcrData |= (1 << 5); break;
        case CParallelPortSettings::ECPModeFastCentronics: nEcrData |= (2 << 5); break;
        case CParallelPortSettings::ECPModeECP:         nEcrData |= (3 << 5); break;
        case CParallelPortSettings::ECPModeEPP:         nEcrData |= (4 << 5); break;
        case CParallelPortSettings::ECPModeTest:        nEcrData |= (6 << 5); break;
        case CParallelPortSettings::ECPModeConfiguration: nEcrData |= (7 << 5); break;
        default: ASSERT(FALSE);                         break;
    }
    _outp(nEcrAddress, nEcrData);

    //Update the value in our cached array
    settings.m_ECPMode = mode;
}

```

```

/*
Module : PARALLEL.H
Purpose: Declaration for an MFC class to encapsulate parallel ports
Created: PJN / 28-12-1999
History: None
Copyright (c) 1999 - 2000 by PJ Naughter.
All rights reserved.
*/

////////// Macros / Structs etc //////////

#ifndef __PARALLEL_H__
#define __PARALLEL_H__
#ifndef __AFXTEMPL_H__
#pragma message("To avoid this message, put afxtempl.h in your pre compiled header (normally stdafx.h)")
#include <afxtempl.h>
#endif
////////// Classes //////////

///// Parallel port exception class //////////

void AfxThrowParallelException(DWORD dwError = 0);

class CParallelException : public CException
{
public:
//Constructors / Destructors
    CParallelException(DWORD dwError);
    ~CParallelException();

//Methods
#ifdef _DEBUG
    virtual void Dump(CDumpContext& dc) const;
#endif
    virtual BOOL GetErrorMessage(LPTSTR lpstrError, UINT nMaxError, PUINT pnHelpContext = NULL);
    CString GetErrorMessage();

//Data members
    DWORD m_dwError;

protected:
    DECLARE_DYNAMIC(CParallelException)
};

///// The class used to hold a parallel ports settings //////////

class CParallelPortSettings
{
public:
//Typedefs / Defines
    enum ParallelPortType
    {
        ParallelTypeUndefined,
        ParallelTypeSPP,
        ParallelTypePS2,
        ParallelTypeEPP,
        ParallelTypeECP,
    };

    enum ECPPortMode

```



```

{
    ECPModeUndefined,
    ECPModeSPP,
    ECPModePS2,
    ECPModeFastCentronics,
    ECPModeECP,
    ECPModeEPP,
    ECPModeTest,
    ECPModeConfiguration
};

//Constructors / Destructors
CParallelPortSettings();
CParallelPortSettings(const CParallelPortSettings& state);

//Standard methods
CParallelPortSettings& operator=(const CParallelPortSettings& state);

//Member variables
unsigned short  m_nBaseAddress; //The base address of the parallel port e.g. 0x278
ParallelPortType m_Type;        //The port type
ECPPortMode     m_ECPMode;      //The ECP mode if the type is ECP
};

//// The actual parallel port class //////////////////////////////////////

class CParallelPort : public CObject
{
public:
    void SetECPMode2(int val);
//Constructors / Destructors
    CParallelPort();
    ~CParallelPort();

//Methods
    void          Open(int nPort=1);
    inline BOOL   IsOpen() const;
    void          Close();
    void          SetECPMode(CParallelPortSettings::ECPPortMode mode);
    CParallelPortSettings::ECPPortMode GetECPMode();
    void          WriteControl(int nData);
    int           ReadControl();
    void          WriteData(int nData);
    int           ReadData();
    int           ReadStatus();
    BOOL          ReadByteUsingNibbleMode(BYTE& byData);
    BOOL          WriteByteUsingNibbleMode(BYTE byData);
    inline BOOL   ReadUsingNibbleMode(void* lpBuf, DWORD dwBytes);
    inline BOOL   WriteUsingNibbleMode(const void* lpBuf, DWORD dwBytes);
    void          SetTimeout(DWORD dwTimeout) { m_dwTimeout = dwTimeout; };
    DWORD         GetTimeout() const { return m_dwTimeout; };
    CParallelPortSettings GetSettings() const;

//Static methods
    static BOOL PortPresent(int nPort);
//MFC Diagnostics
#ifdef _DEBUG
    void Dump(CDumpContext& dc) const;
#endif
protected:

```

```

//typedefs
typedef UCHAR (_stdcall DLPORTREADPORTUCHAR)(ULONG Port);
typedef DLPORTREADPORTUCHAR* LPDLPORTREADPORTUCHAR;
typedef VOID (_stdcall DLPORTWRITEPORTUCHAR)(ULONG Port, UCHAR Value);
typedef DLPORTWRITEPORTUCHAR* LPDLPORTWRITEPORTUCHAR;

//Methods
static BOOL      InitializeDriverLINX();
static void      DeInitializeDriverLINX();
static BOOL      RunningOnNT();
static BOOL      GetPort(unsigned short nBaseAddress, CParallelPortSettings& settings);
static BOOL      GetECPPort(unsigned short nBaseAddress);
static BOOL      GetEPPPort(unsigned short nBaseAddress);
static BOOL      GetPS2Port(unsigned short nBaseAddress);
static BOOL      GetSPPPort(unsigned short nBaseAddress);
static int       GetEPPTimeoutBit(unsigned short nBaseAddress);
static CParallelPortSettings::ECPPortMode ReadECPPortMode(unsigned short nBaseAddress);
inline static int _inp(unsigned short port);
inline static int _outp(unsigned short port, int databyte);
inline static void WriteControl(unsigned short nBaseAddress, int nData);
inline static int  ReadControl(unsigned short nBaseAddress);
inline static void WriteData(unsigned short nBaseAddress, int nData);
inline static int  ReadData(unsigned short nBaseAddress);
inline static int  ReadStatus(unsigned short nBaseAddress);

//Member variables
HANDLE      m_hPort;      //SDK handle to lock Win32 access to the port
int          m_nPortIndex; //index of the current port into m_Ports
DWORD        m_dwTimeout; //Timeout in Milliseconds for reads and writes
unsigned short m_nBaseAddress; //The base address of the port

//Static member variables
static BOOL      sm_bRunningOnNT;      //Are we running on NT / Windows 2000
static int       sm_nRefCount;          //Reference count for the class
static HINSTANCE sm_hDLINX;            //handle to the DriverLINX dll.
static LPDLPORTREADPORTUCHAR sm_lpfndlPortReadUchar; //Function pointer for
writing via DriverLINX
static LPDLPORTWRITEPORTUCHAR sm_lpfndlPortWriteUchar; //Function pointer for
reading via DriverLINX
static CArray<CParallelPortSettings, CParallelPortSettings&> sm_Ports; //All the parallel ports
available on this machine
DECLARE_DYNAMIC(CParallelPort)
};

//// A Simple class to transfer and receive files using nibble mode
class CParallelPortFileTransfer
{
public:
//Constructors / Destructors
    CParallelPortFileTransfer(CParallelPort* pPort);
    ~CParallelPortFileTransfer();
//Methods
    void SendFile(const CString& sLocalFile);
    void ReceiveFile(const CString& sLocalFile);

protected:
    CParallelPort* m_pPort;
};
#endif // __PARALLEL_H__

```

```
// stdafx.cpp : source file that includes just the standard includes
// LaddningST7.pch will be the pre-compiled header
// stdafx.obj will contain the pre-compiled type information

#include "stdafx.h"
```

```

//{{NO_DEPENDENCIES}}
// Microsoft Developer Studio generated include file.
// Used by LaddningST7.rc
//
#define IDM_ABOUTBOX            0x0010
#define IDD_ABOUTBOX            100
#define IDS_ABOUTBOX            101
#define IDD_LADDNINGST7_DIALOG  102
#define IDR_MAINFRAME           128
#define IDR_MENU1               129
#define IDC_AmperetimmarRuta    1000
#define IDC_Antalcellerruta     1001
#define IDC_SerienummerRuta     1002
#define IDC_LaddningskurvaCombo 1003
#define IDC_GrundladdartypVoltRuta 1005
#define IDC_KalibreringsdataRuta 1006
#define IDC_LaddartypVoltRuta    1007
#define IDC_HardvaruversionRuta 1008
#define IDC_LasKnapp            1009
#define IDC_SkrivKnapp          1010
#define IDC_GrundladdartypAmpereRuta 1011
#define IDC_LaddartypAmpereRuta 1012
#define IDC_ProgramCombo        1014
#define IDC_VerifieraKnapp      1017
#define IDC_SparaloggfilKnapp   1018
#define IDC_DataRuta15          1020
#define IDC_Skriv16byteKnapp    1021
#define IDC_DataRuta12          1022
#define IDC_AdressRuta          1023
#define IDC_DataRuta14          1024
#define IDC_DataRuta13          1026
#define IDC_TextLaddningskurva  1027
#define IDC_TextAntalceller     1028
#define IDC_TextAmperetimmar    1029
#define IDC_TextProgram         1030
#define IDC_DataRuta8           1031
#define IDC_DataRuta9           1032
#define IDC_DataRuta11          1033
#define IDC_DataRuta10          1034
#define IDC_TextHardvaruversion 1035
#define IDC_DataRuta7           1036
#define IDC_DataRuta4           1037
#define IDC_DataRuta6           1038
#define IDC_DataRuta5           1039
#define IDC_DataRuta0           1040
#define IDC_DataRuta1           1041
#define IDC_DataRuta3           1042
#define IDC_DataRuta2           1043
#define IDC_TextKalibreringsdata 1044
#define IDC_TextSerienummer     1045
#define IDC_TextLaddartyp       1046
#define IDC_TextGrundladdartyp  1047
#define IDC_TextAdress          1048
#define IDC_TextData15till8     1049
#define IDC_TextData7till0      1050
#define IDC_DATETIMEPICKER1     1051
#define IDC_Felmeddelande       1053
#define IDC_Las16byte           1054
#define IDC_ProgramNamnRuta     1055
#define IDC_ProgramRevisionRuta 1056

```

```

#define IDC_TextProgramRevision    1057
#define IDC_TextProgramNamn       1058
#define IDC_TextLaddningskurvaNamn 1059
#define IDC_LaddningskurvaNamnRuta 1060
#define IDC_LaddningskurvaRevisionRuta 1061
#define IDC_TextLaddningskurvaRevision 1062
#define IDC_EDIT1                 1064
#define MenyArkivAvsluta          32771
#define MenyVisaLaddningskurva    32772
#define MenyVisaAntalceller       32773
#define MenyVisaAmperetimmar      32774
#define MenyVisaSerienummer       32775
#define MenyVisaProgram           32776
#define MenyVisaHardvaruversion   32777
#define MenyVisaGrundladdartyp    32778
#define MenyVisaLaddartyp         32779
#define MenyVisaKalibreringsdata  32780
#define MenyHjalpOm               32781
#define MenyVisaDefaultanpassning 32782
#define MenyVisaDefaultTekniker   32783
#define MenyArkivLas              32784
#define MenyArkivSkriv            32785
#define MenyVisaDefaultProduktion 32786
#define ID_ArkivSattKatalog       32788
#define MenyArkivSkrivochSpara    32789

// Next default values for new objects
//
#ifdef APSTUDIO_INVOKED
#ifndef APSTUDIO_READONLY_SYMBOLS
#define _APS_NEXT_RESOURCE_VALUE    132
#define _APS_NEXT_COMMAND_VALUE    32790
#define _APS_NEXT_CONTROL_VALUE    1065
#define _APS_NEXT_SYMED_VALUE      101
#endif
#endif

```

```

// stdafx.h : include file for standard system include files,
// or project specific include files that are used frequently, but
// are changed infrequently
//

#ifndef AFX_STDAFX_H__A449CF28_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_
#define AFX_STDAFX_H__A449CF28_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#define VC_EXTRALEAN // Exclude rarely-used stuff from Windows headers

#include <afxwin.h> // MFC core and standard components
#include <afxext.h> // MFC extensions
#include <afxdisp.h> // MFC Automation classes
#include <afxdtctl.h> // MFC support for Internet Explorer 4 Common Controls
#ifndef _AFX_NO_AFXCMN_SUPPORT
#include <afxcmn.h> // MFC support for Windows Common Controls
#endif // _AFX_NO_AFXCMN_SUPPORT

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif // !defined(AFX_STDAFX_H__A449CF28_60C6_11D6_8F07_FA8BA4D2501F__INCLUDED_)

```



Växjö
universitet

Matematiska och systemtekniska institutionen

351 95 Växjö

tel. 0470-70 80 00, fax 0470-840 04

www.msi.vxu.se